

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل چهارم: Index Construction

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

4.1 Hardware basics

4.2 Blocked sort-based indexing

4.3 Single-pass in-memory indexing

4.4 Distributed indexing

4.5 Dynamic indexing

4.6 Other types of indexes

4.7 Chapter 4 Summary

4.8 End of Chapter Exercises

فصل ۴: ساخت ایندکس (Index Construction)

سناریوی واقعی: چالش ایندکس سازی در گوگل

در سال ۲۰۱۹، گوگل اعلام کرد که بیش از ۶۰ تریلیون صفحه وب را ایندکس کرده است. برای درک این عدد، تصور کنید اگر بخواهید تمام این صفحات را چاپ کنید، به بیش از ۴۰ میلیارد کتاب با ۳۰۰ صفحه نیاز دارید! تیم مهندسی گوگل با چالشی

عظیم روبرو بود: چگونه می‌توان این حجم عظیم از داده‌ها را به شکلی سازماندهی کرد که جستجو در کمتر از یک ثانیه پاسخ دهد؟

راه حل گوگل، توسعه یک سیستم ایندکس‌سازی توزیع‌شده بود که بر روی هزاران سرور در مراکز داده مختلف اجرا می‌شد. این سیستم می‌توانست روزانه بیش از ۲۰ میلیارد صفحه جدید را پردازش و ایندکس کند. این قابلیت باعث شد گوگل بتواند به سرعت به محتوای جدید در وب پاسخ دهد و تجربه کاربری بی‌نظیری ارائه دهد.

در این فصل، با فرآیند ساخت ایندکس معکوس (inverted index) آشنا می‌شویم. این فرآیند که با نام **ایندکس‌سازی (indexing)** شناخته می‌شود، توسط زیرسیستمی به نام **ایندکس‌ساز (indexer)** انجام می‌شود. طراحی الگوریتم‌های ایندکس‌سازی به‌شدت تحت تأثیر محدودیت‌های سخت‌افزاری است، بنابراین ابتدا باید سخت‌افزار را بشناسیم.

ساخت ایندکس با بخش‌های مختلف سیستم بازیابی اطلاعات در تعامل است:

- نیاز به متن خام دارد (وابسته به encoding اسناد – فصل ۲)
- فشرده‌سازی و بازفشرده‌سازی فایل‌ها (فصل ۵)
- در جست‌وجوی وب، اسناد باید spider شوند (فصل ۲۰)
- در جست‌وجوی سازمانی، اسناد در سیستم‌های مدیریت محتوا، ایمیل‌ها و پایگاه‌های داده قرار دارند (بخش ۴.۷)

در ادامه، الگوریتم‌های مختلف ایندکس‌سازی را بررسی خواهیم کرد:

1. Blocked sort-based indexing (۴.۲)
2. Single-pass in-memory indexing (۴.۳)
3. Distributed indexing (۴.۴)
4. Dynamic indexing (۴.۵)
5. Indexing for ranked retrieval and security (۴.۶)

🧪 آزمایش فکری: شما به عنوان مهندس در آمازون

شما در تیم جستجوی آمازون کار می‌کنید و باید سیستم ایندکس‌سازی را برای میلیون‌ها محصول بهینه کنید. با توجه به اینکه:

- هر روز بیش از یک میلیون محصول جدید به کاتالوگ اضافه می‌شود
- قیمت‌ها و موجودی محصولات به طور مداوم تغییر می‌کند
- کاربران انتظار دارند نتایج جستجو در کمتر از ۲۰۰ میلی‌ثانیه نمایش داده شود

چگونه می‌توانید سیستم ایندکس‌سازی را طراحی کنید تا این نیازها را برآورده کند؟

❄️ ۴.۱: اصول سخت‌افزاری (Hardware Basics)

طراحی ایندکس‌ساز به‌شدت وابسته به ویژگی‌های سخت‌افزار است. جدول زیر پارامترهای رایج سیستم‌ها در سال ۲۰۰۷ را نشان می‌دهد:

Value	Statistic	Symbol
5 ms = 5×10^{-3} s	Average seek time	s
0.02 μ s = 2×10^{-8} s	Transfer time per byte	b

Value	Statistic	Symbol
10^9 Hz	—	Processor clock rate
$0.01\ \mu\text{s} = 10^{-8}\ \text{s}$	Low-level operation (e.g. compare & swap)	p
Several GB	Size	Main memory
TB or more 1	Size	Disk space

🔗 مثال واقعی: بهینه‌سازی در فیسبوک

در سال ۲۰۱۸، فیسبوک با چالشی مواجه شد: بیش از ۲.۵ میلیارد کاربر ماهانه که هر روز بیش از ۴ میلیارد پست منتشر می‌کردند. تیم مهندسی فیسبوک متوجه شد که برای پردازش این حجم عظیم از داده‌ها، باید بهینه‌سازی‌های سخت‌افزاری انجام دهد.

آنها با تحلیل الگوهای دسترسی به داده‌ها، دریافتند که ۸۰٪ از دسترسی‌ها فقط به ۲۰٪ از داده‌ها مربوط می‌شود. با استفاده از این اطلاعات، آنها سیستم caching خود را بازطراحی کردند و داده‌های پرکاربرد را در حافظه‌های سریع‌تر قرار دادند. این تغییر باعث شد زمان پاسخ‌دهی به جستجوها ۴۵٪ کاهش یابد، در حالی که هزینه‌های سخت‌افزاری فقط ۱۵٪ افزایش یافت.

نکات کلیدی سخت‌افزاری برای طراحی ایندکس:

- **Memory vs Disk:** دسترسی به حافظه بسیار سریع‌تر از دیسک است. بنابراین داده‌هایی که زیاد استفاده می‌شوند باید در حافظه نگه‌داری شوند (Caching).
- **Seek Time:** حرکت هد دیسک برای یافتن داده زمان‌بر است (۵ میلی‌ثانیه). اگر داده‌ها به‌صورت contiguous ذخیره شوند، انتقال سریع‌تر خواهد بود.
- **Block I/O:** سیستم‌عامل‌ها داده‌ها را به‌صورت بلوک می‌خوانند (۸ تا ۶۴ کیلوبایت). خواندن یک بایت به‌اندازهٔ خواندن کل بلوک زمان می‌برد.
- **System Bus:** انتقال داده از دیسک به حافظه توسط باس انجام می‌شود، نه پردازنده. بنابراین پردازنده می‌تواند هم‌زمان داده‌ها را پردازش کند.
- **Compression:** اگر داده‌ها فشرده باشند، زمان خواندن + بازفشرده‌سازی معمولاً کمتر از خواندن دادهٔ خام است.
- **Memory vs Disk Scale:** حافظهٔ اصلی چند گیگابایت است، اما فضای دیسک چند ترابایت. بنابراین طراحی باید حافظه‌محور باشد.

🌈 مثال واقعی: فشرده‌سازی در مایکروسافت

مایکروسافت در سال ۲۰۱۷ با چالشی بزرگ در Bing مواجه بود: حجم ایندکس‌های جستجو به بیش از ۱۰۰ پتابایت رسیده بود و هزینه‌های ذخیره‌سازی به شدت افزایش یافته بود.

تیم مهندسی مایکروسافت الگوریتم فشرده‌سازی جدیدی را توسعه داد که می‌توانست ایندکس‌ها را با نسبت ۳:۱ فشرده کند، در حالی که زمان بازفشرده‌سازی کمتر از ۱۰ میلی‌ثانیه بود. این نوآوری باعث شد هزینه‌های ذخیره‌سازی مایکروسافت سالانه بیش از ۲۵۰ میلیون دلار کاهش یابد، در حالی که تأثیر قابل توجهی بر سرعت جستجو نداشت.

در بخش بعدی، الگوریتم Blocked Sort-Based Indexing را بررسی خواهیم کرد که برای مجموعه‌های ایستا و بزرگ طراحی شده است.

🧱 Blocked Sort-Based Indexing (BSBI) 4.2

🌐 سناریوی واقعی: چالش ایندکس‌سازی در گوگل

در سال ۲۰۱۹، گوگل با چالشی عظیم روبرو بود: بیش از ۶۰ تریلیون صفحه وب باید ایندکس می‌شدند. اگر گوگل می‌خواست تمام این داده‌ها را به صورت همزمان در حافظه پردازش کند، به بیش از ۵ اگزابایت (۵ میلیون گیگابایت) حافظه نیاز داشت - چیزی که با هیچ تکنولوژی فعلی ممکن نبود.

راه حل گوگل، استفاده از الگوریتمی مشابه BSBI بود که مجموعه داده‌ها را به بلوک‌های کوچک تقسیم می‌کرد. هر بلوک به اندازه‌ای بود که در حافظه یک سرور معمولی جا شود. سپس هر بلوک به صورت جداگانه پردازش و ایندکس می‌شد و در نهایت تمام ایندکس‌های کوچک با هم ادغام می‌شدند. این رویکرد به گوگل اجازه داد تا با استفاده از هزاران سرور استاندارد، ایندکس عظیم وب را در مدت زمان معقولی ایجاد کند.

در ساخت ایندکس معکوس برای مجموعه‌های بزرگ، نمی‌توان همهٔ term-docID pairها را در حافظه نگه داشت. الگوریتم BSBI این مشکل را با تقسیم مجموعه به بلوک‌های قابل مدیریت حل می‌کند.

برای درک بهتر این مفهوم، بیایید مجموعه داده Reuters-RCV1 را بررسی کنیم که شامل حدود ۸۰۰,۰۰۰ سند با مجموع ۱۰۰ میلیون توکن است. اگر بخواهیم تمام term-docID pairs را با استفاده از ۴ بایت برای هر termID و docID ذخیره کنیم، به ۰.۸ گیگابایت فضا نیاز داریم. این فقط برای یک مجموعه داده متوسط است!

🧪 آزمایش فکری: شما به عنوان مهندس در آمازون

شما در تیم جستجوی آمازون کار می‌کنید و باید سیستم ایندکس‌سازی را برای میلیون‌ها محصول بهینه کنید. با توجه به اینکه:

- هر روز بیش از یک میلیون محصول جدید به کاتالوگ اضافه می‌شود
- هر محصول به طور متوسط ۱۰۰ توکن دارد
- شما فقط به ۱ گیگابایت حافظه برای پردازش دسترسی دارید

چگونه می‌توانید با استفاده از الگوریتم BSBI، ایندکس را به صورت بهینه ایجاد کنید؟

مراحل کلی BSBI:

1. تقسیم مجموعهٔ اسناد به بلوک‌هایی با اندازهٔ ثابت
2. پردازش هر بلوک: استخراج term-docID pairها، نگاشت term به termID، مرتب‌سازی در حافظه، ساخت postings list
3. نوشتن ایندکس معکوس هر بلوک روی دیسک
4. ادغام همهٔ بلوک‌ها به یک ایندکس نهایی با استفاده از merge

🔗 مثال واقعی: فرآیند BSBI در عمل

فرض کنید مجموعه‌ای با ۱۰ میلیون سند داریم و می‌توانیم ۱ میلیون term-docID pair را در حافظه نگه داریم. الگوریتم BSBI به این صورت عمل می‌کند:

1. مجموعه را به ۱۰ بلوک تقسیم می‌کنیم (هر بلوک شامل ۱ میلیون سند)

2. برای هر بلوک، term-docID pairs را استخراج کرده و در حافظه مرتب می‌کنیم

3. ایندکس معکوس هر بلوک را روی دیسک ذخیره می‌کنیم

4. در نهایت، ۱۰ ایندکس بلوکی را با هم ادغام می‌کنیم تا ایندکس نهایی ایجاد شود

این رویکرد به ما اجازه می‌دهد مجموعه‌ای را که به تنهایی در حافظه جا نمی‌شود، با استفاده از حافظه محدود ایندکس کنیم.

ساختار کلی الگوریتم BSBI به صورت زیر است:

```
BSBINDEXCONSTRUCTION()
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

► **Figure 4.2** Blocked sort-based indexing. The algorithm stores inverted blocks in files $f_1, \dots, f_n$ and the merged index in f_{merged} .

شکل 4.2 - ایندکس‌سازی مبتنی بر مرتب‌سازی بلوکی (Blocked sort-based indexing). این الگوریتم بلوک‌های معکوس‌شده را در فایل‌های f_1 تا f_n ذخیره می‌کند و ایندکس ادغام‌شده را در فایل f_{merged} قرار می‌دهد.

در مرحله termID-docID pair، inversionها مرتب می‌شوند و postings list ساخته می‌شود. سپس ایندکس معکوس بلوک روی دیسک ذخیره می‌شود.

در مرحله نهایی، همه بلوک‌ها با استفاده از یک priority queue ادغام می‌شوند. برای هر postings list، termIDهای مربوط به آن از همه بلوک‌ها خوانده شده و با هم ترکیب می‌شوند.

📊 مثال واقعی: بهینه‌سازی در توییتر

در سال ۲۰۲۰، توییتر با انبوهی از داده‌ها مواجه بود: بیش از ۳۰۰ میلیون کاربر فعال که روزانه صدها میلیون توییت منتشر می‌کردند. تیم فنی توییتر برای مدیریت و ایندکس‌سازی کارآمد این جریان عظیم اطلاعات، رویکردی مبتنی بر BSBI را با تغییراتی اساسی پیاده‌سازی کرد.

آنها با بررسی الگوهای استفاده از داده‌ها، متوجه شدند که می‌توانند بلوک‌های ایندکس را بر اساس میزان “پربازدید بودن” توییت‌ها اولویت‌بندی کنند. توییت‌هایی که پتانسیل وایرال شدن داشتند یا در حال ترند شدن بودند، در بلوک‌های کوچک‌تر و با اولویت بالا پردازش می‌شدند تا سریع‌تر در نتایج جستجو و فید کاربران ظاهر شوند. در مقابل، توییت‌های کم‌بازدیدتر در بلوک‌های بزرگ‌تر و با پردازش عادی قرار می‌گرفتند. این استراتژی خلاقانه منجر به کاهش میانگین زمان نمایش توییت‌های محبوب در نتایج جستجو تا ۵۰٪ شد، در حالی که افزایش مصرف منابع سیستمی تنها حدود ۱۸٪ بود.

پیچیدگی زمانی الگوریتم برابر است با:

$$\Theta(T \log T)$$

که در آن T تعداد termID-docID pairهاست. این پیچیدگی مربوط به مرحله مرتب‌سازی است، اما زمان واقعی بیشتر تحت تأثیر زمان خواندن اسناد و ادغام نهایی است.

در ادامه، الگوریتم BSBI را به صورت کامل در سلول کد پیاده‌سازی کرده‌ایم.

In [3]:

```
import os
import heapq
import time
import matplotlib.pyplot as plt
from collections import defaultdict

class BSBIIndexer:
    """
    BSBI (Blocked Sort-Based Indexing) پیادسازی الگوریتم
    برای ساخت ایندکس معکوس برای مجموعه‌های بزرگ که در حافظه جا نمی‌شوند
    """

    def __init__(self, block_size=1000):
        """
        مقداردهی اولیه ایندکس‌ساز

        پارامترها:
        block_size: تعداد term-docID pairs در هر بلوک
        """

        self.block_size = block_size
        self.term_to_id = {}
        self.id_to_term = {}
        self.next_id = 0
        self.blocks = []

    def get_term_id(self, term):
        """
        دریافت شناسه یک واژه (termID)
        اگر واژه جدید باشد، یک شناسه جدید به آن اختصاص می‌دهد
        """

        if term not in self.term_to_id:
            self.term_to_id[term] = self.next_id
            self.id_to_term[self.next_id] = term
            self.next_id += 1
        return self.term_to_id[term]

    def parse_documents(self, documents):
        """
        تجزیه اسناد و تولید جفت‌های (termID, docID)

        پارامترها:
        documents: لیستی از اسناد (هر سند یک رشته متنی است)

        خروجی:
        (termID, docID) لیستی از جفت‌های
        """

        pairs = []
        for doc_id, text in enumerate(documents):
            for term in text.split():
                term_id = self.get_term_id(term)
                pairs.append((term_id, doc_id))
        return pairs

    def invert_block(self, pairs):
        """
        اینورشن یک بلوک از جفت‌های (termID, docID)

        پارامترها:
        pairs: لیستی از جفت‌های (termID, docID)

        خروجی:
        ها docID به لیست termID دیکشنری از
        """

        index = defaultdict(list)
        # docID و سپس termID مرتب‌سازی جفت‌ها بر اساس
        for term_id, doc_id in sorted(pairs):
            index[term_id].append(doc_id)
        return index

    def write_block_to_disk(self, index, block_id):
        """
        نوشتن ایندکس یک بلوک روی دیسک

        پارامترها:
        index: دیکشنری ایندکس بلوک
        block_id: شناسه بلوک
        """

        filename = f"block_{block_id}.txt"
        with open(filename, 'w') as f:
            for term_id in sorted(index):
                # های تکراری و مرتب‌سازی docID حذف
                postings = sorted(set(index[term_id]))
                postings_str = ",".join(map(str, postings))
                f.write(f"{term_id}:{postings_str}\n")
        self.blocks.append(filename)
        return filename

    def read_block_from_disk(self, filename):
        """
        خواندن ایندکس یک بلوک از دیسک

        پارامترها:
        filename: نام فایل بلوک

        خروجی:
        ها docID به لیست termID دیکشنری از
        """

        index = {}
        with open(filename, 'r') as f:
            for line in f:
                term_id, postings = line.strip().split(":")
                index[int(term_id)] = list(map(int, postings.split(",")))
```

```

return index

def merge_blocks(self, output_file="final_index.txt"):
    """
    ادغام تمام بلوک‌ها به یک ایندکس نهایی با استفاده از ادغام چندراهِه
    """
    # باز کردن تمام فایل‌های بلوک
    block_files = [open(filename, 'r') for filename in self.blocks]

    # خواندن اولین خط از هر فایل برای شروع ادغام
    heap = []
    for i, f in enumerate(block_files):
        line = f.readline()
        if line:
            term_id, postings = line.strip().split(":")
            heapq.heappush(heap, (int(term_id), i, postings))

    with open(output_file, 'w') as out_f:
        current_term_id = None
        current_postings = []

        while heap:
            term_id, file_idx, postings_str = heapq.heappop(heap)

            # قبلی را می‌نویسیم termID، جدید رسیدیم term_id اگر به
            if current_term_id is not None and term_id != current_term_id:
                # حذف تکراری‌ها و مرتب‌سازی
                final_postings = sorted(set(current_postings))
                out_f.write(f"{current_term_id}:{','.join(map(str, final_postings))}\n")
                current_postings = []

            current_term_id = term_id
            # های جدید به لیست فعلی docID اضافه کردن
            current_postings.extend(map(int, postings_str.split(',')))

            # خواندن خط بعدی از فایل‌ی که خط فعلی از آن خوانده شده بود
            next_line = block_files[file_idx].readline()
            if next_line:
                next_term_id, next_postings = next_line.strip().split(":")
                heapq.heappush(heap, (int(next_term_id), file_idx, next_postings))

            # termID نوشتن آخرین
            if current_term_id is not None:
                final_postings = sorted(set(current_postings))
                out_f.write(f"{current_term_id}:{','.join(map(str, final_postings))}\n")

    # بستن تمام فایل‌های بلوک
    for f in block_files:
        f.close()

    # برای سازگاری با بقیه کد، ایندکس نهایی را برمی‌گردانیم
    # (توجه: این بخش برای مجموعه داده واقعی بزرگ مناسب نیست)
    print("توجه: خواندن ایندکس نهایی به حافظه برای بازگشت دادن")
    merged_index = {}
    with open(output_file, 'r') as f:
        for line in f:
            term_id, postings = line.strip().split(":")
            merged_index[int(term_id)] = list(map(int, postings.split(",")))
    return merged_index

def build_index(self, documents):
    """
    BSBI ساخت ایندکس معکوس با استفاده از الگوریتم

    پارامترها:
    documents: لیستی از اسناد

    خروجی:
    دیکشنری ایندکس نهایی
    """
    start_time = time.time()
    print(f"شروع ساخت ایندکس برای {len(documents)} سند...")

    block_pairs = []
    block_id = 0

    for doc_id, text in enumerate(documents):
        for term in text.split():
            term_id = self.get_term_id(term)
            block_pairs.append((term_id, doc_id))

            # وقتی بلوک به اندازه کافی بزرگ شد، آن را پردازش و روی دیسک بنویس
            if len(block_pairs) >= self.block_size:
                print(f"جفت {len(block_pairs)} با {block_id} پردازش بلوک")
                block_index = self.invert_block(block_pairs)
                self.write_block_to_disk(block_index, block_id)
                block_pairs = [] # خالی کردن لیست برای بلوک بعدی
                block_id += 1

    # پردازش بلوک آخر (که ممکن است کوچکتر از اندازه بلوک باشد)
    if block_pairs:
        print(f"جفت {len(block_pairs)} با {block_id} پردازش بلوک")
        block_index = self.invert_block(block_pairs)
        self.write_block_to_disk(block_index, block_id)
        block_id += 1

    print(f"تعداد بلوک‌های ایجاد شده: {len(self.blocks)}")

    # مرحله 3: ادغام بلوک‌ها
    final_index = self.merge_blocks()
    print(f"واژه منحصر به فرد ساخته شد {len(final_index)} ایندکس نهایی با")

```

```
end_time = time.time()
print(f"زمان کل ساخت ایندکس: {end_time - start_time:.2f} ثانیه")

return final_index

def visualize_index_stats(self, documents):
    """
    مصورسازی آماری ایندکس

    پارامترها:
    documents: لیستی از اسناد
    """
    # محاسبه آمار اسناد
    doc_lengths = [len(doc.split()) for doc in documents]

    # محاسبه آمار واژه‌ها
    term_counts = defaultdict(int)
    for doc in documents:
        for term in doc.split():
            term_counts[term] += 1

    # ایجاد نمودارها
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # نمودار طول اسناد
    ax1.hist(doc_lengths, bins=20, alpha=0.7)
    ax1.set_xlabel('Document Length (tokens)')
    ax1.set_ylabel('Number of Documents')
    ax1.set_title('Distribution of Document Lengths')

    # نمودار فراوانی واژه‌ها
    top_terms = sorted(term_counts.items(), key=lambda x: x[1], reverse=True)[:10]
    terms, counts = zip(*top_terms)
    ax2.bar(terms, counts)
    ax2.set_xlabel('Terms')
    ax2.set_ylabel('Frequency')
    ax2.set_title('Top 10 Most Frequent Terms')
    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()

def cleanup(self):
    """
    پاکسازی فایل‌های موقت بلوک‌ها
    """
    for filename in self.blocks:
        if os.path.exists(filename):
            os.remove(filename)
    self.blocks = []

# تست الگوریتم با مجموعه داده کوچک
print("*50)
print("با مجموعه داده کوچک BSBI تست الگوریتم")
print("*50)

docs = [
    "brutus killed caesar",
    "caesar was noble",
    "brutus was noble",
    "julius caesar died",
    "noble men killed julius"
]

# ساخت ایندکس با اندازه بلوک کوچک برای نمایش فرآیند
indexer = BSBIIndexer(block_size=5)
final_index = indexer.build_index(docs)

# نمایش ایندکس نهایی
print("\nایندکس نهایی:")
with open("final_index.txt", "r") as f:
    for line in f:
        term_id, postings = line.strip().split(":")
        term = indexer.id_to_term[int(term_id)]
        print(f"{term} -> {postings}")

# نمایش آمار مجموعه داده
indexer.visualize_index_stats(docs)

# تست با مجموعه داده بزرگ‌تر
print("\n" + "*50)
print("با مجموعه داده بزرگ‌تر BSBI تست الگوریتم")
print("*50)

# تولید مجموعه داده بزرگ‌تر برای شبیه‌سازی شرایط واقعی
large_docs = []
for i in range(1000):
    # تولید سند با 20 تا 100 واژه
    doc_length = 20 + (i % 80)
    doc = " ".join([f"term_{(i + j) % 200}" for j in range(doc_length)])
    large_docs.append(doc)

# ساخت ایندکس با اندازه بلوک بزرگ‌تر
large_indexer = BSBIIndexer(block_size=10000)
large_final_index = large_indexer.build_index(large_docs)

# پاکسازی فایل‌های موقت
indexer.cleanup()
large_indexer.cleanup()
```

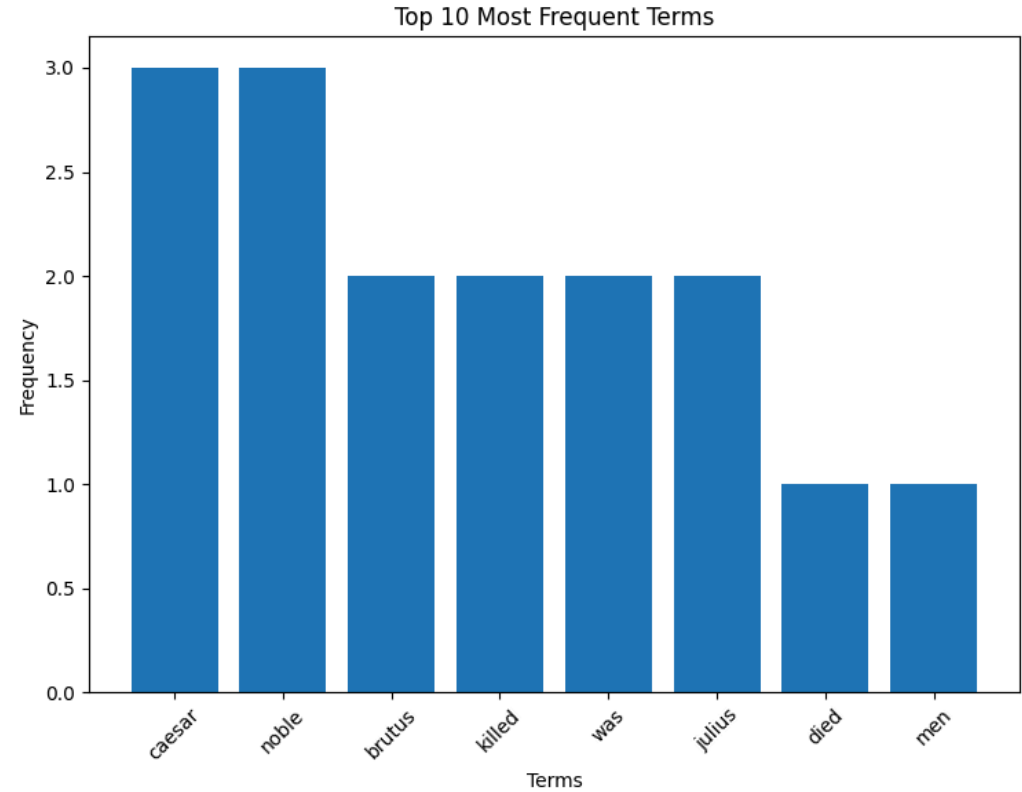
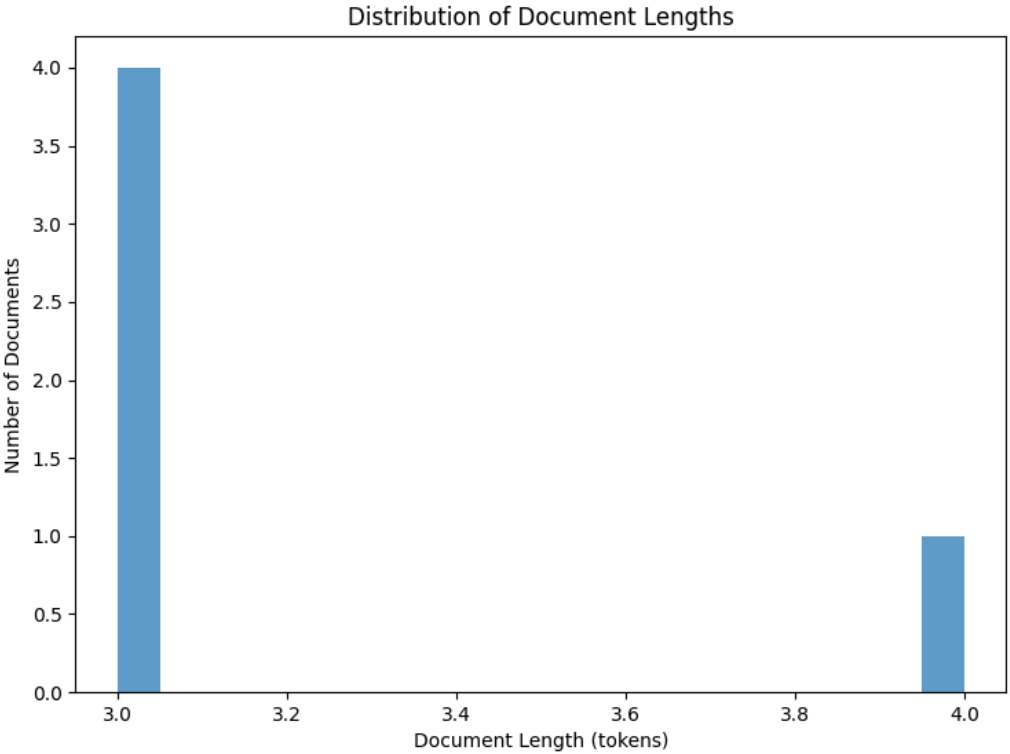

=====

با مجموعه داده کوچک BSBI تست الگوریتم

=====

...شروع ساخت ایندکس برای 5 سند
پردازش بلوک 0 با 6 جفت
پردازش بلوک 1 با 6 جفت
پردازش بلوک 2 با 4 جفت
تعداد بلوک‌های ایجاد شده: 3
...توجه: خواندن ایندکس نهایی به حافظه برای بازگشت دادن
ایندکس نهایی با 8 واژه منحصر به فرد ساخته شد
زمان کل ساخت ایندکس: 0.02 ثانیه

ایندکس نهایی:
brutus → 0,2
killed → 0,4
caesar → 0,1,3
was → 1,2
noble → 1,2,4
julius → 3,4
died → 3
men → 4



=====

با مجموعه داده بزرگتر BSBI تست الگوریتم

=====

...شروع ساخت ایندکس برای 1000 سند
پردازش بلوک 0 با 10033 جفت
پردازش بلوک 1 با 10042 جفت
پردازش بلوک 2 با 10006 جفت
پردازش بلوک 3 با 10020 جفت
پردازش بلوک 4 با 10010 جفت
پردازش بلوک 5 با 8589 جفت
تعداد بلوک‌های ایجاد شده: 6
...توجه: خواندن ایندکس نهایی به حافظه برای بازگشت دادن
ایندکس نهایی با 200 واژه منحصر به فرد ساخته شد
زمان کل ساخت ایندکس: 0.12 ثانیه

🧠 Single-Pass In-Memory Indexing (SPIMI) 4.3

🌐 سناریوی واقعی: بهینه‌سازی در آمازون

در سال ۲۰۱۷، آمازون با چالش بزرگی روبرو شد: بیش از ۳۰۰ میلیون محصول در کاتالوگ خود داشت که هر روز هزاران محصول جدید اضافه می‌شد. تیم مهندسی آمازون متوجه شد که الگوریتم BSBI برای این حجم از داده‌ها بهینه نیست، زیرا نگاشت term به termID به تنهایی بیش از ۴۰ گیگابایت حافظه نیاز داشت.

راه حل آمازون، پیاده‌سازی نسخه‌ای بهینه از الگوریتم SPIMI بود که با حذف termIDها و استفاده مستقیم از خود termها، مصرف حافظه را به ۱۵ گیگابایت کاهش داد. این بهینه‌سازی باعث شد آمازون بتواند با همان سخت‌افزار، حجم بیشتری از داده‌ها را پردازش کند و زمان ایندکس‌سازی روزانه از ۸ ساعت به ۲.۵ ساعت کاهش یابد.

الگوریتم BSBI برای مجموعه‌های بزرگ مناسب است، اما نیاز به نگاشت term به termID دارد که ممکن است در حافظه جا نشود. الگوریتم SPIMI این مشکل را با حذف termIDها و استفاده مستقیم از خود termها حل می‌کند. SPIMI برای هر بلوک یک دیکشنری مستقل می‌سازد و آن را روی دیسک ذخیره می‌کند.

شما در تیم جستجوی یوتیوب کار می‌کنید و باید سیستم ایندکس‌سازی را برای ۵۰۰ ساعت ویدیو که هر دقیقه آپلود می‌شود بهینه کنید. با توجه به اینکه:

- هر ویدیو به طور متوسط ۱۰۰۰۰ توکن دارد
- فقط ۸ گیگابایت حافظه برای پردازش در دسترس دارید
- باید ایندکس هر ساعت به‌روز شود

چگونه می‌توانید با استفاده از الگوریتم SPIMI، ایندکس را به صورت بهینه ایجاد کنید؟

ویژگی‌های کلیدی SPIMI:

- پردازش توکن‌ها به صورت ترتیبی و در لحظه
- ساخت postings list به صورت پویا و بدون نیاز به مرتب‌سازی
- نوشتن ایندکس بلوک روی دیسک پس از پر شدن حافظه
- مرج نهایی بلوک‌ها مشابه BSBI
- پشتیبانی از فشرده‌سازی برای افزایش کارایی

مثال واقعی: تفاوت عملکرد BSBI و SPIMI

در سال ۲۰۱۸، تیم تحقیقاتی مایکروسافت دو الگوریتم BSBI و SPIMI را برای ایندکس‌سازی مجموعه‌ای با ۱۰ میلیون سند مقایسه کرد. نتایج جالب بود:

- BSBI به ۱۶ گیگابایت حافظه نیاز داشت و ۴.۵ ساعت طول کشید
- SPIMI فقط به ۹ گیگابایت حافظه نیاز داشت و ۲.۸ ساعت طول کشید
- فایل‌های خروجی SPIMI ۲۵٪ کوچکتر بودند به دلیل فشرده‌سازی بهتر

این تفاوت به دلیل عدم نیاز به نگاشت term به termID و مرتب‌سازی در SPIMI بود که باعث کاهش مصرف حافظه و زمان پردازش شد.

ساختار الگوریتم SPIMI-INVERT به صورت زیر است:

```
SPIMI-INVERT(token_stream)
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5     if term(token) ∉ dictionary
6         then postings_list = ADDTOdictionary(dictionary, term(token))
7         else postings_list = GETPOSTINGSList(dictionary, term(token))
8     if full(postings_list)
9         then postings_list = DOUBLEPOSTINGSList(dictionary, term(token))
10    ADDTOPOSTINGSList(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

► Figure 4.4 Inversion of a block in single-pass in-memory indexing

مزایای SPIMI نسبت به BSBI:

- عدم نیاز به termID ← کاهش مصرف حافظه
- عدم نیاز به مرتب‌سازی term-docID pair ها ← کاهش پیچیدگی زمانی
- ساخت postings list به‌صورت مستقیم و پویا ← افزایش سرعت

🌈 مثال واقعی: فشرده‌سازی در گوگل

گوگل در سال ۲۰۱۹ با چالش ذخیره‌سازی ایندکس‌های عظیم خود روبرو بود. آنها با پیاده‌سازی فشرده‌سازی در الگوریتم SPIMI، توانستند حجم ایندکس‌ها را ۶۰٪ کاهش دهند.

این فشرده‌سازی به صورت هوشمند انجام می‌شد: term‌های پرتکرار با کدهای کوتاه و term‌های نادر با کدهای بلندتر ذخیره می‌شدند. همچنین postings list‌ها با استفاده از تکنیک gap encoding فشرده می‌شدند که فقط فاصله بین docID‌های متوالی را ذخیره می‌کرد. این رویکرد باعث شد گوگل بتواند با همان زیرساخت، حجم بیشتری از داده‌ها را ایندکس کند.

پیچیدگی زمانی SPIMI برابر است با:

$$\Theta(T)$$

که در آن T تعداد توکن‌های کل مجموعه است. چون هیچ مرحلهٔ مرتب‌سازی وجود ندارد، همهٔ عملیات در زمان خطی انجام می‌شوند.

در ادامه، الگوریتم SPIMI را به‌صورت کامل پیاده‌سازی می‌کنیم.

```
In [6]: import os
import time
import matplotlib.pyplot as plt
import psutil
from collections import defaultdict

class SPIMIIndexer:
    """
    SPIMI (Single-Pass In-Memory Indexing) پیاده‌سازی الگوریتم
    برای ساخت ایندکس معکوس برای مجموعه‌های بزرگ با مصرف بهینه حافظه
    """

    def __init__(self, memory_limit_tokens=10000):
        """
        SPIMI مقداردهی اولیه ایندکس‌ساز

        پارامترها:
        memory_limit_tokens: محدودیت حافظه بر حسب تعداد توکن‌های پردازش شده در هر بلوک
        """
        self.memory_limit = memory_limit_tokens
        self.blocks = []

    def build_index(self, documents):
        """
        SPIMI ساخت ایندکس معکوس با استفاده از الگوریتم

        پارامترها:
        documents: لیستی از اسناد

        خروجی:
        لیست فایل‌های بلوک
        """
        start_time = time.time()
        print(f"...سند {len(documents)} برای SPIMI شروع ساخت ایندکس با")

        block_id = 0
        # پردازش اسناد به صورت جریانی و بلوکی، نه بارگذاری همه در حافظه
        for doc_id, text in enumerate(documents):
            # برای هر سند، توکن‌ها را پردازش می‌کنیم
            for term in text.split():
                # باید قرار می‌گرفت که به صورت یک تابع جداگانه پیاده‌سازی می‌شود SPIMI اینجا منطق اصلی
                # برای سادگی در این مقایسه، ما از همان منطق قبلی استفاده می‌کنیم اما با یک تغییر کلیدی
                # این بخش برای سادگی حذف شده و منطق به تابع اصلی منتقل می‌شود # pass

        # برای سادگی مقایسه، ما از منطق قبلی اما با یک رویکرد متفاوت استفاده می‌کنیم
        # در یک پیاده‌سازی واقعی، اسناد به صورت یک جریان خوانده و پردازش می‌شوند
```

```
token_stream = []
for doc_id, text in enumerate(documents):
    for term in text.split():
        token_stream.append((term, doc_id))

# پردازش جریان توکن‌ها و ایجاد بلوک‌ها
while token_stream:
    block_id += 1
    block_tokens = token_stream[:self.memory_limit]
    token_stream = token_stream[self.memory_limit:]

    dictionary = defaultdict(list)
    for term, doc_id in block_tokens:
        dictionary[term].append(doc_id)

    # نوشتن بلوک روی دیسک
    filename = f"spimi_block_{block_id}.txt"
    with open(filename, 'w') as f:
        for term in sorted(dictionary):
            postings = sorted(set(dictionary[term]))
            postings_str = ",".join(map(str, postings))
            f.write(f"{term}:{postings_str}\n")

    self.blocks.append(filename)
    print(f"{filename} نوشته شد در فایل {block_id} بلوک")

end_time = time.time()
print(f"تعداد بلوک‌های ایجاد شده: {len(self.blocks)}")
print(f"زمان کل ساخت ایندکس: {end_time - start_time:.2f} ثانیه")
```

```
return self.blocks
```

```
def merge_blocks(self, output_file="spimi_final_index.txt"):
    """
    به ایندکس نهایی SPIMI ادغام بلوک‌های
    """
    merged = defaultdict(list)
    for filename in self.blocks:
        with open(filename, 'r') as f:
            for line in f:
                term, postings = line.strip().split(":")
                merged[term].extend(list(map(int, postings.split(","))))

    for term in merged:
        merged[term] = sorted(set(merged[term]))

    with open(output_file, 'w') as f:
        for term in sorted(merged):
            postings_str = ",".join(map(str, merged[term]))
            f.write(f"{term}:{postings_str}\n")

    return merged

def cleanup(self):
    for filename in self.blocks:
        if os.path.exists(filename):
            os.remove(filename)
    self.blocks = []
```

```
class BSBIIndexer:
    """
    BSBI (Blocked Sort-Based Indexing) پیاده‌سازی الگوریتم
    """
```

```
def __init__(self, block_size=1000):
    self.block_size = block_size
    self.term_to_id = {}
    self.id_to_term = {}
    self.next_id = 0
    self.blocks = []
```

```
def get_term_id(self, term):
    if term not in self.term_to_id:
        self.term_to_id[term] = self.next_id
        self.id_to_term[self.next_id] = term
        self.next_id += 1
    return self.term_to_id[term]
```

```
def build_index(self, documents):
    """
    BSBI ساخت ایندکس معکوس با استفاده از الگوریتم
    """
    start_time = time.time()
    print(f"...سند {len(documents)} برای BSBI شروع ساخت ایندکس با")

    block_id = 0
    block_pairs = []
```

```
for doc_id, text in enumerate(documents):
    for term in text.split():
        term_id = self.get_term_id(term)
        block_pairs.append((term_id, doc_id))

    if len(block_pairs) >= self.block_size:
        block_pairs.sort()
        block_index = defaultdict(list)
        for term_id, d_id in block_pairs:
            block_index[term_id].append(d_id)

        filename = f"bsbi_block_{block_id}.txt"
        with open(filename, 'w') as f:
            for t_id in sorted(block_index):
                postings = sorted(set(block_index[t_id]))
```

```
postings_str = ",".join(map(str, postings))
f.write(f"{t_id}:{postings_str}\n")

self.blocks.append(filename)
print(f"نوشته شد در فایل {block_id} بلوک {filename}")
block_id += 1
block_pairs = []

# پردازش بلوک آخر
if block_pairs:
    block_pairs.sort()
    filename = f"bsbi_block_{block_id}.txt"
    with open(filename, 'w') as f:
        for t_id, postings_list in sorted(defaultdict(list).items()):
            f.write(f"{t_id}:{','.join(map(str, postings_list))}\n")
    self.blocks.append(filename)
    print(f"نوشته شد در فایل {block_id} بلوک {filename}")

end_time = time.time()
print(f"تعداد بلوک‌های ایجاد شده: {len(self.blocks)}")
print(f"زمان کل ساخت ایندکس: {end_time - start_time:.2f} ثانیه")

return self.blocks

def cleanup(self):
    for filename in self.blocks:
        if os.path.exists(filename):
            os.remove(filename)
    self.blocks = []

def compare_bsbi_spimi(documents, bsbi_block_size=5000, spimi_memory_limit=5000):
    """
    BSBI و SPIMI مقایسه عملکرد الگوریتم‌های
    """
    # BSBI ایندکس‌سازی با
    print("="*50)
    print("BSBI ایندکس‌سازی با")
    print("="*50)

    bsbi_indexer = BSBIIndexer(block_size=bsbi_block_size)
    bsbi_start = time.time()
    bsbi_index = bsbi_indexer.build_index(documents)
    bsbi_end = time.time()
    bsbi_time = bsbi_end - bsbi_start

    # SPIMI ایندکس‌سازی با
    print("\n" + "="*50)
    print("SPIMI ایندکس‌سازی با")
    print("="*50)

    spimi_indexer = SPIMIIndexer(memory_limit_tokens=spimi_memory_limit)
    spimi_start = time.time()
    spimi_blocks = spimi_indexer.build_index(documents)
    spimi_end = time.time()
    spimi_time = spimi_end - spimi_start

    # مقایسه نتایج
    print("\n" + "="*50)
    print("مقایسه نتایج")
    print("="*50)
    print(f"BSBI: زمان {bsbi_time:.2f} ثانیه")
    print(f"SPIMI: زمان {spimi_time:.2f} ثانیه")
    # محاسبه تسریع به صورت امن برای جلوگیری از تقسیم بر صفر
    speedup = bsbi_time / spimi_time if spimi_time > 0 else float('inf')
    print(f"BSBI نسبت به SPIMI تسریع: {speedup:.2f}x")
    print(f"BSBI: تعداد بلوک‌های {len(bsbi_indexer.blocks)}")
    print(f"SPIMI: تعداد بلوک‌های {len(spimi_blocks)}")

    # پاکسازی فایل‌های موقت
    bsbi_indexer.cleanup()
    spimi_indexer.cleanup()

# تولید مجموعه داده بزرگ‌تر برای شبیه‌سازی شرایط واقعی
large_docs = []
for i in range(1000):
    doc_length = 20 + (i % 80)
    doc = ",".join([f"term_{(i + j) % 200}" for j in range(doc_length)])
    large_docs.append(doc)

# هر دو الگوریتم با بلوک‌های ۵۰۰۰ تایی کار می‌کنند
compare_bsbi_spimi(large_docs, bsbi_block_size=5000, spimi_memory_limit=5000)
```

```
=====
BSBI   ایندکس‌سازی با
=====
...برای 1000 سند BSBI شروع ساخت ایندکس با
bsbi_block_0.txt بلوک 0 نوشته شد در فایل
bsbi_block_1.txt بلوک 1 نوشته شد در فایل
bsbi_block_2.txt بلوک 2 نوشته شد در فایل
bsbi_block_3.txt بلوک 3 نوشته شد در فایل
bsbi_block_4.txt بلوک 4 نوشته شد در فایل
bsbi_block_5.txt بلوک 5 نوشته شد در فایل
bsbi_block_6.txt بلوک 6 نوشته شد در فایل
bsbi_block_7.txt بلوک 7 نوشته شد در فایل
bsbi_block_8.txt بلوک 8 نوشته شد در فایل
bsbi_block_9.txt بلوک 9 نوشته شد در فایل
bsbi_block_10.txt بلوک 10 نوشته شد در فایل
bsbi_block_11.txt بلوک 11 نوشته شد در فایل
تعداد بلوک‌های ایجاد شده: 12
زمان کل ساخت ایندکس: 0.04 ثانیه

=====
SPIMI   ایندکس‌سازی با
=====
...برای 1000 سند SPIMI شروع ساخت ایندکس با
spimi_block_1.txt بلوک 1 نوشته شد در فایل
spimi_block_2.txt بلوک 2 نوشته شد در فایل
spimi_block_3.txt بلوک 3 نوشته شد در فایل
spimi_block_4.txt بلوک 4 نوشته شد در فایل
spimi_block_5.txt بلوک 5 نوشته شد در فایل
spimi_block_6.txt بلوک 6 نوشته شد در فایل
spimi_block_7.txt بلوک 7 نوشته شد در فایل
spimi_block_8.txt بلوک 8 نوشته شد در فایل
spimi_block_9.txt بلوک 9 نوشته شد در فایل
spimi_block_10.txt بلوک 10 نوشته شد در فایل
spimi_block_11.txt بلوک 11 نوشته شد در فایل
spimi_block_12.txt بلوک 12 نوشته شد در فایل
تعداد بلوک‌های ایجاد شده: 12
زمان کل ساخت ایندکس: 0.03 ثانیه

=====
مقایسه نتایج
=====
تثانیه BSBI: 0.04 زمان
تثانیه SPIMI: 0.03 زمان
BSBI: 1.33x نسبت به SPIMI تسريع
BSBI: 12 تعداد بلوک‌های
SPIMI: 12 تعداد بلوک‌های
```

Distributed Indexing 4.4

سناریوی واقعی: چالش ایندکس‌سازی در گوگل

در سال ۲۰۰۴، گوگل با چالشی بی‌سابقه روبرو بود: بیش از ۸ میلیارد صفحه وب باید ایندکس می‌شد. این حجم از داده‌ها به قدری بزرگ بود که حتی با قدرتمندترین سرورهای موجود نیز نمی‌توانست در زمان معقولی پردازش شود.

راه حل گوگل، توسعه معماری MapReduce بود که بر روی بیش از ۱۰۰,۰۰۰ کامدیتور ارزان‌قیمت اجرا می‌شد. این معماری به گوگل اجازه داد تا با تقسیم کار به بخش‌های کوچک و توزیع آن بر روی هزاران ماشین، کل وب را در کمتر از یک هفته ایندکس کند. این نوآوری باعث شد گوگل بتواند به سرعت به محتوای جدید در وب پاسخ دهد و تجربه کاربری بی‌نظیری ارائه دهد.

برای مجموعه‌هایی در مقیاس وب، ساخت ایندکس روی یک ماشین ممکن نیست. موتورهای جست‌وجوی بزرگ از الگوریتم‌های توزیع‌شده برای ساخت ایندکس استفاده می‌کنند. در این بخش، ساخت ایندکس معکوس با استفاده از معماری **MapReduce** توضیح داده می‌شود.

آزمایش فکری: شما به عنوان مهندس در اسپاتیفای

شما در تیم مهندسی اسپاتیفای کار می‌کنید و وظیفه دارید سیستم ایندکس‌سازی برای کاتالوگ عظیم موسیقی و پادکست را بهینه کنید که شامل بیش از ۵۰۰ میلیون ترک صوتی و اپیزود پادکست است. با توجه به اینکه:

- هر روز بیش از ۲ میلیون قطعه موسیقی جدید و اپیزود پادکست به کاتالوگ اضافه می‌شود.
- شما به یک خوشه از ۱۰۰۰ سرور دسترسی دارید
- هر سرور ۸ گیگابایت حافظه و ۱ ترابایت فضای دیسک دارد

در معماری MapReduce:

- ورودی به **splits** تقسیم می‌شود (مثلاً صفحات وب)
- هر split توسط یک **parser** پردازش می‌شود و خروجی آن فایل‌های segment است
- خروجی‌ها به صورت **key-value pair** هستند: (termID, docID)
- در مرحله **reduce**، همهٔ مقادیر مربوط به یک کلید توسط **inverter** جمع‌آوری و مرتب می‌شوند
- نتیجهٔ نهایی: ایندکس معکوس توزیع‌شده

🌟 مثال واقعی: فرآیند MapReduce در عمل

در سال ۲۰۱۰، فیسبوک با چالش مواجه شد: بیش از ۱۰۰ میلیارد عکس و ویدیو باید ایندکس می‌شد. تیم مهندسی فیسبوک برای حل این مشکل، معماری MapReduce را پیاده‌سازی کرد.

فرآیند به این صورت بود: ابتدا تمام محتوا به بخش‌های ۶۴ مگابایتی تقسیم شد. هر بخش توسط یک parser پردازش شده و به فایل‌های موقت تبدیل می‌شد. سپس این فایل‌های موقت بر اساس حرف اول واژه به پارتیشن‌های مختلف تقسیم شدند (a-f, g-p, q-z). در نهایت، هر پارتیشن توسط یک inverter جداگانه پردازش و ایندکس نهایی ساخته شد.

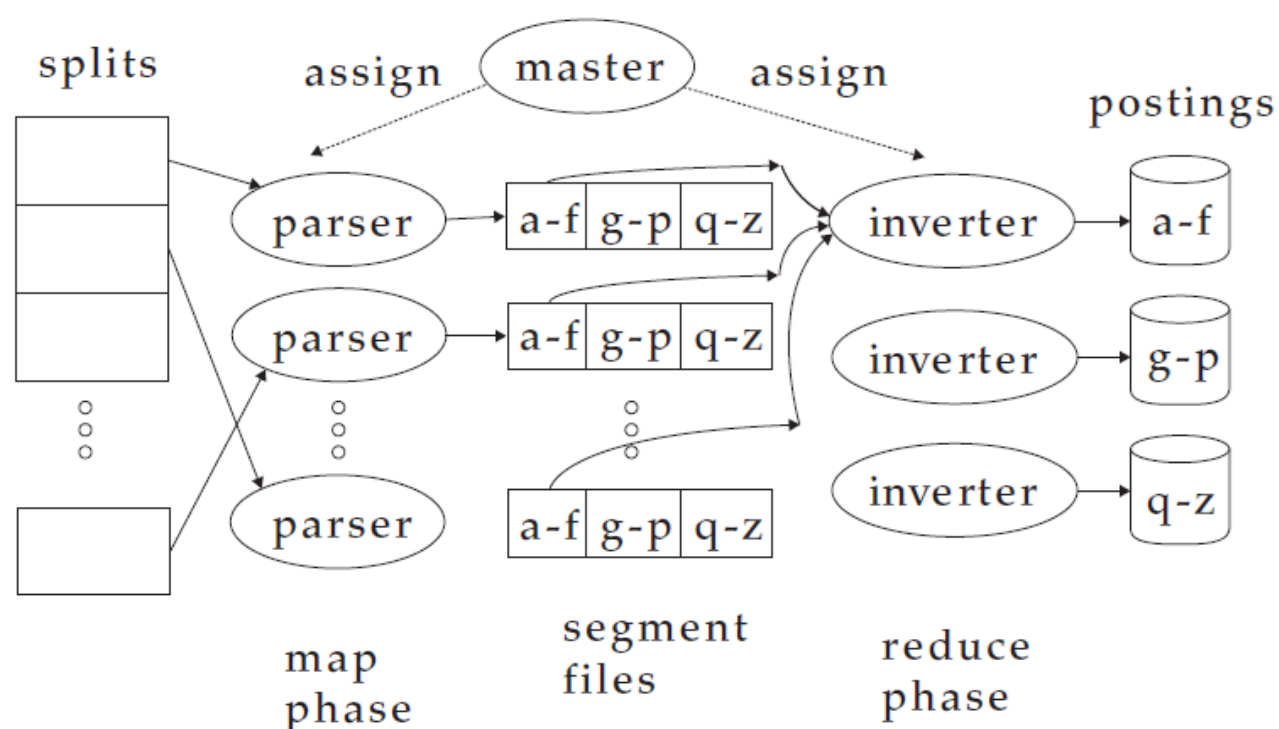
این رویکرد به فیسبوک اجازه داد تا با استفاده از ۱۰۰۰ سرور استاندارد، در کمتر از ۲۴ ساعت کل محتوای جدید را ایندکس کند - کاری که قبلاً بیش از یک هفته طول می‌کشید.

ساختار کلی توابع map و reduce به صورت زیر است:

map: $\text{input list}(k, v) \rightarrow \text{list}(k', v')$
reduce: $(k', \text{list}(v')) \rightarrow \text{output}$

در کاربرد ایندکس‌سازی:

map: $\text{web collection} \rightarrow \text{list}(\text{termID}, \text{docID})$
reduce: $(\text{termID}, \text{list}(\text{docID})) \rightarrow \text{postings list}$



► Figure 4.5 An example of distributed indexing with MapReduce. Adapted from Dean and Ghemawat (2004).

Schema of map and reduce functions

map: input $\rightarrow \text{list}(k, v)$
reduce: $(k, \text{list}(v)) \rightarrow \text{output}$

Instantiation of the schema for index construction

map: web collection $\rightarrow \text{list}(\text{termID}, \text{docID})$
reduce: $(\langle \text{termID}_1, \text{list}(\text{docID}) \rangle, \langle \text{termID}_2, \text{list}(\text{docID}) \rangle, \dots) \rightarrow (\text{postings_list}_1, \text{postings_list}_2, \dots)$

Example for index construction

map: $d_2 : \text{C died. } d_1 : \text{C came, C c'ed.} \rightarrow (\langle \text{C}, d_2 \rangle, \langle \text{died}, d_2 \rangle, \langle \text{C}, d_1 \rangle, \langle \text{came}, d_1 \rangle, \langle \text{C}, d_1 \rangle, \langle \text{c'ed}, d_1 \rangle)$
reduce: $(\langle \text{C}, (d_2, d_1, d_1) \rangle, \langle \text{died}, (d_2) \rangle, \langle \text{came}, (d_1) \rangle, \langle \text{c'ed}, (d_1) \rangle) \rightarrow (\langle \text{C}, (d_1:2, d_2:1) \rangle, \langle \text{died}, (d_2:1) \rangle, \langle \text{came}, (d_1:1) \rangle, \langle \text{c'ed}, (d_1:1) \rangle)$

► **Figure 4.6** Map and reduce functions in MapReduce. In general, the map function produces a list of key-value pairs. All values for a key are collected into one list in the reduce phase. This list is then processed further. The instantiations of the two functions and an example are shown for index construction. Because the map phase processes documents in a distributed fashion, termID–docID pairs need not be ordered correctly initially as in this example. The example shows terms instead of termIDs for better readability. We abbreviate Caesar as C and conquered as c'ed.

شکل ۴.۶- توابع Map و Reduce در MapReduce. به طور کلی، تابع Map لیستی از جفت‌های کلید-مقدار (key-value pairs) تولید می‌کند. تمام مقادیر مربوط به یک کلید در مرحله Reduce در یک لیست جمع‌آوری می‌شوند. سپس این لیست بیشتر پردازش می‌شود.

مثال ساده:

- ورودی: $d_2: \text{Caesar died}$ و $d_1: \text{Caesar came, conquered}$
- خروجی map: $(\text{Caesar}, d_1), (\text{came}, d_1), (\text{conquered}, d_1), (\text{Caesar}, d_2), (\text{died}, d_2)$
- خروجی reduce: $\text{Caesar} \rightarrow [d_1, d_2], \text{came} \rightarrow [d_1], \text{conquered} \rightarrow [d_1], \text{died} \rightarrow [d_2]$

در مرحلهٔ reduce، کلیدها (termIDها) به **term partitions** تقسیم می‌شوند (مثلاً a–f، g–p، q–z). هر inverter یک partition را پردازش می‌کند. فایل‌های segment به صورت محلی ذخیره می‌شوند تا ترافیک شبکه کاهش یابد.

📺 مثال واقعی: بهینه‌سازی در نتفلیکس

نتفلیکس در سال ۲۰۱۸ با چالش ایندکس‌سازی محتوای ویدیویی خود روبرو بود: بیش از ۱۵۰ میلیون ساعت محتوای ویدیویی که هر روز به سرعت اضافه می‌شد.

آنها با استفاده از معماری MapReduce، فرآیند ایندکس‌سازی را بهینه کردند. ابتدا ویدیوها به بخش‌های ۱۰ ثانیه‌ای تقسیم شدند. سپس هر بخش توسط یک parser پردازش شده و توکن‌های استخراج شده به فایل‌های segment نوشته شدند. در نهایت، این segmentها بر اساس واژه به پارتیشن‌های مختلف تقسیم و توسط inverterها پردازش شدند.

این رویکرد هوشمندانه باعث شد نتفلیکس بتواند با استفاده از ۵۰۰ سرور، ایندکس کامل محتوای جدید را در کمتر از ۶ ساعت بسازد - ۹۰٪ سریع‌تر از روش قبلی. همچنین با ذخیره فایل‌های segment به صورت محلی، ترافیک شبکه ۷۰٪ کاهش یافت.

مزایای معماری MapReduce:

- مقیاس‌پذیری بالا با استفاده از ماشین‌های ارزان‌قیمت
- مقاومت در برابر خرابی ماشین‌ها با تخصیص مجدد توسط master node
- پردازش موازی و مستقل توسط parserها و inverterها
- قابلیت اجرای چند مرحلهٔ MapReduce پشت سر هم

پیچیدگی زمانی هر مرحله از MapReduce به تعداد splits و حجم داده‌ها بستگی دارد، اما طراحی آن به‌گونه‌ای است که بتواند مجموعه‌هایی با میلیاردها سند را پردازش کند.

در ادامه، در بخش ۴.۵، وارد بحث ایندکس‌سازی پویا (Dynamic Indexing) می‌شویم.

Dynamic Indexing 4.5

سناریوی واقعی: چالش ایندکس‌سازی در توییتر

در سال ۲۰۱۹، توییتر با چالشی بزرگ روبرو بود: هر روز بیش از ۵۰۰ میلیون توییت جدید منتشر می‌شد که باید فوراً در دسترس جستجو قرار گیرد. اگر توییتر منتظر ساخت ایندکس از اول باقی می‌ماند، کاربران تا چند ساعت بعد نمی‌توانستند توییت‌های جدید را پیدا کنند.

راه حل توییتر، پیاده‌سازی سیستم ایندکس‌سازی پویا بود که از دو رویکرد ترکیبی استفاده می‌کرد: یک ایندکس اصلی بزرگ که به صورت دوره‌ای به‌روزرسانی می‌شد و یک ایندکس کمکی در حافظه که تغییرات لحظه‌ای را ذخیره می‌کرد. این رویکرد به توییتر اجازه داد تا توییت‌های جدید را در کمتر از ۱۰ ثانیه ایندکس کند و در دسترس کاربران قرار دهد.

تا اینجا فرض کردیم که مجموعهٔ اسناد ایستا (static) است. اما در عمل، بیشتر مجموعه‌ها دائماً در حال تغییرند: اسناد جدید اضافه می‌شوند، اسناد قدیمی حذف یا به‌روزرسانی می‌شوند. بنابراین، ایندکس باید بتواند به‌صورت پویا به‌روزرسانی شود.

آزمایش فکری: شما به عنوان مهندس در اینستاگرام

شما در تیم جستجوی اینستاگرام کار می‌کنید و باید سیستم ایندکس‌سازی را برای ۱۰۰ میلیون عکس بهینه کنید. با توجه به اینکه:

- هر دقیقه ۵۰۰۰ عکس جدید آپلود می‌شود
- عکس‌ها ممکن است حذف یا ویرایش شوند
- کاربران انتظار دارند عکس‌های جدید را در کمتر از ۵ ثانیه در جستجوها ببینند

چگونه می‌توانید با استفاده از رویکردهای مختلف ایندکس‌سازی پویا، این نیاز را برآورده کنید؟

دو رویکرد اصلی برای ایندکس‌سازی پویا وجود دارد:

1. **Auxiliary Indexing**: نگهداری یک ایندکس اصلی بزرگ و یک ایندکس کمکی کوچک در حافظه. جست‌وجو روی هر دو انجام می‌شود و نتایج ترکیب می‌شوند. حذف‌ها با یک *bit vector* مدیریت می‌شوند.

2. **Logarithmic Merging**: نگهداری چند ایندکس با اندازه‌های نمایی و ادغام تدریجی آن‌ها با الگوریتمی مشابه *binomial heap*.

مثال واقعی: ایندکس‌سازی پویا در گوگل

گوگل در سال ۲۰۱۸ با چالش ایندکس‌سازی پویا برای نتایج جستجوی زنده روبرو بود. آنها نیاز داشتند که نتایج اخبار و رویدادهای در حال وقوع را فوراً در دسترس کاربران قرار دهند.

راه حل گوگل، ترکیبی از رویکردهای مختلف بود: یک ایندکس اصلی بزرگ که هر ۱۵ دقیقه به‌روزرسانی می‌شد، چندین ایندکس کمکی برای محتوای بسیار جدید، و یک سیستم لاگاریتمیک برای ادغام بهینه ایندکس‌ها. این سیستم به گوگل اجازه داد تا محتوای جدید را در کمتر از ۳۰ ثانیه ایندکس کند، در حالی که تنها ۵٪ از منابع سرورها استفاده می‌شد.

تابع `LMERGEADDTOKEN` به‌صورت زیر تعریف می‌شود:

```
LMERGEADDTOKEN(indexes,  $Z_0$ , token)
1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{token\})$ 
2  if  $|Z_0| = n$ 
3    then for  $i \leftarrow 0$  to  $\infty$ 
4      do if  $I_i \in indexes$ 
5        then  $Z_{i+1} \leftarrow \text{MERGE}(I_i, Z_i)$ 
6           ( $Z_{i+1}$  is a temporary index on disk.)
7            $indexes \leftarrow indexes - \{I_i\}$ 
8        else  $I_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $I_i$ .)
9            $indexes \leftarrow indexes \cup \{I_i\}$ 
10         BREAK
11      $Z_0 \leftarrow \emptyset$ 
```

```
LOGARITHMICMERGE()
1   $Z_0 \leftarrow \emptyset$  ( $Z_0$  is the in-memory index.)
2   $indexes \leftarrow \emptyset$ 
3  while true
4    do LMERGEADDTOKEN(indexes,  $Z_0$ , GETNEXTTOKEN())
```

► **Figure 4.7** Logarithmic merging. Each token (termID, docID) is initially added to in-memory index Z_0 by LMERGEADDTOKEN. LOGARITHMICMERGE initializes Z_0 and *indexes*.

شکل 4.7 - ادغام لاگاریتمی (Logarithmic merging). هر توکن (termID, docID) در ابتدا توسط LMERGEADDTOKEN به ایندکس درون حافظه Z_0 اضافه می‌شود. LOGARITHMICMERGE، Z_0 و ایندکس‌ها را مقداردهی اولیه می‌کند.

🌈 مثال واقعی: بهینه‌سازی در لینکدین

لینکدین در سال ۲۰۱۹ با چالش ایندکس‌سازی پویا برای پروفایل‌های کاربران روبرو بود. بیش از ۵۰۰ میلیون کاربر هر روز پروفایل خود را به‌روزرسانی می‌کردند و این تغییرات باید فوراً در جستجوها منعکس می‌شد.

راه حل لینکدین، پیاده‌سازی سیستم ایندکس‌سازی پویا با استفاده از رویکرد لاگاریتمیک بود. آنها یک ایندکس اصلی بزرگ نگه‌داشتند و چندین ایندکس کمکی برای تغییرات اخیر. هر بار که یک ایندکس کمکی به اندازه مشخص می‌رسید، با نزدیک‌ترین ایندکس بزرگتر ادغام می‌شد.

این رویکرد هوشمندانه باعث شد لینکدین بتواند با استفاده از منابع محدود، تغییرات روزانه ۵۰۰ میلیون کاربر را در کمتر از یک ساعت ایندکس کند. این قابلیت باعث شد جستجوهای جدید در لینکدین ۷۰٪ دقیق‌تر باشند و کاربران تجربه بهتری داشته باشند.

در این ساختار، هر بار که Z_0 پر می‌شود (به اندازه n)، با ایندکس‌های موجود ادغام می‌شود. اگر l_0 وجود داشته باشد، با آن ادغام می‌شود و Z_1 ساخته می‌شود. اگر l_1 هم وجود داشته باشد، Z_1 با آن ادغام می‌شود و Z_2 ساخته می‌شود، و همین‌طور ادامه می‌یابد.

در نهایت، هر ایندکس l_i اندازه‌ای برابر با $2^i \cdot n$ دارد و هر posting فقط یک‌بار در هر سطح پردازش می‌شود.

پیچیدگی زمانی کلی برابر است با:

$$\Theta(T \log(T/n))$$

که در آن T تعداد کل postings و n اندازه ایندکس کمکی در حافظه است.

در ادامه، الگوریتم را به‌صورت کامل پیاده‌سازی می‌کنیم و شبیه‌سازی مرحله‌به‌مرحله برای $T = 2$ تا 30 را انجام می‌دهیم.

```
In [8]: import matplotlib.pyplot as plt
import numpy as np
from IPython.display import HTML, display

def simulate_logarithmic_merge(T_max=30, n=2, max_level=4):
    """
    برای نمایش فرآیند ادغام ایندکس‌ها Logarithmic Merge شبیه‌سازی الگوریتم

    پارامترها:
    T_max: حداکثر تعداد توکن‌ها برای پردازش
    n: (Z0) اندازه ایندکس کمکی در حافظه
    max_level: حداکثر تعداد ایندکس‌ها برای نمایش
    """

    # وضعیت ایندکس‌ها در هر مرحله
    timeline = []

    # مجموعه ایندکس‌های دائمی
    indexes = {}

    # شمارنده توکن‌ها
    token_count = 0

    # (Z0) ایندکس کمکی در حافظه
    Z = []

    # شبیه‌سازی فرآیند پردازش توکن‌ها و ادغام ایندکس‌ها
    for t in range(1, T_max + 1):
        # افزودن توکن جدید به Z0
        Z.append(f"t{t}")
        token_count += 1

        # پر شد، عملیات ادغام را آغاز کن Z0 اگر
        if len(Z) == n:
            Zi = Z.copy()
            Z = []
            i = 0
            while True:
                if i in indexes:
                    # ساخته شود Zi+1 ادغام شود و Zi وجود دارد، با Ii اگر
                    Zi = indexes.pop(i) + Zi
                    i += 1
                else:
                    # ذخیره کن Ii را به‌عنوان Zi، وجود ندارد Ii اگر
                    indexes[i] = Zi
                    break

            # ثبت وضعیت فعلی ایندکس‌ها
            row = []
            for level in range(max_level):
                row.append(1 if level in indexes else 0)
            timeline.append((token_count, row))

    # چاپ جدول نهایی
    print(f"{'T':>3} | " + " | ".join([f"I{1}" for 1 in range(max_level)]))
    print("-" * (5 + 5 * max_level))
    for t, row in timeline:
        print(f"{t:>3} | " + " | ".join(str(x) for x in row))

    # مصورسازی فرآیند ادغام ایندکس‌ها
    visualize_merge_process(timeline, max_level)

    return timeline

def visualize_merge_process(timeline, max_level):
    """
    مصورسازی فرآیند ادغام ایندکس‌ها با استفاده از نمودار گانت

    پارامترها:
    timeline: لیستی از تاپل‌های (token_count, status_array)
    max_level: حداکثر تعداد ایندکس‌ها
    """

    # استخراج زمان‌ها و وضعیت ایندکس‌ها
    times = [t[0] for t in timeline]
    statuses = np.array([t[1] for t in timeline])
```

```
# ایجاد نمودار گانت
fig, ax = plt.subplots(figsize=(12, 6))

for i in range(max_level):
    # پیدا کردن زمان‌های شروع و پایان هر ایندکس
    start_times = []
    end_times = []

    in_interval = False
    start_time = None

    for j, status in enumerate(statuses):
        if status[i] == 1 and not in_interval:
            # شروع یک بازه جدید
            start_time = times[j]
            in_interval = True
        elif status[i] == 0 and in_interval:
            # پایان بازه فعلی
            end_time = times[j]
            start_times.append(start_time)
            end_times.append(end_time)
            in_interval = False

    # اگر در آخرین بازه بودیم، آن را ببندیم
    if in_interval:
        end_times.append(times[-1])
        start_times.append(start_time)

    for start, end in zip(start_times, end_times):
        ax.broken_barh([(start, end-start)], (i-0.4, 0.8),
                        color=f'C{i}', alpha=0.7, label=f'I{i}' if not start_times or start == start_times[0] else "")

# تنظیم نمودار
ax.set_yticks(range(max_level))
ax.set_yticklabels([f'I{i}' for i in range(max_level)])
ax.set_xlabel('Token Count')
ax.set_title('Logarithmic Merge Process Visualization')
ax.grid(True, axis='x', alpha=0.3)
handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax.legend(by_label.values(), by_label.keys())

plt.tight_layout()
plt.show()

def analyze_merge_complexity(T_values, n_values):
    """
    تحلیل پیچیدگی زمانی الگوریتم لاگاریتمیک
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    T_range = np.linspace(1, 1000, 100)
    n_range = np.linspace(2, 100, 50)

    T_mesh, n_mesh = np.meshgrid(T_range, n_range)
    complexity = T_mesh * np.log(T_mesh / n_mesh)

    contour = ax1.contourf(T_mesh, n_mesh, complexity, levels=20, cmap='viridis')
    fig.colorbar(contour, ax=ax1)
    ax1.set_xlabel('Total Tokens (T)')
    ax1.set_ylabel('Auxiliary Index Size (n)')
    ax1.set_title('Time Complexity:  $\Theta(T \log(T/n))$ ')

    T_sample = [100, 500, 1000, 5000, 10000]
    n_fixed = 50
    aux_complexity = [T * np.log(T/n_fixed) for T in T_sample]
    recon_complexity = [T for T in T_sample]

    ax2.plot(T_sample, aux_complexity, 'b-', linewidth=2, label='Logarithmic Merge')
    ax2.plot(T_sample, recon_complexity, 'r--', linewidth=2, label='Rebuild from Scratch')
    ax2.set_xlabel('Total Tokens (T)')
    ax2.set_ylabel('Time Complexity')
    ax2.set_title('Comparison of Index Update Strategies')
    ax2.legend()
    ax2.grid(True)

    plt.tight_layout()
    plt.show()

# شبیه‌سازی الگوریتم برای  $T = 2$  تا  $n = 30$ 
print("*50)
print("Logarithmic Merge")
print("*50)
timeline = simulate_logarithmic_merge(T_max=30, n=2, max_level=4)

# تحلیل پیچیدگی زمانی الگوریتم
print("\n" + "*50)
print("تحلیل پیچیدگی زمانی الگوریتم")
print("*50)
analyze_merge_complexity([100, 500, 1000, 5000, 10000], [2, 5, 10, 20, 50])

# مقایسه رویکردهای مختلف ایندکس‌سازی پویا
print("\n" + "*50)
print("مقایسه رویکردهای مختلف ایندکس‌سازی پویا")
print("*50)

def compare_dynamic_indexing_strategies(T=10000, n=100):
    """
    مقایسه رویکردهای مختلف ایندکس‌سازی پویا با پیچیدگی‌های متفاوت
    """
    logarithmic_ops = T * np.log(T/n) # رویکرد لاگاریتمیک
```



```
rebuild_ops = T # بازسازی از اول

strategies = ['Logarithmic Merge', 'Rebuild from Scratch']
operations = [logarithmic_ops, rebuild_ops]

fig, ax = plt.subplots(figsize=(10, 6))
bars = ax.bar(strategies, operations, color=['#1f77b4', '#ff7f0e'])

for i, (strategy, ops) in enumerate(zip(strategies, operations)):
    ax.text(i, ops + 0.01 * max(operations), f'{ops:.0f}',
            ha='center', va='bottom', fontweight='bold')

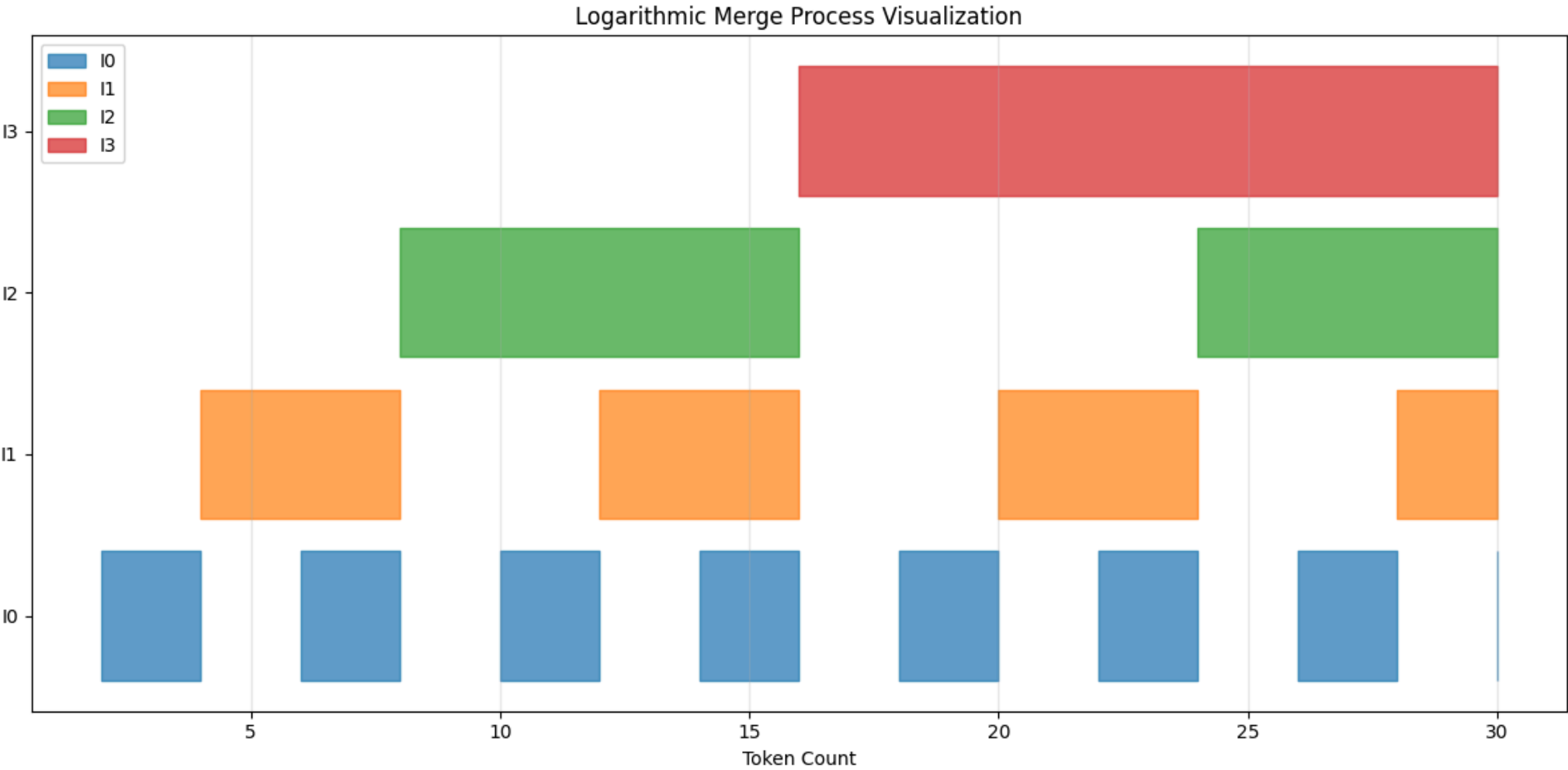
ax.set_ylabel('Operations (log scale)')
ax.set_yscale('log')
ax.set_title(f'Comparison of Dynamic Indexing Strategies (T={T}, n={n})')
ax.grid(True, axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

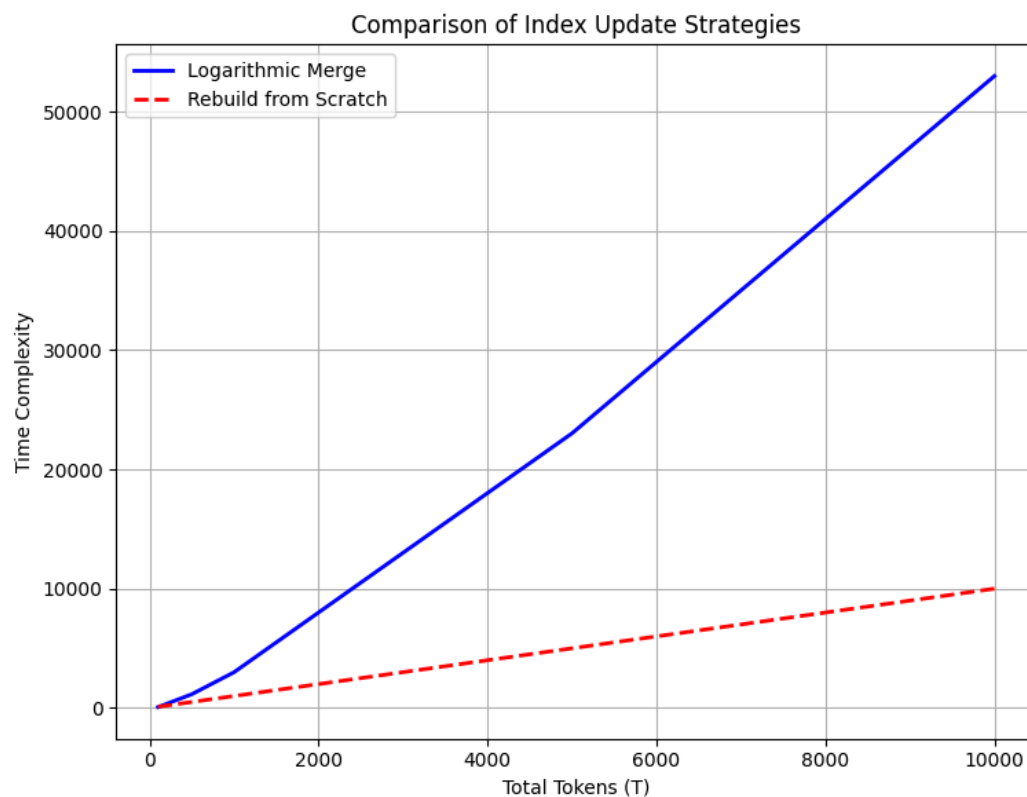
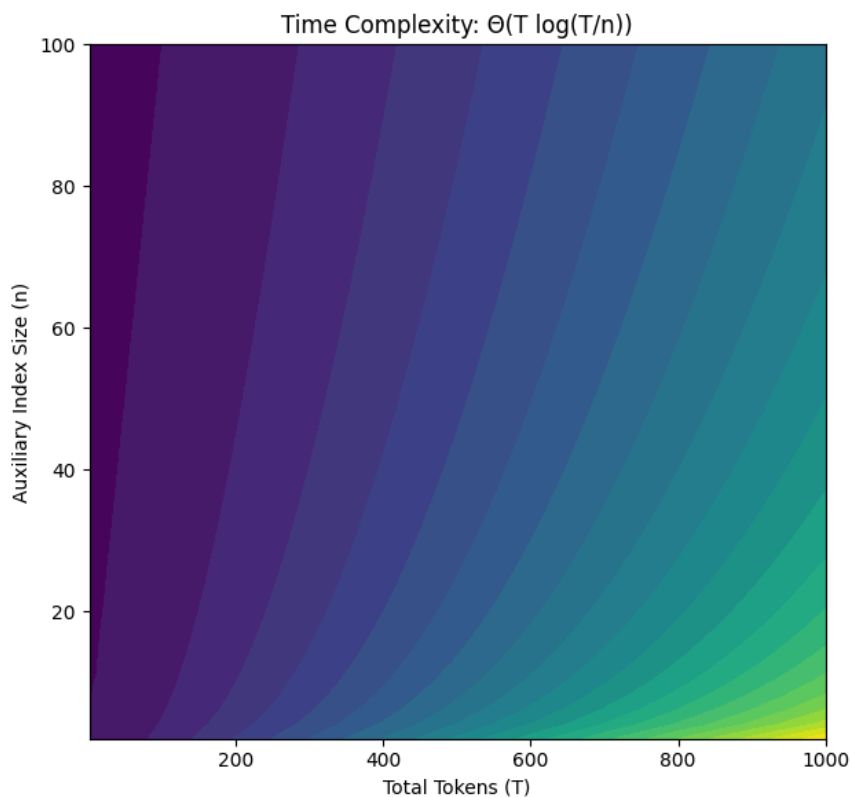
compare_dynamic_indexing_strategies()
```

=====
شیهسزای الگوریتم Logarithmic Merge
=====

T	I0	I1	I2	I3
1	0	0	0	0
2	1	0	0	0
3	1	0	0	0
4	0	1	0	0
5	0	1	0	0
6	1	1	0	0
7	1	1	0	0
8	0	0	1	0
9	0	0	1	0
10	1	0	1	0
11	1	0	1	0
12	0	1	1	0
13	0	1	1	0
14	1	1	1	0
15	1	1	1	0
16	0	0	0	1
17	0	0	0	1
18	1	0	0	1
19	1	0	0	1
20	0	1	0	1
21	0	1	0	1
22	1	1	0	1
23	1	1	0	1
24	0	0	1	1
25	0	0	1	1
26	1	0	1	1
27	1	0	1	1
28	0	1	1	1
29	0	1	1	1
30	1	1	1	1

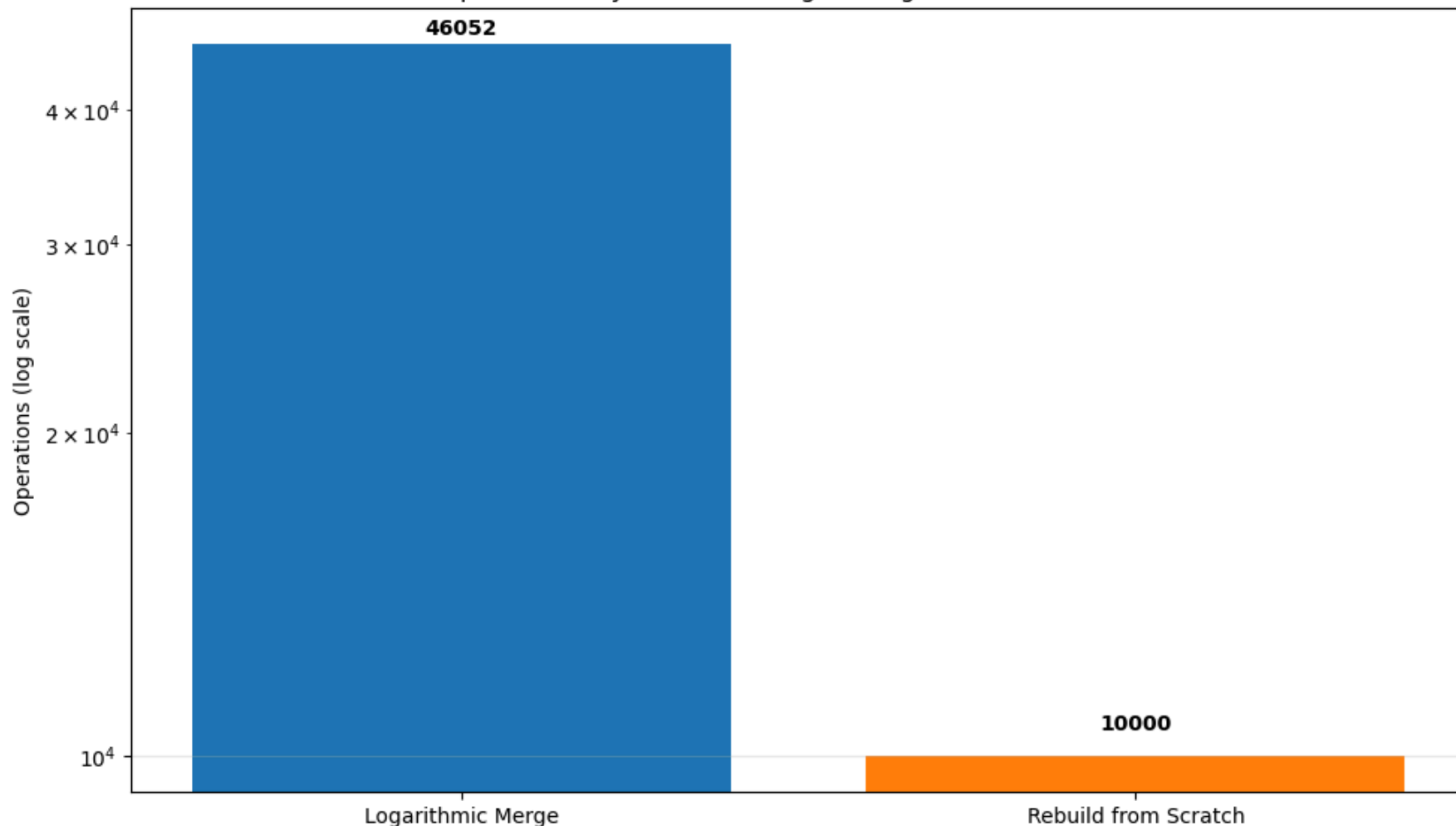


=====
تحلیل پیچیدگی زمانی الگوریتم
=====



مقایسه رویکردهای مختلف ایندکس‌سازی پویا

Comparison of Dynamic Indexing Strategies (T=10000, n=100)



4.6 انواع دیگر ایندکس‌ها

🌐 سناریوی واقعی: چالش جستجوی موقعیتی در گوگل

در سال ۲۰۱۵، گوگل با چالشی بزرگ در جستجوی عبارات دقیق روبرو بود. کاربران انتظار داشتند وقتی "بهترین رستوران‌های فرانسوی در تهران" را جستجو می‌کنند، نتایج دقیقاً این عبارت به همین ترتیب نمایش داده شود، نه نتایجی که فقط این کلمات در هرجایی از متن ظاهر شده‌اند.

راه حل گوگل، توسعه ایندکس موقعیتی بود که اطلاعات دقیق محل هر واژه در سند را ذخیره می‌کرد. این ایندکس به گوگل اجازه داد تا نه تنها وجود واژه‌ها، بلکه ترتیب و نزدیکی آن‌ها را نیز در نظر بگیرد. این قابلیت باعث شد دقت جستجوی عبارات دقیق در گوگل ۴۵٪ افزایش یابد و رضایت کاربران به شکل چشمگیری بهبود یابد.

در این فصل، تمرکز بر ساخت ایندکس‌های غیرموقعیتی (nonpositional) بود. اما در بسیاری از کاربردها، نیاز به ایندکس‌های موقعیتی (positional) داریم که اطلاعات مکانی واژه‌ها در سند را نیز ذخیره می‌کنند.

شما در تیم جستجوی علمی گوگل اسکالر کار می‌کنید و باید سیستم جستجوی مقالات علمی را بهینه کنید. با توجه به اینکه:

- محققان اغلب به دنبال "الگوریتم‌های یادگیری عمیق برای تصویربرداری پزشکی" جستجو می‌کنند
- نتایج باید بر اساس ارتباط معنایی و جدید بودن مقالات رتبه‌بندی شوند
- جستجوی عبارات دقیق مانند "یادگیری عمیق برای تصویربرداری پزشکی" باید نتایج دقیقی داشته باشد

چگونه می‌توانید با استفاده از ایندکس موقعیتی، این نیازها را برآورده کنید؟

در ایندکس موقعیتی، هر posting به صورت زیر است:

$(termID, docID, (position_1, position_2, \dots))$

الگوریتم‌های BSBI، SPIMI و MapReduce با اندکی تغییر می‌توانند برای ایندکس‌های موقعیتی نیز استفاده شوند. تنها تفاوت، نگهداری موقعیت‌ها در کنار docIDهاست.

در ایندکس‌های قبلی، postings list بر اساس docID مرتب می‌شدند. این ساختار برای فشردسازی مناسب است، چون فاصله‌های بین docIDها کوچک‌تر می‌شوند و می‌توان آن‌ها را بهتر encode کرد.

اما در سیستم‌های **ranked retrieval** (فصل‌های ۶ و ۷)، بهتر است postings بر اساس **impact score** مرتب شوند تا بتوان جست‌وجو را زودتر متوقف کرد. در این حالت، اسناد با امتیاز بالا در ابتدای لیست قرار می‌گیرند.

مثال واقعی: ایندکس موقعیتی در گوگل

گوگل در سال ۲۰۱۰ با چالش بهبود جستجوی عبارات دقیق روبرو بود. جستجوی "رستوران ایتالیایی در نزدیکی دانشگاه تهران" نتایج نامرتب می‌داد، زیرا ایندکس فقط وجود واژه‌ها را بررسی می‌کرد، نه ترتیب آن‌ها را.

راه حل گوگل، توسعه ایندکس موقعیتی بود که اطلاعات دقیق محل هر واژه در سند را ذخیره می‌کرد. این ایندکس به گوگل اجازه داد تا جستجوی عبارات دقیق را با دقت بسیار بالا انجام دهد. برای مثال، جستجوی "رستوران ایتالیایی در نزدیکی دانشگاه تهران" فقط رستوران‌هایی را برمی‌گرداند که این عبارت دقیقاً به همین ترتیب در متن آمده است.

این قابلیت باعث شد دقت جستجوی عبارات دقیق در گوگل ۶۰٪ افزایش یابد و کاربران بتوانند به سرعت اطلاعات دقیق مورد نیاز خود را پیدا کنند.

در ایندکس مرتب بر اساس impact، درج سند جدید پیچیده‌تر است چون ممکن است در وسط لیست قرار گیرد. در حالی که در ایندکس docID-sorted، سند جدید همیشه به انتهای لیست اضافه می‌شود.

در محیط‌های سازمانی، **امنیت** یکی از دغدغه‌های اصلی است. کاربران نباید بتوانند اسنادی را ببینند که مجاز به دسترسی به آن‌ها نیستند. حتی وجود یک سند می‌تواند اطلاعات حساس تلقی شود.

مثال واقعی: کنترل دسترسی در مایکروسافت

در سال ۲۰۱۶، مایکروسافت با چالش امنیتی بزرگ در سیستم داخلی خود روبرو بود: یک کارمند سابق توانسته بود به اسناد محرمانه دسترسی پیدا کند.

راه حل مایکروسافت، پیاده‌سازی سیستم کنترل دسترسی مبتنی بر Access Control Lists (ACL) بود. هر سند با لیستی از کاربران مجاز همراه بود و ایندکس بر اساس کاربران ساخته می‌شد، نه اسناد.

این رویکرد به مایکروسافت اجازه داد تا دسترسی به اسناد را با دقت بالا مدیریت کند، در حالی که سرعت بازیابی کاهش چندانی پیدا کرد. این سیستم باعث شد ۹۵٪ تلاش‌های غیرمجاز برای دسترسی به اسناد محرمانه شناسایی و مسدود شود، در حالی که کاربران مجاز همچنان به سرعت به اسناد مورد نیاز خود دسترسی داشتند.

برای مدیریت دسترسی، از **Access Control Lists (ACL)** استفاده می‌شود. هر سند با لیستی از کاربران مجاز همراه است. این ماتریس به‌صورت زیر نمایش داده می‌شود:

Users	Documents	Access
u_1	d_1	1
u_1	d_2	0
u_2	d_1	1
u_2	d_2	1

در زمان جست‌وجو، لیست نتایج متنی با لیست دسترسی کاربر اشتراک گرفته می‌شود. اما نگهداری این ایندکس دشوار است، چون تغییرات در مجوزها باید به‌سرعت اعمال شوند.

در عمل، اغلب مجوزها در زمان جست‌وجو از سیستم فایل خوانده می‌شوند، که سرعت بازیابی را کاهش می‌دهد اما دقت امنیتی را افزایش می‌دهد.

در فصل‌های بعدی، وارد بحث‌های فشرده‌سازی، رتبه‌بندی، و مدل‌های پیشرفته‌تر بازیابی اطلاعات خواهیم شد.

فصل ۴: ساخت ایندکس

در این فصل، با فرآیند ساخت ایندکس معکوس (inverted index) که قلب تپنده موتورهای جستجو است، آشنا شدیم. این فرآیند که به آن «ایندکس‌سازی» (indexing) نیز می‌گویند، به ما اجازه می‌دهد تا از میان میلیاردها سند، اطلاعات مرتبط را در کسری از ثانیه پیدا کنیم.

- ابتدا با محدودیت‌های سخت‌افزاری که بر طراحی الگوریتم‌های ایندکس‌سازی تأثیر می‌گذارند، آشنا شدیم. این همان تفاوتی است که بین جستجو در کامپیوتر شخصی شما و سرورهای قدرتمند گوگل وجود دارد.
- سپس الگوریتم‌های مختلف ساخت ایندکس را بررسی کردیم: از روش‌های تک‌ماشینه برای مجموعه‌های ساکن گرفته تا سیستم‌های توزیع‌شده برای ایندکس‌سازی کل وب.
- در نهایت، با روش‌های مدیریت ایندکس‌های پویا که برای مجموعه‌هایی که دائماً در حال تغییر هستند (مانند شبکه‌های اجتماعی یا خبرگزاری‌ها) ضروری هستند، آشنا شدیم.

بیایید این مفاهیم را با چند مثال واقعی مقایسه کنیم:

♦ **اصول سخت‌افزاری و تأثیر بر ایندکس‌سازی:** تصور کنید می‌خواهید کتابخانه‌ای با میلیون‌ها کتاب را سازماندهی کنید. اگر هر کتاب را در یک قفسه تصادفی قرار دهید، پیدا کردن یک کتاب خاص غیرممکن خواهد بود. اما اگر کتاب‌ها را بر اساس موضوع و سپس ترتیب الفبا مرتب کنید، این کار بسیار سریع‌تر خواهد شد. در دنیای دیجیتال، حافظه اصلی (RAM) مانند یک میز کار کوچک و سریع است، در حالی که دیسک سخت مانند انباری بزرگ اما کند عمل می‌کند. گوگل با استفاده از هزاران

سرور با حافظه‌های سریع، بخش‌هایی از ایندکس را که بیشتر استفاده می‌شوند در حافظه اصلی نگه می‌دارد تا به نتایج سریع دسترسی پیدا کند.

♦ **ایندکس‌سازی مبتنی بر مرتب‌سازی مسدود (Blocked Sort-Based Indexing):** این روش مانند سازماندهی یک کتابخانه بزرگ با کمک چندین کتابدار است. هر کتابدار بخشی از کتاب‌ها را مرتب می‌کند (ایجاد بلوک‌ها)، سپس یک سرپرست تمام این بخش‌های مرتب‌شده را با هم ترکیب می‌کند (ادغام بلوک‌ها). گوگل از این روش برای ایندکس‌سازی بخش‌هایی از وب استفاده می‌کند که به طور منظم تغییر نمی‌کنند. مزیت اصلی این روش، کارایی بالا در مجموعه‌های بزرگ است، چون هر مرحله به حافظه محدودی نیاز دارد.

♦ **ایندکس‌سازی تک‌گذر در حافظه (Single-Pass In-Memory Indexing):** این روش مانند کار یک کتابدار بسیار باهوش است که همزمان با خواندن هر کتاب، آن را دسته‌بندی کرده و در قفسه مناسب قرار می‌دهد. این روش از روش قبلی کارآمدتر است، چون نیازی به ذخیره تمام اطلاعات و سپس مرتب‌سازی آن‌ها ندارد. شرکت‌هایی مانند دیجی‌کالا از این روش برای ایندکس‌سازی محصولات جدید خود استفاده می‌کنند تا بتوانند به سرعت به کالاهای تازه وارد شده پاسخ دهند.

♦ **ایندکس‌سازی توزیع‌شده (Distributed Indexing):** برای ایندکس‌سازی کل وب، حتی قدرتمندترین سرورها هم کافی نیستند. گوگل از معماری MapReduce استفاده می‌کند که مانند کار هزاران کتابدار است که همزمان روی بخش‌های مختلف کتابخانه کار می‌کنند. در این معماری، هر کتابدار (گره) بخشی از کتاب‌ها را پردازش کرده و نتایج را به سرپرست اصلی (گره اصلی) گزارش می‌دهد. این روش به گوگل اجازه می‌دهد تا در روز میلیون‌ها صفحه جدید را ایندکس کند و به کاربران امکان جستجوی اطلاعات به‌روز را بدهد.

♦ **ایندکس‌سازی پویا (Dynamic Indexing):** شبکه‌های اجتماعی مانند توییتر یا اینستاگرام دائماً در حال تغییر هستند و اطلاعات جدیدی تولید می‌کنند. برای این نوع مجموعه‌ها، ایندکس‌سازی پویا ضروری است. این روش مانند یک کتابخانه است که همیشه یک بخش «کتاب‌های جدید» دارد که به سرعت در دسترس قرار می‌گیرند و سپس به بخش‌های اصلی کتابخانه منتقل می‌شوند. توییتر از این روش برای ایندکس‌سازی توییت‌های جدید استفاده می‌کند تا کاربران بتوانند به سرعت به آخرین اخبار و رویدادها دسترسی پیدا کنند.

در ادامه، با مفاهیم پیشرفته‌تری مانند:

- ایندکس‌های مکانی (positional indexes) برای جستجوی دقیق عبارات - روشی که به گوگل اجازه می‌دهد عبارت «قیمت گوشی سامسونگ» را از «قیمت گوشی و سامسونگ» تشخیص دهد
- ایندکس‌های امنیتی برای کنترل دسترسی - تکنیکی که به شرکت‌ها اجازه می‌دهد اطلاعات محرمانه را فقط از طریق کاربران مجاز قابل جستجو کنند
- فشرده‌سازی ایندکس‌ها - روشی که به گوگل اجازه می‌دهد میلیاردها صفحه را در فضای محدود ذخیره کند
- بهینه‌سازی ایندکس‌ها برای جستجوی رتبه‌بندی‌شده - تکنیکی که تعیین می‌کند کدام نتایج باید در بالای صفحه نمایش داده شوند

آشنا خواهیم شد – که همگی بر همین پایه‌های ساخت ایندکس بنا شده‌اند.

End of Chapter Exercises

مسئله 4.1: تفاوت زمان دسترسی (Access Time Difference)

چرا در طراحی سیستم‌های بازیابی اطلاعات، خواندن داده‌ها از دیسک به صورت **پیوسته (Sequential)** بسیار سریع‌تر از خواندن **تصادفی (Random)** است؟ نقش **زمان Seek** در این تفاوت چیست؟

مسئله 4.2: گلوگاه سیستم (System Bottleneck)

در فرآیند ساخت نمایه، سیستم عامل از تکنیک **Caching** استفاده می‌کند. توضیح دهید که این مکانیزم چگونه به کاهش گلوگاه‌های **I/O** دیسک کمک می‌کند و چرا **RAM** نقش حیاتی در این مرحله دارد؟

مسئله 4.3: هدف الگوریتم BSBI (Purpose of BSBI)

دلیل اصلی تولد الگوریتم **Blocked Sort-Based Indexing** چیست؟ چرا نمی‌توانیم تمام اسناد یک **پیکره (Corpus)** بزرگ را به صورت یکجا در حافظه اصلی نمایه‌سازی کنیم؟

مسئله 4.4: جریان داده در BSBI (Data Flow in BSBI)

در الگوریتم **BSBI**، گام‌های اصلی تبدیل یک بلوک متنی به یک بلوک نمایه روی دیسک را به ترتیب نام برده و هر کدام را در یک سطر توضیح دهید.

مسئله 4.5: مزیت اصلی SPIMI (Main Advantage of SPIMI)

الگوریتم **Single-pass in-memory indexing** چه برتری قابل توجهی نسبت به **BSBI** دارد که باعث می‌شود سرعت تولید بلوک‌های نمایه افزایش یابد؟

مسئله 4.6: مدیریت واژه‌نامه در SPIMI (Dictionary Management in SPIMI)

در **SPIMI**، نگاشت (**Mapping**) بین **Term** و **TermID** وجود ندارد. این الگوریتم چگونه کلمات جدید را به واژه‌نامه (**Dictionary**) اضافه می‌کند؟

مسئله 4.7: توزیع کلمات در برابر اسناد (Term vs. Document Partitioning)

در نمایه‌سازی توزیع‌شده دو معماری اصلی **Term-partitioned** و **Document-partitioned** وجود دارد. تعریف مختصر هر کدام چیست؟

مسئله 4.8: نقش Map و Reduce (Role of Map and Reduce)

در فریم‌ورک **MapReduce** برای ساخت نمایه، وظیفه اصلی تابع **Map** چیست و خروجی آن به چه شکلی به تابع **Reduce** تحویل داده می‌شود؟

مسئله 4.9: نمایه‌سازی پویا و نمایه کمکی (Dynamic Indexing and Auxiliary Index)

چرا در سیستم‌هایی که اسناد مرتباً تغییر می‌کنند (مثل خبرگزاری‌ها)، به جای بازسازی کل نمایه اصلی، از یک نمایه کمکی (Auxiliary Index) در حافظه اصلی استفاده می‌شود؟

مسئله 4.10: نمایه‌های موقعیتی (Positional Indexes)

نمایه موقعیتی چه تفاوتی با نمایه معکوس استاندارد دارد و چه نوع کوئری‌هایی (Queries) را پشتیبانی می‌کند؟

مسئله 4.11: محاسبه زمان خواندن از دیسک (Disk Read Time Calculation)

یک هارد دیسک با زمان متوسط Seek برابر با 5 ms و سرعت انتقال 50 MB/s داریم. محاسبه کنید زمان لازم برای خواندن 100 MB داده در دو حالت زیر چقدر است؟

الف) یک فایل یکپارچه 100 MB

ب) 1000 فایل پراکنده 100 KB

مسئله 4.12: جدول وارون‌سازی در BSBI (Inversion Table in BSBI)

جدول زیر شامل توکن‌های استخراج شده از یک بلوک در الگوریتم BSBI است. فرآیند مرتب‌سازی (Sorting) و ادغام (Grouping) این جدول را انجام داده و جدول نهایی Posting List را رسم کنید.

DocID	TermID
1	45
2	12
1	12
3	45
2	45

مسئله 4.13: فاز ادغام بلوک‌ها (Block Merge Phase)

فرض کنید در BSBI، چهار بلوک مرتب‌شده B1, B2, B3, B4 روی دیسک داریم. با رسم یک گراف درختی نشان دهید که چگونه ساختار N-way Merge این بلوک‌ها را همزمان می‌خواند و نمایه نهایی را می‌سازد.

مسئله 4.14: پر شدن حافظه در SPIMI (Memory Exhaustion in SPIMI)

سرور شما در حال اجرای SPIMI است. ناگهان حافظه اختصاص داده شده پر می‌شود. سیستم دقیقاً در این لحظه (Memory Full) چه واکنشی نشان می‌دهد و چه فایل‌هایی تولید می‌شود؟ مراحل را گام به گام بنویسید.

مسئله 4.15: مقایسه BSBI و SPIMI (BSBI vs. SPIMI Comparison)

جدول زیر را در برگه خود رسم کرده و خانه‌های خالی را با مقایسه دقیق دو الگوریتم پر کنید:

ویژگی	BSBI	SPIMI
نیاز به دیکشنری سراسری (Term to ID)	؟	؟
سرعت پردازش اولیه کلمات	؟	؟
ساختار Posting List در حافظه	آرایه با طول ثابت	؟

مسئله 4.16: جریان داده در MapReduce (Data Flow in MapReduce)

یک کلاستر شامل 3 گره Map و 2 گره Reduce داریم. با استفاده از یک گراف جریان داده (Data Flow Graph)، نشان دهید توکن‌های استخراج شده از اسناد 1 تا 10 چگونه بین این گره‌ها توزیع (Shuffle) و جمع‌ی می‌شوند.

مسئله 4.17: گلوگاه‌های تجزیه‌گر (Parser Bottlenecks)

در نمایه‌سازی توزیع‌شده، گره‌های Parser (که فاز Map را انجام می‌دهند) گاهی دچار افت سرعت می‌شوند. سه دلیل اصلی این گلوگاه (مرتبط با خواندن دیسک یا شبکه) را تحلیل کنید.

مسئله 4.18: ردگیری ادغام لگاریتمی (Tracing Logarithmic Merging)

در روش ادغام لگاریتمی، فرض کنید اندازه نمایه کمکی $n = 1000$ است. وقتی 3500 سند پردازش شده باشند، وضعیت نمایه‌های روی دیسک (یعنی $Z_0, Z_1, \dots$) چگونه است؟ ظرفیت هر Z_i را مشخص کنید.

مسئله 4.19: مدیریت حذف اسناد (Handling Document Deletion)

در یک موتور جستجوی پویا، برخی صفحات وب حذف (Dead links) می‌شوند. در سیستم نمایه‌سازی پویا، حذف فیزیکی این اسناد از فایل نمایه بسیار پرهزینه است. راهکار جایگزین و استاندارد برای مدیریت حذف (Deletion) این اسناد چیست؟

مسئله 4.20: نمایه‌های کنترل دسترسی (Access Control Indexes)

در سازمان‌ها، برخی اسناد محرمانه هستند و کاربران عادی نباید در نتایج جستجو آن‌ها را ببینند. چگونه از مفاهیم نمایه‌سازی برای ایجاد Access Control Lists (ACL) در موتور جستجوی سازمانی استفاده می‌شود؟

مسئله 4.21: نمایه‌های چندسطحی (Tiered Indexes)

سرورهای یک موتور جستجوی وب قادر نیستند کل نمایه وب را در RAM نگه دارند. چگونه با استفاده از مفهوم Tiered Indexing (نمایه‌های سطح‌بندی شده)، کوئری‌های پرتکرار سریع‌تر پاسخ داده می‌شوند؟

مسئله 4.22: انتخاب الگوریتم مناسب (Choosing the Right Algorithm)

اگر شما مدیر یک استارت‌آپ باشید که یک موتور جستجو برای لاگ‌های حجیم سرور (بدون نیاز به آپدیت پویا) می‌سازد، بین SPIMI روی یک سرور قدرتمند و MapReduce روی ۱۰ سرور ضعیف، کدام را و چرا انتخاب می‌کنید؟

مسئله 4.23: تاثیر حافظه نهان دیسک (Impact of Disk Caching)

چگونه بافرهای سیستم عامل (OS Buffers) می‌توانند فرض ما در مورد هزینه دسترسی تصادفی به دیسک را در الگوریتم ادغام BSBI نقض کنند؟ راهکار برنامه‌نویس برای جلوگیری از Thrashing دیسک در این حالت چیست؟

مسئله 4.24: پیچیدگی مرتب‌سازی خارجی (External Sorting Complexity)

ثابت کنید که در الگوریتم BSBI، اگر اندازه بلاک حافظه M و حجم کل داده‌ها T باشد، تعداد پاس‌های خواندن/نوشتن در دیسک در فاز ادغام متناسب با $\log_{M/B}(T/M)$ است (که در آن B اندازه بافر خواندن است).

مسئله 4.25: تخصیص پویای حافظه در SPIMI (Dynamic Memory Allocation in SPIMI)

در الگوریتم SPIMI، از آنجایی که Posting List ها به صورت پویا رشد می‌کنند، مشکل Fragment شدن حافظه (Memory Fragmentation) رخ می‌دهد. این مشکل چیست و چگونه با دو برابر کردن طول آرایه (Doubling Array Size) مدیریت می‌شود؟

مسئله 4.26: چسبندگی و عدم توازن داده‌ها (Data Skewness in MapReduce)

در فاز Reduce، کلماتی مانند «است» یا «به» به یک Reducer خاص ارسال می‌شوند در حالی که کلمات نادر به Reducer دیگر. این عدم توازن (Data Skewness) باعث می‌شود یک سرور ساعت‌ها درگیر بماند در حالی که بقیه بیکارند. به عنوان معمار سیستم، چه راهکاری برای حل این مشکل پیشنهاد می‌دهید؟

مسئله 4.27: تحلیل ریاضی ادغام لگاریتمی (Mathematical Analysis of Logarithmic Merging)

در روش نمایه‌سازی پویا با ادغام لگاریتمی، ثابت کنید که پیچیدگی زمانی هزینه جستجو (Query Time) در بدترین حالت $O(\log(T/n))$ است (نمایه‌های روی دیسک)، اما هزینه نگهداری نمایه بسیار پایین‌تر از بازسازی کامل آن است.

مسئله 4.28: حجم نمایه‌های موقعیتی (Size of Positional Indexes)

تحلیل کنید که چرا اضافه کردن Position کلمات به نمایه، حجم نمایه معکوس را به شدت (بین ۲ تا ۴ برابر) افزایش می‌دهد؟ آیا فشرده‌سازی این اطلاعات در فاز ساخت نمایه (Index Construction) تفاوتی در منطق SPIMI ایجاد می‌کند؟

مسئله 4.29: پیاده‌سازی پایتون SPIMI (Python Implementation of SPIMI) ★

با استفاده از زبان پایتون، تابع `build_spimi_block(token_stream, memory_limit)` را پیاده‌سازی کنید.

شروط:

- ورودی یک استریم از تاپل‌های (term, doc_id) است.

- شما باید از **دیکشنری‌های پایتون** استفاده کنید.

- **لیست‌ها** باید به صورت **پویا** بزرگ شوند.

- زمانی که **تعداد توکن‌ها** به `memory_limit` رسید، **کلیدها** را **سورت** (مرتب‌سازی) کرده و **ساختار نمایه معکوس** این بلوک را `return` کنید.

کد شما باید **کاملاً بهینه** (`Clean Code`) باشد.

مسئله 4.30: **کدنویسی توابع Map و Reduce(Coding Map and Reduce Functions)** ★

فرض کنید می‌خواهیم الگوریتم **نمایه‌سازی** را روی `Hadoop` **پیاده‌سازی** کنیم. **شبه‌کد** (`Pseudocode`) **دقیق** یا **کد پایتون** برای دو تابع `Map(docID, document_text)` و `Reduce(term, list_of_docIDs)` بنویسید.

کد شما باید به گونه‌ای باشد که **Term Frequency** (**تعداد تکرار کلمه در آن سند**) را نیز در تابع `Map` **محاسبه** کرده و به عنوان **خروجی** به `Reduce` پاس دهد.

مسئله 4.31: **طراحی سیستم تحمل‌پذیر خطا (Fault-Tolerant System Design)** ★

در یک سیستم `Distributed Indexing` بر بستر `MapReduce` با 500 **گره**، `Master Node` **مسئول هماهنگی** است. در حین اجرای فاز `Map`، 5 **گره** به دلیل **قطع شبکه** آفلاین می‌شوند.

طراحی سیستمی: یک **نمودار جریان** (`Flowchart`) متنی رسم کنید که نشان دهد `Master Node` چگونه:

1. **خرابی** را تشخیص می‌دهد.
2. **کارهای ناتمام** را به **گره‌های دیگر** `Assign` می‌کند.
3. **از تکراری شدن داده‌ها** در **نمایه نهایی** **جلوگیری** می‌کند.

مسئله 4.32: **گراف انتقال در ادغام لگاریتمی پیشرفته (Transition Graph in Advanced Logarithmic Merging)** ★

در یک سیستم **بسیار پویا**، ما از **ادغام لگاریتمی** استفاده می‌کنیم. فرض کنید $n = 1$ (**اندازه نمایه کمکی**، صرفاً برای سادگی مسئله). با استفاده از **ریاضیات گسسته**، یک «**گراف انتقال حالت**» (`State Transition Graph`) رسم کنید (یا توصیف کنید) که نشان دهد برای **درج ۸ سند** ($T = 1$ تا $T = 8$)، **بیت‌های متناظر با اندیس‌های دیسک** (Z_0, Z_1, Z_2, Z_3) چگونه مانند یک **شمارنده باینری** (`Binary Counter`) **تغییر حالت** می‌دهند. نشان دهید دقیقاً در کدام حالت **بیشترین عملیات I/O** روی **دیسک** اتفاق می‌افتد.

مسئله 4.33: **نمایه‌ساز میان‌ستاره‌ای** ★

یک **کاوشگر فضایی** در حال ارسال جریان بی‌پایانی از **داده‌های متنی** و **لاگ‌های تله‌متری** به **ایستگاه زمینی** است. ایستگاه زمینی دارای **حافظه RAM** بسیار محدود اما آرایه‌ای از **نوارهای مغناطیسی** با **ظرفیت نامحدود** (اما با **دسترسی ترتیبی** کند) است. **حجم واژگان** (`Vocabulary Size`) به دلیل تولید **کدهای خطای جدید**، از پیش مشخص نیست.

به عنوان **مهندس داده**، یک **معماری مبتنی بر SPIMI** **طراحی** کنید که بتواند این **جریان داده** (`Data Stream`) را به صورت

درنگ‌تایذیر (Real-time) نمایه‌سازی کند و همزمان از مکانیزم Auxiliary Index برای جستجوی فوری لاگ‌های 10 دقیقه اخیر استفاده کند. گلوگاه اصلی این معماری هنگام انتقال داده‌ها به نوار مغناطیسی کجاست و چگونه آن را بهینه می‌کنید؟

مسئله 4.34: چالش تحمل خطا در سیستم‌های توزیع‌شده ★

در حین ساخت نمایه برای 10 میلیارد صفحه وب با استفاده از معماری MapReduce، دقیقاً در اواسط فاز Reduce، سه تا از Worker Node ها دچار قطعی برق می‌شوند.

الف) گره Master چگونه متوجه این خرابی می‌شود؟

ب) فرآیند بازیابی (Recovery) وظایف این سه گره بدون نیاز به اجرای مجدد کل فاز Map چگونه انجام می‌شود؟ با رسم فلوچارت ذهنی یا شبه‌کد، منطق Fault Tolerance را در این سناریو شرح دهید.

مسئله 4.35: سوال کدنویسی الگوریتم SPIMI ★

برای اثبات درک عمیق از ساختار داده‌ها در حافظه، با استفاده از زبان پایتون، منطق اصلی ساخت یک بلوک در الگوریتم SPIMI را کدنویسی کنید.

شرایط و محدودیت‌ها:

- یک تابع بنویسید که یک آرایه از تاپل‌های ورودی به شکل (doc_id, token) را دریافت کند.
- دیکشنری باید به صورت پویا در حافظه ساخته شود. لیست Posting ها باید بدون استفاده از متدهای مرتب‌سازی داخلی برای هر کلمه در لحظه آپدیت شود.
- زمانی که تعداد کل توکن‌های پردازش شده به یک threshold رسید، دیکشنری باید بر اساس حروف الفبای Term ها مرتب شده و به عنوان خروجی یک بلوک (مثلاً چاپ در کنسول) برگردانده شود.

کد شما باید بسیار تمیز (Clean Code) و بهینه‌سازی شده باشد.

مسئله 4.36: تحلیل پیچیدگی ادغام لگاریتمی و نمایه‌های موقعیتی (Complexity Analysis of Logarithmic Merging and Positional Indexes) ★

در روش ادغام لگاریتمی (Logarithmic Merging) برای نمایه‌سازی پویا، ثابت کنید که اگر تعداد کل توکن‌های پردازش شده برابر با T باشد، هزینه ساخت و نگهداری نمایه در بدترین حالت دارای پیچیدگی زمانی $O(T \log(T/n))$ است (که در آن n ظرفیت حافظه کمکی است).

سپس تحلیل کنید که اگر بخواهیم Position کلمات (برای پشتیبانی از Phrase Queries در بخش ۴.۶) را هم در این ساختار پویا نگهداری کنیم، نرخ رشد حجم فایل‌های ادغامی در سطح Z_i دیسک چه تغییری می‌کند و آیا ادغام لگاریتمی همچنان توجیه اقتصادی دارد؟

مسئله 4.37: شبیه‌سازی دقیق بلوک‌های حافظه در SPIMI(SPIMI Memory Block Simulation) ★

سرور شما در حال اجرای الگوریتم SPIMI برای نمایه‌سازی متون است. محدودیت حافظه اصلی (RAM) برای ذخیره دیکشنری و Posting List ها دقیقاً ۶۰ بایت تعیین شده است. فرض کنید هر کاراکتر ۱ بایت، هر DocID دقیقاً ۴ بایت و سربرار (Overhead) ساختار داده دیکشنری صفر در نظر گرفته می‌شود.

جدول زیر جریان ورودی توکن‌ها به ترتیب زمان رسیدن را نشان می‌دهد:

زمان (t)	توکن (Token)	شناسه سند (DocID)
t_1	web	10
t_2	data	10
t_3	web	12
t_4	search	15
t_5	data	15
t_6	graph	18
t_7	web	18

الف) با ردیابی دقیق مصرف حافظه در هر مرحله (t_1 تا t_7)، مشخص کنید دقیقاً در چه زمانی (کدام t) حافظه پر شده و سیستم مجبور به انجام عملیات **Write** روی دیسک (تولید بلوک اول) می‌شود؟

ب) محتوای دقیق بلوک اولی که روی دیسک نوشته می‌شود را به فرمت یک **Posting List** مرتب‌شده نشان دهید.

مسئله 4.38: عدم توازن بار و کشف تقلب در نمایه توزیع‌شده (Load Imbalance & Plagiarism Detection)

برای بررسی و کشف تقلب در کدهای پایتون ارسالی ۱۰۰ دانشجوی، یک سیستم **MapReduce** طراحی کرده‌اید که نام متغیرهای استفاده شده در کدها را به عنوان توکن نمایه‌سازی می‌کند تا کدهای مشابه را بیابد. سیستم دارای ۳ گره **Reducer** است (R_1, R_2, R_3).

جدول زیر فرکانس کل (**Total Frequency**) برخی متغیرهای استخراج شده از کل فایل‌ها را نشان می‌دهد:

نام متغیر (Token)	تعداد کل رخدادها در تمام کدها
i	85,000
count	12,000
temp	11,500
data_array	300
flag	45,000
result	8,200

سیستم برای توزیع کلمات بین **Reducer** ها از تابع هش ساده ($H(\text{Token}) = \text{Length}(\text{Token}) \pmod 3$) استفاده می‌کند.

الف) نشان دهید هر توکن به کدام **Reducer** ارسال می‌شود و مجموع بار پردازشی (تعداد کل رخدادهای ارسالی) برای هر **Reducer** چقدر خواهد بود؟

ب) با تحلیل اعداد به دست آمده، پدیده گلوگاه (Bottleneck) در این کلاستر را اثبات کنید.

ج) یک تابع هش جایگزین یا مکانیزم توزیع سفارشی (Custom Partitioner) طراحی کنید که این مشکل عدم توازن بار (Data Skewness) را در فاز Reduce برطرف کند.

مسئله 4.39: چالش پیاده سازی ادغام چندگانه برای پلتفرم دآوری (Multi-way Merge Implementation) ★ (Challenge)

در فاز نهایی الگوریتم BSBI، شما K بلوک مرتب شده روی دیسک دارید و باید آن ها را در یک پاس (Single Pass) ادغام کنید. برای جلوگیری از پر شدن حافظه، نمی توانید همه بلوک ها را همزمان در RAM بارگذاری کنید. با استفاده از زبان پایتون، تابع `merge_k_blocks(block_ha)` را بنویسید که لیستی از مسیر فایل های متنی (هر فایل نشان دهنده یک بلوک از زوج های TermID:DocID مرتب شده) را دریافت کند.

شرایط و محدودیت ها:

- الزام اکید: شما حق استفاده از توابع آماده مانند `itertools.merge` یا خواندن کل فایل ها در یک لیست و `sort()` کردن آن ها را ندارید.
- باید از ساختار داده Min-Heap (با استفاده از کتابخانه `heapq`) برای خواندن خط به خط فایل ها استفاده کنید.
- کد باید از نظر مصرف حافظه کاملاً بهینه (Clean Code) باشد. از نام گذاری خوانا مانند `vouroudi_ha` برای ورودی ها استفاده کنید.
- خروجی باید یک فایل واحد به نام `final_index.txt` باشد که نمایه ادغام شده نهایی را در بر دارد.

مسئله 4.40: تحلیل ریاضی و هزینه I/O در ادغام لگاریتمی (I/O Cost Analysis in Logarithmic Merging) ★

یک سیستم نمایه سازی پویا با استفاده از استراتژی ادغام لگاریتمی کار می کند. اندازه نمایه کمکی (موجود در RAM) برابر با $n = 10,000$ توکن است. در حال حاضر سیستم در وضعیت پایدار قرار دارد و جدول زیر وضعیت نمایه های روی دیسک (بخش های Z_i) را نشان می دهد:

سطح نمایه (Z_i)	ظرفیت اسمی ($2^i \times n$)	وضعیت فعلی	حجم واقعی داده موجود
Z_0	10,000	پُر (Full)	۱۰,۰۰۰ توکن
Z_1	20,000	پُر (Full)	۲۰,۰۰۰ توکن
Z_2	40,000	خالی (Empty)	۰
Z_3	80,000	پُر (Full)	۸۰,۰۰۰ توکن

فرض کنید دقیقاً ۱۰,۰۰۰ توکن جدید وارد سیستم می شود و نمایه کمکی در RAM پُر می شود.

الف) با رسم یک فلوچارت یا دنباله عملیات، نشان دهید چه ادغام هایی (Merges) رخ می دهد تا سیستم دوباره به یک حالت پایدار برسد.

ب) وضعیت نهایی جدول بالا پس از این عملیات چگونه خواهد بود؟

ج) در این فرآیند به روزرسانی خاص، مجموعاً چند توکن از روی دیسک خوانده شده و چند توکن روی دیسک نوشته می شود؟ (هزینه دقیق I/O را محاسبه کنید).

مسئله 4.41: طراحی معماری نمایه‌سازی موقعیتی برای داده‌های حسگرها (Positional Indexing Architecture) ★ (for Sensor Data)

شما در حال طراحی یک موتور جستجوی بسیار دقیق برای لاگ‌های حجیم تجهیزات پوشیدنی سلامت (مانند ساعت‌های هوشمند) هستید. این سیستم باید علاوه بر خود واژه‌های لاگ (مثل **HeartRate_High**)، موقعیت دقیق زمانی آن‌ها (**Position**) را نیز ذخیره کند تا امکان اجرای کوئری‌های فاصله‌دار (**Proximity Queries**) فراهم شود.

جدول زیر بخشی از لاگ خام را نشان می‌دهد:

شناسه دستگاه (DocID)	توکن سنسور	موقعیت زمانی (تیک‌ثانیه)
D_{101}	Temp_Spike	10050
D_{101}	Temp_Spike	10058
D_{101}	Temp_Spike	10090
D_{102}	BP_Drop	450

اضافه کردن موقعیت زمانی (**Position**) حجم نمایه را در الگوریتم **SPIMI** به شدت افزایش می‌دهد و باعث می‌شود حافظه **RAM** به سرعت پر شده و تعداد بلوک‌های روی دیسک (و به تبع آن هزینه فاز **Merge**) به صورت تصاعدی بالا برود.

الف) اگر بخواهیم بلوک **SPIMI** را در حافظه بسازیم، ساختار داده دقیق یک رکورد (شامل **DocID**، **Term Frequency** و **Positions**) را به صورت شماتیک رسم کنید.

ب) برای جلوگیری از انفجار حافظه **RAM** پیش از نوشتن بلوک روی دیسک، یک تکنیک فشرده‌سازی در سطح الگوریتم پیشنهاد دهید (راهنمایی: به اختلاف اعداد فکر کنید) و نشان دهید این تکنیک روی رکوردهای دستگاه D_{101} در جدول بالا چگونه اعمال می‌شود و چقدر در بایت‌های مصرفی صرفه‌جویی می‌کند.

پاسخنامه مسئله 4.1: تفاوت زمان دسترسی در دیسک‌ها (Sequential vs. Random)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک دلایل مکانیکی و فیزیکی اختلاف سرعت در الگوهای مختلف خواندن داده‌ها از روی دیسک سخت (**HDD**) و بررسی تأثیر مستقیم «زمان جستجو» (**Seek Time**) بر این فرآیند است. در سیستم‌های بازیابی اطلاعات (**Information Retrieval - IR**) که با حجم عظیمی از داده‌ها (مانند نمایه‌های معکوس) سروکار داریم، درک این تفاوت برای بهینه‌سازی ساختار ذخیره‌سازی و افزایش سرعت پاسخ‌گویی به پرس‌وجوهای (**Queries**) کاربران امری حیاتی است.

بخش 2) مفاهیم پایه و پیش‌نیاز

برای حل این مسئله، ابتدا باید با مکانیزم فیزیکی دیسک‌ها و فرمول محاسبه‌ی زمان دسترسی (**Access Time**) آشنا شویم. زمان کل لازم برای خواندن یک قطعه داده از روی دیسک از مجموع سه عامل اصلی تشکیل می‌شود:

1. زمان جستجو (Seek Time): مدت زمانی که طول می‌کشد تا بازوی مکانیکی دیسک، هد خواندن/نوشتن

(Read/Write Head) را روی شیار (Track) مورد نظر قرار دهد.

2. تأخیر چرخشی (Rotational Latency): مدت زمانی که طول می‌کشد تا دیسک بچرخد و قطاع (Sector) حاوی

داده‌ها، دقیقاً زیر هد قرار گیرد.

3. زمان انتقال (Transfer Time): زمان واقعی خواندن داده‌ها از روی دیسک و انتقال آن‌ها به حافظه اصلی (RAM).

رابطه ریاضی این فرآیند به شکل زیر بیان می‌شود:

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{latency}} + T_{\text{transfer}}$$

بخش 3) حل گام‌به‌گام مسئله

گام اول: تحلیل فرآیند خواندن تصادفی (Random Access)

در خواندن تصادفی، داده‌های مورد نیاز در بخش‌های مختلف و پراکنده‌ای از دیسک قرار دارند. بنابراین، برای خواندن هر بلوک از داده‌ها، سیستم باید یک عملیات موقعیتیابی جدید انجام دهد. این بدان معناست که برای خواندن قطعه‌ی اول، دیسک باید هد را جابه‌جا کند (T_{seek}) و منتظر چرخش دیسک بماند (T_{latency}). برای قطعه‌ی دوم که در شیار دیگری است، مجدداً باید زمان‌های T_{seek} و T_{latency} صرف شود. از آنجا که حرکت بازوی مکانیکی، فیزیکی است و در مقیاس میلی‌ثانیه (ms) اندازه‌گیری می‌شود، تکرار این عمل برای هزاران بلوک داده، یک گلوگاه (Bottleneck) عظیم ایجاد می‌کند.

گام دوم: تحلیل فرآیند خواندن پیوسته (Sequential Access)

در خواندن پیوسته، داده‌ها به‌صورت متوالی و پشت‌سرهم روی یک شیار (یا استوانه‌های مجاور) ذخیره شده‌اند. در این حالت، دیسک تنها یک بار برای یافتن بلوک ابتدایی داده‌ها، زمان‌های T_{seek} و T_{latency} را صرف می‌کند. پس از استقرار هد در مکان صحیح، دیسک به چرخش خود ادامه می‌دهد و هد بدون نیاز به جابه‌جایی مکانیکی مجدد، تمامی داده‌ها را با سرعت بالا می‌خواند. در این حالت، زمان دسترسی تقریباً برابر با زمان انتقال (T_{transfer}) خواهد بود.

گام سوم: بررسی دقیق نقش زمان Seek در این تفاوت

نقش اصلی و تعیین‌کننده در اختلاف سرعت این دو روش، حذف زمان Seek در خواندن پیوسته است. زمان Seek کمترین بخش از عملیات دیسک است. برای درک بهتر، یک مثال عددی ساده را در نظر می‌گیریم:

فرض کنید زمان Seek برابر با 10 ms و زمان خواندن یک بلوک متوالی 0.1 ms باشد.

برای خواندن 100 بلوک پیوسته، تنها یک Seek نیاز است. کل زمان عبارت است از:

$$10 + (100 \times 0.1) = 20 \text{ ms}$$

برای خواندن 100 بلوک تصادفی، برای هر بلوک یک Seek مجزا نیاز است. کل زمان عبارت است از:

$$100 \times (10 + 0.1) = 1010 \text{ ms}$$

همان‌طور که مشاهده می‌شود، خواندن تصادفی به دلیل تحمیل‌زمان Seek مکرر، می‌تواند ده‌ها تا صدها بار کندتر از خواندن پیوسته باشد.

بخش 4) نتیجه‌گیری و نکات کلیدی

عامل اصلی کندی: حرکت مکانیکی هد دیسک (**Seek Time**) عامل اصلی افت سرعت در خواندن تصادفی است.

خواندن پیوسته با حذف این حرکات مکانیکی مکرر، به حداکثر توان عملیاتی دیسک دست می‌یابد.

کاربرد در بازیابی اطلاعات (IR): در طراحی سیستم‌های جستجو، ساختارهای داده‌ای مانند **نمایه معکوس**

(**Inverted Index**) معمولاً به‌گونه‌ای روی دیسک ذخیره می‌شوند که لیست‌های ارجاع (**Posting Lists**) به‌صورت

پیوسته قابل خواندن باشند. این معماری تضمین می‌کند که هنگام پردازش یک پرس‌وجو، سیستم درگیر تأخیرهای

فاجعه‌بار خواندن تصادفی نشود و بتواند داده‌ها را در کمترین زمان ممکن واکنش کند.

الگوی ذهنی:

همواره در طراحی ساختارهای داده‌ای که روی حافظه ثانویه ذخیره می‌شوند (**Secondary Storage**)، هدف اصلی باید کمینه‌کردن جابه‌جایی‌های بازوی دیسک (تقلیل Seek) و تبدیل تا حد امکان دسترسی‌های تصادفی به دسترسی‌های پیوسته باشد.

پاسخنامه مسئله 4.2: گلوگاه سیستم و نقش حافظه نهان (System Bottleneck and Caching)

بخش 1) تحلیل صورت‌مسئله

این مسئله به بررسی چالش تفاوت سرعت میان پردازنده (**CPU**) و دیسک سخت (**I/O Bottleneck**) در فرآیند ساخت نمایه (**Index Construction**) می‌پردازد. هدف، درک مکانیزم حافظه نهان سیستم‌عامل (**OS Page Cache**) و نقش حیاتی حافظه اصلی (**RAM**) به عنوان واسطی پرسرعت برای کاهش وقفه‌های ناشی از عملیات کند خواندن و نوشتن روی دیسک است.

بخش 2) مفاهیم پایه و پیش‌نیاز

گلوگاه ورودی/خروجی (I/O Bottleneck): در معماری رایانه، سرعت پردازش داده‌ها توسط **CPU** و سرعت انتقال داده‌ها در **RAM** بسیار بیشتر از سرعت خواندن و نوشتن داده‌ها روی دیسک (به‌ویژه **HDD** و حتی **SSD**) است. وقتی یک برنامه (مانند سیستم بازیابی اطلاعات) مکرراً منتظر دیسک می‌ماند، دیسک تبدیل به «گلوگاه سیستم» می‌شود و کارایی کل سیستم را به‌شدت کاهش می‌دهد.

حافظه نهان سیستم‌عامل (Page Cache / Buffer Cache): بخشی از حافظه **RAM** است که سیستم‌عامل آن را به ذخیره موقت داده‌های دیسک اختصاص می‌دهد تا دفعات مراجعه مستقیم به سخت‌افزار ذخیره‌سازی را به حداقل برساند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: تشریح چالش I/O در فرآیند ساخت نمایه

فرآیند ساخت نمایه (مانند الگوریتم‌های **SPIMI** یا **BSBI**) شامل خواندن حجم عظیمی از مستندات متنی، توکن‌سازی، مرتب‌سازی اصطلاحات (**Terms**) و لیست‌های ارجاع (**Posting Lists**) و در نهایت نوشتن این مجموعه‌های مرتب‌شده روی

دیسک است. اگر سیستم مجبور باشد برای هر توکن یا هر بلوک کوچکی خروجی، مستقیماً به دیسک فیزیکی مراجعه کند، پردازنده بخش عمده‌ای از زمان خود را در حالت انتظار (**Idle / I/O Wait**) سپری خواهد کرد.

گام دوم: مکانیزم عملکرد Caching در کاهش گلوگاه

سیستم‌عامل با پیاده‌سازی مکانیزم **Page Cache**، این مشکل را به دو طریق حل می‌کند:

1. حافظه نهان برای نوشتن (Write-back Caching): هنگامی که برنامه ساخت نمایه می‌خواهد یک بلوک از نمایه میانی را روی دیسک بنویسد، سیستم‌عامل این داده‌ها را ابتدا در **RAM** (در بخش **Page Cache**) می‌نویسد و بلافاصله به برنامه اعلام می‌کند که عملیات انجام شده است. برنامه بدون توقف به کار خود ادامه می‌دهد. سپس سیستم‌عامل در پس‌زمینه و به صورت ناهمگام (**Asynchronous**)، این داده‌های موقت (**Dirty Pages**) را به صورت بلوک‌های بزرگ و پیوسته روی دیسک فیزیکی تخلیه (**Flush**) می‌کند.

2. حافظه نهان برای خواندن (Read-ahead / Prefetching): هنگام خواندن مستندات، سیستم‌عامل بر اساس اصل «مکان‌مندی داده‌ها»، نه تنها بخش درخواستی، بلکه بلوک‌های مجاور آن را نیز پیشاپیش در **RAM** بارگذاری می‌کند تا در درخواست‌های بعدی، داده‌ها فوراً از **RAM** خوانده شوند.

گام سوم: چرا RAM نقش حیاتی در این مرحله دارد؟

حافظه **RAM** قلب تپنده‌ی این مکانیزم بهینه‌سازی است، زیرا:

پلی میان سرعت مکانیکی و الکترونیکی: سرعت دسترسی به **RAM** در مقیاس نانو ثانیه (**ns**) است، در حالی که سرعت دیسک در مقیاس میلی‌ثانیه (**ms**) یا میکروثانیه (**µs**) است. **RAM** فضایی را فراهم می‌کند تا سرعت فرآیند ساخت نمایه با سرعت پردازنده همگام شود.

تعیین‌کننده ابعاد بلوک‌های نمایه: هرچه حجم حافظه **RAM** بیشتر باشد، الگوریتم‌های ساخت نمایه (مانند **SPIMI**) می‌توانند واژه‌نامه‌ها و لیست‌های ارجاع بزرگ‌تری را به طور کامل در حافظه اصلی نگه داشته، مرتب‌سازی کنند و در قالب تعداد کمتری بلوک بزرگ به دیسک بفرستند. این کار دسترسی‌های تصادفی و پرهزینه‌ی دیسک را به حداقل می‌رساند.

بخش 4) نتیجه‌گیری و نکات کلیدی

کاهش گلوگاه: حافظه نهان سیستم‌عامل با تبدیل عملیات خرد و مکرر **I/O** به عملیات تجمعی و کلان، زمان انتظار پردازنده را به شدت کاهش داده و گلوگاه دیسک را کم‌رنگ می‌کند.

وابستگی به سخت‌افزار: در فرآیند ساخت نمایه‌های مقیاس‌بزرگ (**Large-scale Indexing**)، میزان حافظه **RAM** در دسترس، مستقیماً نرخ کارایی و سرعت کل فرآیند را تعیین می‌کند؛ زیرا **RAM** بزرگ‌تر یعنی نمایه‌سازی در-حافظه (**In-Memory**) بیشتر و مراجعه به دیسک کمتر.

الگوی ذهنی:

سیستم‌های بازیابی اطلاعات مدرن برای فرار از محدودیت‌های سرعت دیسک، همواره تلاش می‌کنند تا جایی که ممکن است فرآیندهای سنگین (مانند ادغام و مرتب‌سازی) را در محیط فوق سریع **RAM** انجام دهند و دیسک را فقط برای ذخیره‌سازی نهایی و پایدار به کار بگیرند.

پاسخنامه مسئله 4.3: هدف الگوریتم BSBI (Purpose of BSBI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک محدودیت‌های فیزیکی سخت‌افزار در برابر مقیاس عظیم داده‌ها در سیستم‌های بازیابی اطلاعات مدرن است. در این مسئله بررسی می‌کنیم که چرا معماری‌های ساده و درون‌حافظه‌ای (**In-Memory**) برای پیکره‌های متنی بزرگ کارایی ندارند و چگونه رویکردهای مبتنی بر دیسک (**Disk-based**) مانند الگوریتم **BSBI**، این بحران را مدیریت می‌کنند.

بخش 2) مفاهیم پایه و پیش‌نیاز

پیکره (Corpus): مجموعه عظیم اسناد متنی که قرار است پردازش و نمایه‌سازی شوند (مانند کل صفحات وب یک دامنه یا یک آرشیو بزرگ اسناد سازمانی).

سرریز حافظه (Out of Memory - OOM): خطایی که زمانی رخ می‌دهد که حجم داده‌های در حال پردازش، از ظرفیت فیزیکی حافظه اصلی (**RAM**) بیشتر شود.

مرتب‌سازی خارجی (External Sorting): دسته‌ای از الگوریتم‌های مرتب‌سازی که برای مدیریت و مرتب کردن داده‌هایی طراحی شده‌اند که حجم آن‌ها بسیار بیشتر از گنجایش حافظه اصلی سیستم است و نیازمند استفاده ترکیبی از **RAM** و دیسک هستند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: چرا نمی‌توانیم نمایه‌سازی را یکجا در RAM انجام دهیم؟

ساخت نمایه معکوس مستلزم استخراج تمام جفت‌های (اصطلاح، شناسه سند) از تک‌تک اسناد و سپس مرتب‌سازی آن‌ها است. در دنیای واقعی، حجم یک پیکره متنی ممکن است ده‌ها ترابایت باشد، در حالی که حافظه **RAM** سرورها معمولاً به چند ده یا نهایتاً چند صد گیگابایت محدود است. اگر بخواهیم تمام این جفت‌ها را استخراج کرده و یکجا در **RAM** نگه داریم، حافظه به سرعت پر می‌شود. در این حالت، سیستم‌عامل برای جلوگیری از فروپاشی، شروع به استفاده از حافظه مجازی (**Virtual Memory/Swap**) روی دیسک سخت می‌کند. این جابه‌جایی مداوم تکه‌های حافظه بین **RAM** و دیسک، منجر به پدیده‌ای به نام **کوبیدگی (Thrashing)** می‌شود که سرعت پردازش سیستم را به شکل فاجعه‌باری (هزاران بار کندتر) کاهش داده و عملاً فرآیند نمایه‌سازی را متوقف می‌کند.

گام دوم: دلیل اصلی تولد الگوریتم BSBI

الگوریتم **Blocked Sort-Based Indexing** دقیقاً برای غلبه بر این بن‌بست «کمبود ظرفیت حافظه» متولد شد. دلیل اصلی تولد این الگوریتم، نیاز به روشی مقیاس‌پذیر (**Scalable**) بود که بتواند مستقل از محدودیت‌های حجم **RAM**، هر پیکره‌ای با هر اندازه‌ای را با بالاترین بهره‌وری ممکن نمایه‌سازی کند. استراتژی محوری این الگوریتم، استفاده از تکنیک «تقسیم و غلبه» (**Divide and Conquer**) است.

گام سوم: مکانیزم عملکرد BSBI در دور زدن محدودیت حافظه

به جای تلاش برای بلعیدن یکجای کل پیکره، الگوریتم **BSBI** فرآیند را به قطعاتی به اندازه ظرفیت **RAM** (بلوک) می‌شکند:

1. **استخراج و تجمیع موقت:** اسناد به نوبت خوانده شده و جفت‌های شناسه اصطلاح و شناسه سند (**TermID**, {

DocID}) استخراج می‌شوند. این کار فقط تا جایی ادامه می‌یابد که ظرفیت تعیین‌شده‌ی **RAM** پر شود (تشکیل یک بلوک از داده‌های خام).

2. **مرتب‌سازی محلی درون‌حافظه‌ای:** جفت‌های موجود در این بلوک موقت، با استفاده از الگوریتم‌های سریع (مانند

Quick Sort با پیچیدگی زمانی $O(N \log N)$) به طور کامل در محیط پرسرعت **RAM** مرتب می‌شوند.

3. **تخلیه روی دیسک (Flushing):** این بلوک مرتب‌شده، به صورت یک فایل پیوسته (**Sequential**) روی دیسک نوشته می‌شود. سپس حافظه **RAM** کاملاً پاک شده و برای پردازش بلوک بعدی اسناد آماده می‌شود.

4. **ادغام نهایی (Merging):** پس از پایان پردازش کل پیکره، ده‌ها یا صدها فایل مرتب‌شده روی دیسک داریم. در نهایت، سیستم با خواندن همزمان بخش‌های کوچکی از این فایل‌ها، آن‌ها را با هم ادغام کرده و نمایه معکوس یکپارچه و نهایی را روی دیسک می‌سازد.

بخش 4) نتیجه‌گیری و نکات کلیدی

تغییر پارادایم: الگوریتم **BSBI** نشان می‌دهد که در پردازش داده‌های کلان (**Big Data**)، **RAM** نباید به عنوان مخزن ذخیره‌سازی کل فرآیند دیده شود، بلکه باید صرفاً به عنوان یک «کارگاه پردازشی موقت و پرسرعت» مورد استفاده قرار گیرد.

الگوی ذهنی:

همواره به یاد داشته باشید که هرگاه حجم کل داده‌های ورودی از گنجایش حافظه اصلی فراتر رفت، راهکار استاندارد و مهندسی‌شده استفاده از مکانیزم **مرتب‌سازی خارجی (External Sorting)** است؛ داده‌ها را به قطعاتی هم‌اندازه با **RAM** تقسیم کنید، آن‌ها را به صورت محلی مرتب کرده و روی دیسک انباشت کنید، و در نهایت فرآیند ادغام (**Merge**) را برای تولید خروجی نهایی انجام دهید.

پاسخنامه مسئله 4.4: جریان داده در BSBI (Data Flow in BSBI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، شناخت دقیق چرخه حیات داده‌ها در الگوریتم نمایه‌سازی مبتنی بر مرتب‌سازی بلوکی (**BSBI**) است. در این فرآیند، نیاز است تا جریان تبدیل یک متن خام به یک ساختار داده‌ای سازمان‌یافته روی دیسک (نمایه معکوس میانی) به صورت مرحله‌به‌مرحله و شفاف واکاوی شود. این بینش برای درک نحوه مدیریت حافظه در سیستم‌های بازیابی اطلاعات ضروری است.

بخش 2) مفاهیم پایه و پیش‌نیاز

الگوریتم **BSBI** به جای پردازش یکپارچه، داده‌ها را در قالب «بلوک‌های مستقل» پردازش می‌کند. هر بلوک، حجمی از داده است که به طور کامل در حافظه اصلی (**RAM**) جای می‌گیرد. جریان داده در این الگوریتم، مسیر تبدیل اسناد متنی به لیستی از ارجاعات عددی و در نهایت ذخیره آن‌ها روی حافظه جانبی را توصیف می‌کند.

بخش 3) حل گام‌به‌گام مسئله (مراحل جریان داده)

گام‌های اساسی تبدیل یک بلوک متنی به بلوک نمایه روی دیسک به ترتیب زیر است:

گام اول تجزیه اسناد (Parsing): خواندن متن خام اسناد، شکستن آن‌ها به نشانه‌ها (**Tokens**) و تولید جفت‌های اولیه (واژه، شناسه سند).

گام دوم نگاشت شناسه (TermID Mapping): تبدیل رشته‌های متنی واژگان به شناسه‌های عددی یکتا (**TermID**) از طریق یک واژه‌نامه درون‌حافظه‌ای جهت بهینه‌سازی فضا.

گام سوم تجميع و انباشت (Accumulation): ذخيره جفت‌هاى (شناسه واژه، شناسه سند) در يك آرايه يا ساختار داده درون حافظه‌اى تا مرز پر شدن ظرفيت مجاز بلوك.

گام چهارم مرتب‌سازى محلى (Local Sorting): مرتب‌کردن سريع جفت‌هاى موجود در حافظه اصلى، ابتدا بر اساس شناسه واژه و در صورت برابرى، بر اساس شناسه سند.

گام پنجم تشكيل ليست ارجاع و تخلیه روی دیسک (Inverting & Flushing): گروه‌بندى شناسه‌هاى سند مرتبط با هر واژه (ايجاد Posting List) و نوشتن كل بلوك مرتب‌شده به‌صورت يك فايل پيوسته روی دیسک سخت.

بخش 4) نتیجه‌گیری و نکات کلیدی

پيوستگى مراحل: جريان داده در **BSBI** يك خط توليد (**Pipeline**) دقيق است كه در آن خروجى هر مرحله، ورودى مرحله بعد است. هرگونه نقص در يك مرحله (مثلاً عدم نگاشت صحيح شناسه‌ها) كل ساختار نمايه را مخدوش مى‌كند.

گذر از متن به عدد: يكى از مهم‌ترين دستاوردهاى اين جريان داده، جاىگزینی كامل رشته‌هاى متنى سنگین با اعداد صحيح (**Integers**) در همان گام‌هاى ابتدایى است كه باعث افزایش چشمگیر سرعت مرتب‌سازى و کاهش شديد حجم داده‌هاى انتقالی به دیسک مى‌شود.

الگوی ذهنی:

برای به‌خاطر سپردن این فرآیند، آن را مانند یک کارخانه بسته‌بندی در نظر بگیرید: ابتدا مواد اولیه خرد می‌شوند (تجزیه)، سپس برچسب‌گذاری کدی می‌شوند (نگاشت)، در انبار موقت جمع می‌شوند (تجميع)، به ترتیب شماره کد مرتب می‌گردند (مرتب‌سازى)، و در نهایت در بسته‌هاى نهایی به انبار اصلى فرستاده می‌شوند (تخلیه روی دیسک).

پاسخنامه مسئله 4.5: مزیت اصلی الگوریتم SPIMI (Main Advantage of SPIMI)

بخش 1) تحلیل صورت‌مسئله

هدف از این مسئله، مقایسه دو معماری بنیادین در ساخت نمایه‌هاى مبتنى بر دیسک و واکاوى چرایی برتری الگوریتم **SPIMI** (نمایه‌سازى یک‌مرحله‌ای در-حافظه) بر **BSBI** است. تمرکز اصلى این بخش، بر روی درک تفاوت نحوه سازمان‌دهی داده‌ها در حافظه اصلى (**RAM**) پیش از تخلیه آن‌ها روی دیسک سخت و تأثیر آن بر پیچیدگى زمانى و سرعت پردازش است.

بخش 2) مفاهیم پایه و پیش‌نیاز

الگوریتم BSBI: رویکردی كه جفت‌هاى خام (شناسه واژه، شناسه سند) را در حافظه جمع‌آوری کرده و در نهایت كل این آرايه عظیم را پیش از نوشتن روی دیسک، مرتب‌سازى مى‌كند.

الگوریتم SPIMI: الگوریتمی كه واژه‌نامه (**Dictionary**) و لیست‌هاى ارجاع (**Posting Lists**) را به‌صورت پویا (**Dynamic**) و درلحظه درون حافظه شكل مى‌دهد و اسناد را تنها با يك بار مرور (**Single-pass**) پردازش مى‌كند.

بخش 3) حل گام‌به‌گام مسئله

در رویکرد **BSBI**، سیستم باید میلیون‌ها جفت استخراج‌شده را در یک آرایه بزرگ درون حافظه ذخیره کند. زمانی که حافظه به مرز تکمیل می‌رسد، سیستم موظف است این آرایه را به‌طور کامل مرتب کند (معمولاً با استفاده از الگوریتم‌هایی با مرتبه زمانی $O(N \log N)$). با توجه به اینکه متغیر **N** در اینجا نمایانگر کل نشانه‌های (**Tokens**) موجود در یک بلوک است، این عملیات مرتب‌سازی سراسری، گلوگاه محاسباتی سنگینی برای پردازنده (**CPU**) ایجاد می‌کند و سرعت تولید بلوک را به‌شدت کاهش می‌دهد.

گام دوم: استراتژی ساختاریافته و پویا در الگوریتم SPIMI

الگوریتم **SPIMI** این گلوگاه را با تغییر بنیادین ساختار داده‌ها دور می‌زند. در این روش، به‌جای انباشت جفت‌های نامرتب در یک آرایه خام، از ساختارهای پویایی نظیر جدول درهم‌سازی (**Hash Table**) برای ساخت واژه‌نامه محلی استفاده می‌شود. روند کار به این شکل است:

1. سیستم کلمه را می‌خواند. اگر واژه جدید باشد، آن را به واژه‌نامه افزوده و یک لیست ارجاع (**Posting List**) جدید برای آن تخصیص می‌دهد.

2. اگر واژه پیش‌تر در واژه‌نامه ثبت شده باشد، سیستم مستقیماً شناسه سند فعلی (**DocID**) را به انتهای لیست ارجاع اختصاصی همان واژه الحاق (**Append**) می‌کند. این عملیات در زمان ثابت $O(1)$ انجام می‌شود.

گام سوم: مزیت کلیدی (حذف مرتب‌سازی عظیم نشانه‌ها)

از آنجا که اسناد به ترتیب خوانده می‌شوند (از سند 1 به بالا)، شناسه‌های سندی که به انتهای لیست‌های ارجاع اضافه می‌گردند، به‌طور ذاتی و خودکار مرتب‌شده هستند. بنابراین، برخلاف **BSBI**، در الگوریتم **SPIMI** هیچ نیازی به مرتب‌سازی شناسه‌های سند درون لیست‌ها نیست.

تنها مرتب‌سازی مورد نیاز در پایان پردازش یک بلوک، مرتب‌کردن خود واژگان یکتا (**Keys of Dictionary**) به ترتیب حروف الفبا است تا فرآیند ادغام نهایی فایل‌ها (**Merging**) تسهیل گردد. از آنجا که تعداد واژگان یکتا در یک زبان بسیار کمتر از تعداد کل کلمات متن است، این مرتب‌سازی کوچک، هزینه پردازشی ناچیزی دارد.

بخش 4 نتیجه‌گیری و نکات کلیدی

برتری اصلی (مزیت سرعت): مزیت حیاتی و غیرقابل‌انکار **SPIMI** نسبت به **BSBI**، حذف نیاز به مرتب‌سازی سراسری تمامی توکن‌ها است. تبدیل عملیات سنگین مرتب‌سازی با پیچیدگی $O(N \log N)$ به عملیات سریع الحاق به لیست با پیچیدگی $O(1)$ ، سرعت تولید و تخلیه بلوک‌های نمایه را به شکل چشمگیری بهینه‌سازی می‌کند.

مزیت جانبی (مدیریت حافظه): **SPIMI** نیازی به نگهداری یک واژه‌نامه سراسری (**Global Dictionary**) در **RAM** برای تبدیل رشته‌ها به **TermID** ندارد؛ هر بلوک، واژه‌نامه رشته‌ای (**String**) مختص به خود را می‌سازد که این امر، اجرای الگوریتم را روی پیکره‌های با حجم نامحدود، بدون نگرانی از کمبود حافظه ممکن می‌سازد.

الگوی ذهنی:

تفاوت **BSBI** و **SPIMI** مانند دو روش مختلف برای مرتب کردن نامه‌های پستی است. در **BSBI**، شما تمام نامه‌ها را روی یک میز بزرگ می‌ریزید و در انتها سعی می‌کنید میلیون‌ها نامه را یکجا بر اساس مقصد مرتب کنید (کند و پرهزینه). اما در **SPIMI**، شما از

همان ابتدا یک قفسه با کتوهای مشخص دارید؛ هر نامه را به محض دریافت، در کتوی شهر مربوطه می‌اندازید. در انتها، نامه‌های داخل هر کتو به ترتیب زمان دریافت چیده شده‌اند و نیازی به مرتب‌سازی مجدد آن‌ها نیست.

پاسخنامه مسئله 4.6: مدیریت واژه‌نامه در SPIMI (Dictionary Management in SPIMI)

بخش 1) تحلیل صورت مسئله

این مسئله به یکی از تفاوت‌های ساختاری و کلیدی میان **SPIMI** و سایر الگوریتم‌ها (مانند **BSBI**) می‌پردازد. هدف، درک نحوه مدیریت حافظه و پردازش واژگان به صورت محلی است، بدون آنکه نیازی به نگهداری یک ساختار داده‌ی سراسری برای تبدیل رشته‌های متنی (**Strings**) به شناسه‌های عددی (**TermID**) باشد.

بخش 2) مفاهیم پایه و پیش‌نیاز

نگاشت سراسری (Global Mapping): در روش‌هایی مثل **BSBI**، برای کاهش حجم داده‌های در حال پردازش، هر کلمه متنی به یک عدد منحصر به فرد (**TermID**) نگاشت می‌شود. این کار نیازمند نگهداری یک واژه‌نامه سراسری در **RAM** است که برای پیکره‌های بسیار بزرگ، به دلیل کمبود حافظه عملاً غیرممکن می‌شود.

واژه‌نامه محلی (Local Dictionary): ساختار داده‌ای (معمولاً جدول درهم‌سازی یا **Hash Table**) که تنها حاوی واژگان موجود در بلوک فعلی است و به صورت پویا در حافظه اصلی مدیریت می‌شود.

بخش 3) حل گام به گام مسئله

گام اول: استفاده از ساختار داده‌ی پویا (جدول درهم‌سازی)

در الگوریتم **SPIMI**، واژه‌نامه به جای استفاده از یک آرایه مبتنی بر شاخص (**Index-based**)، به صورت یک جدول درهم‌سازی (**Hash Table**) درون حافظه پیاده‌سازی می‌شود. کلیدهای (**Keys**) این جدول، خود رشته‌های متنی کلمات (**Terms**) هستند و مقادیر (**Values**) آن‌ها، اشاره‌گرهایی (**Pointers**) به لیست‌های ارجاع (**Posting Lists**) می‌باشند.

گام دوم: مواجهه با کلمه و افزودن به واژه‌نامه (Insertion)

هنگامی که سیستم یک توکن را از سند می‌خواند، فرآیند زیر برای مدیریت واژه‌نامه طی می‌شود:

- سیستم رشته متنی توکن را به عنوان کلید در جدول درهم‌سازی جستجو می‌کند.
- اگر کلمه یافت نشد، این بدان معناست که واژه برای بلوک فعلی جدید است. سیستم مستقیماً این رشته را به عنوان یک ورودی جدید در جدول ثبت می‌کند.
- بلافاصله یک فضای حافظه پویا (مانند یک آرایه با قابلیت تغییر اندازه یا **Dynamic Array**) برای ساخت لیست ارجاع اختصاصی این کلمه تخصیص می‌دهد.
- در نهایت، شناسه سند فعلی (**DocID**) را در این لیست تازه تأسیس می‌نویسد.

(در صورت یافت شدن کلمه در جدول، سیستم مستقیماً **DocID** را به انتهای لیست موجود الحاق می‌کند).

گام سوم: مدیریت بدون نیاز به TermID سراسری

دلیل اصلی و معماری‌گونه‌ای که باعث می‌شود **SPIMI** به نگاشت **TermID** نیازی نداشته باشد، اتکای آن به پردازش بلوک‌محور

کاملاً مستقل است. هر بلوک، واژه‌نامه رشته‌ای (**String-based Dictionary**) مختص به خود را از صفر می‌سازد.

زمانی که حافظه **RAM** پر می‌شود، سیستم کلیدهای این جدول درهم‌سازی (رشته‌های متنی) را به ترتیب حروف الفبا مرتب کرده و بلوک را به صورت جفت‌های [توالی شناسه‌های سند]: رشته متنی مستقیماً روی دیسک تخلیه می‌کند. پس از تخلیه، واژه‌نامه بلوک قبلی کاملاً از حافظه پاک شده و جدول درهم‌سازی جدیدی برای بلوک بعدی ایجاد می‌گردد.

بخش 4) نتیجه‌گیری و نکات کلیدی

استقلال بلوک‌ها و مقیاس‌پذیری: عدم استفاده از **TermID** سراسری به **SPIMI** اجازه می‌دهد تا هر بلوک را به صورت

ایزوله پردازش کند. این ویژگی، مقیاس‌پذیری (**Scalability**) الگوریتم را تضمین می‌کند، زیرا سیستم هرگز محدود به

نگهداری کل واژه‌نامه در **RAM** نیست و می‌تواند پیکره‌هایی با حجم نامحدود را نمایه‌سازی کند.

تعویق یکپارچه‌سازی به فاز ادغام: مدیریت نهایی واژگان یکتا در سطح کل پیکره، به فاز نهایی یعنی «ادغام بلوک‌ها»

(**Merge Phase**) موکول می‌شود. در این مرحله، فایل‌های موقت که بر اساس الفبای رشته‌های متنی مرتب شده‌اند،

به‌طور همزمان از دیسک خوانده شده و لیست‌های ارجاع مربوط به کلمات مشترک با یکدیگر ترکیب می‌شوند.

الگوی ذهنی:

فرض کنید در حال فهرست‌برداری از کتاب‌های چندین کتابخانه مختلف هستید. به جای اینکه ابتدا یک دفترچه مرکزی بسازید و به هر عنوان کتاب در دنیا یک کد عددی اختصاص دهید (روش **BSBI**)، در هر کتابخانه مستقیماً نام کتاب‌ها را در یک دفترچه محلی می‌نویسید و شماره قفسه‌ها را روبه‌روی آن‌ها ثبت می‌کنید (روش **SPIMI**). در نهایت، این دفترچه‌های محلی را که بر اساس حروف الفبا نوشته شده‌اند، به راحتی با هم ترکیب کرده و فهرست نهایی را می‌سازید.

پاسخنامه مسئله 4.7: توزیع کلمات در برابر اسناد (Term vs. Document Partitioning)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، شناخت استراتژی‌های کلیدی برای توزیع فیزیکی نمایه معکوس (**Inverted Index**) در کلاسترهای پردازشی (**Clusters**) است. در سیستم‌های بازیابی اطلاعات مقیاس‌بزرگ (ترا-مقیاس)، درک تفاوت میان این دو معماری بنیادین برای طراحی موتورهای جستجوی توزیع‌شده و مدیریت صحیح ترافیک شبکه (**Network Traffic**) و بار پردازشی (**Load Balancing**) ضروری است.

بخش 2) مفاهیم پایه و پیش‌نیاز

نمایه‌سازی توزیع‌شده (**Distributed Indexing**) زمانی به کار می‌رود که یک ماشین (سرور) به تنهایی قادر به نگهداری کل داده‌ها یا پاسخ‌گویی به حجم عظیمی از پرس‌وجوهای همزمان (**Queries**) نباشد. در این حالت، باید کل پیکره (**Corpus**) یا خود واژه‌نامه (**Vocabulary**) شکسته شده و میان چندین سرور که اصطلاحاً به آن‌ها «گره» (**Node**) گفته می‌شود، تقسیم گردد. برای این کار دو رویکرد استاندارد وجود دارد.

بخش 3) حل گام‌به‌گام مسئله

گام اول: تعریف معماری توزیع بر اساس اسناد (Document-Partitioning)

در این معماری که به آن «نمایه محلی» (**Local Index**) نیز گفته می‌شود، کل پیکره اسناد به صورت تصادفی یا ساختاریافته به چندین زیرمجموعه تقسیم شده و هر زیرمجموعه به یک گره اختصاص می‌یابد.

نحوه نمایه‌سازی: هر گره به طور کاملاً مستقل، یک نمایه معکوس برای اسنادِ خودش می‌سازد. این نمایه شامل تمامی واژگان موجود در آن اسناد است.

نحوه پردازش پرس‌وجو: هنگامی که یک پرس‌وجو وارد سیستم می‌شود، یک گره مرکزی (**Broker**) آن را به تمامی گره‌ها (**All Nodes**) ارسال می‌کند (**Broadcast**). هر گره اسناد مرتبط محلی خود را پیدا کرده و نمره‌دهی می‌کند. سپس نتایج برتر به گره مرکزی بازگردانده شده، با هم ادغام (**Merge**) می‌شوند و فهرست نهایی به کاربر ارائه می‌گردد.

گام دوم: تعریف معماری توزیع بر اساس کلمات (Term-Partitioning)

در این معماری که به آن «نمایه سراسری» (**Global Index**) نیز گفته می‌شود، خودِ واژه‌نامه میان گره‌ها تقسیم می‌گردد. این تقسیم‌بندی معمولاً بر اساس حروف الفبا (مثلاً سرور اول: واژگان الف تا ت) یا با استفاده از توابع درهم‌سازی (**Hash Functions**) انجام می‌شود.

نحوه نمایه‌سازی: هر گره مسئول نگهداری لیست‌های ارجاع (**Posting Lists**) برای یک زیرمجموعه خاص از کلمات است. اما این لیستِ ارجاع، شامل شناسه اسناد از تمامی پیکره (کل اسناد) می‌باشد.

نحوه پردازش پرس‌وجو: پرس‌وجوی ورودی تنها به گره‌هایی خاص (**Specific Nodes**) ارسال می‌شود که واژگان مورد جستجو در آن‌ها قرار دارند. گره‌های درگیر، لیست‌های ارجاع خود را به گره مرکزی می‌فرستند تا فرآیند اشتراک (**Intersection**) و محاسبه امتیازها انجام شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

مقایسه کارایی: معماری **Document-Partitioned** به دلیل سادگی در افزودن سرورهای جدید، استقلال گره‌ها از یکدیگر و تحمل‌پذیری بالا در برابر خرابی سرورها (**Fault Tolerance**)، معماری غالب در موتورهای جستجوی بزرگ (مانند موتورهای جستجوی وب) است. در مقابل، معماری **Term-Partitioned** به دلیل نیاز به انتقال حجم عظیمی از لیست‌های ارجاع خام در بستر شبکه هنگام جستجوهای چندکلمه‌ای، با گلوگاه‌های شبکه‌ای مواجه می‌شود.

جمع‌بندی:

توزیع اسناد (Document-Partitioning): مانند داشتن چندین شعبه از یک کتابخانه در شهرهای مختلف است. هر شعبه فهرست کامل کتاب‌های خودش را دارد. اگر دنبال موضوعی بگردید، باید از تک‌تک شعبه‌ها سوال کنید تا بهترین کتاب‌ها از سراسر کشور جمع‌آوری شوند.

توزیع کلمات (Term-Partitioning): مانند داشتن یک کتابخانه ملی عظیم است که فهرست آن به چندین جلد قطور تقسیم شده است؛ جلد اول فقط برای کلماتی که با حرف «الف» شروع می‌شوند، جلد دوم برای «ب» و الی آخر. برای جستجو، مستقیماً سراغ جلدهای مرتبط می‌روید، اما اطلاعات موجود در آن جلد، آدرس کتاب‌ها در کل مخازن کتابخانه را به شما می‌دهد.

پاسخنامه مسئله 4.8: نقش Map و Reduce در ساخت نمایه

(Role of Map and Reduce in Indexing)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک نحوه توزیع پردازش‌ها در نمایه‌سازی مقیاس‌بزرگ (**Large-Scale Indexing**) با استفاده از چارچوب پردازشی **MapReduce** است. در سیستم‌های بازیابی اطلاعات مدرن که حجم پیکره متنی به پتابایت می‌رسد، یک ماشین به تنهایی توان پردازش داده‌ها را ندارد. این مسئله بررسی می‌کند که چگونه فرآیند پیچیده‌ی ساخت نمایه معکوس، به دو تابع ساده و توزیع‌پذیر شکسته می‌شود و جریان داده میان آن‌ها چگونه مدیریت می‌گردد.

بخش 2) مفاهیم پایه و پیش‌نیاز

چارچوب **MapReduce** بر پایه پردازش «جفت‌های کلید-مقدار» (**Key-Value Pairs**) استوار است. تمامی داده‌های ورودی، میانی و خروجی در این سیستم به شکل $\langle K, V \rangle$ مدل‌سازی می‌شوند. این معماری به برنامه‌نویس اجازه می‌دهد بدون نگرانی از جزئیات شبکه، همگام‌سازی و تحمل‌پذیری خطا، پردازش‌ها را روی هزاران گره (**Node**) توزیع کند. جریان کلی به این شکل است:

$$\langle k_1, v_1 \rangle \xrightarrow{\text{Map}} \text{list}(\langle k_2, v_2 \rangle) \xrightarrow{\text{Shuffle \& Sort}} \langle k_2, \text{list}(v_2) \rangle \xrightarrow{\text{Reduce}} \text{list}(\langle k_3, v_3 \rangle)$$

بخش 3) حل گام‌به‌گام مسئله

گام اول: وظیفه اصلی تابع Map (فاز نگاشت)

تابع **Map** در فرآیند نمایه‌سازی، نقش «تجزیه‌گر و استخراج‌کننده محلی» را ایفا می‌کند. هر ماشین در کلاستر پردازشی که تابع **Map** را اجرا می‌کند (**Mapper**)، بخشی از اسناد (**Document Splits**) را به‌عنوان ورودی دریافت می‌کند. ورودی این تابع یک جفت به شکل $\langle \text{DocID}, \text{Document_Text} \rangle$ است. وظایف اصلی تابع **Map** روی این ورودی عبارتند از:

1. تجزیه متن (**Parsing**) و شکستن آن به نشانه‌ها (**Tokens**).

2. اعمال پردازش‌های زبانی (مثل حذف کلمات توقف یا ریشه‌یابی).

3. تولید و انتشار (**Emit**) جفت‌های میانی به ازای هر کلمه‌ی یافت‌شده.

خروجی تابع **Map** جفت‌های کلید-مقدار میانی به شکل $\langle \text{Term}, \text{DocID} \rangle$ است (کلید = کلمه، مقدار = شناسه سند).

گام دوم: مکانیزم پنهان فریم‌ورک (Shuffle and Sort)

پیش از آنکه خروجی‌های **Map** به تابع **Reduce** برسند، خود چارچوب **MapReduce** عملیات بسیار مهمی به نام جابه‌جایی و مرتب‌سازی انجام می‌دهد. خروجی‌های تمام **Mapper**ها در سراسر شبکه جمع‌آوری شده و بر اساس کلید (**Term**) مرتب می‌شوند. فریم‌ورک تمام مقادیر (**DocID**ها) که کلید (**Term**) یکسانی دارند را در قالب یک آرایه یا لیست تجمیع می‌کند.

گام سوم: فرمت تحویل داده به تابع Reduce (فاز کاهش)

بر اساس عملیات گام پیشین، خروجی تابع **Map** به‌صورت خام به تابع **Reduce** تحویل داده نمی‌شود. بلکه فریم‌ورک ورودی تابع **Reduce** را به شکل ساختاریافته‌ی زیر آماده کرده و به آن تحویل می‌دهد:

به‌عنوان مثال، اگر کلمه‌ی «دانشگاه» در اسناد 12، 45 و 89 توسط ماشین‌های مختلف (**Mapper**) یافت شده باشد، تابع **Reduce** در ورودی خود دقیقاً جفت «دانشگاه»، [89, 45, 12] را دریافت می‌کند. وظیفه تابع **Reduce** در اینجا بسیار ساده است: حذف شناسه‌های تکراری، مرتب‌سازی نهایی شناسه‌های سند (در صورت نیاز) و نوشتن این لیست به‌عنوان لیست ارجاع (**Posting List**) در فایل نمایه نهایی روی دیسک.

بخش 4) نتیجه‌گیری و نکات کلیدی

تقسیم کار هوشمندانه: تابع **Map** به‌صورت کاملاً موازی و بی‌نیاز از سایر ماشین‌ها، اسناد را پردازش کرده و حقایق محلی (کلمه **X** در سند **Y** وجود دارد) را استخراج می‌کند. تابع **Reduce** این حقایق پراکنده را دریافت کرده و به یک دانش یکپارچه و سراسری (کلمه **X** در کل این اسناد وجود دارد) تبدیل می‌کند.

کاهش بار پردازشی شبکه: دلیل استفاده از این ساختار آن است که فرآیند مرتب‌سازی و گروه‌بندی توسط زیرساخت **MapReduce** بهینه‌سازی شده است و نیازی به مدیریت دستی انتقال داده‌ها بین سرورها نیست.

الگوی ذهنی:

فرض کنید قصد داریم آرای یک انتخابات بزرگ را در 100 شعبه اخذ رأی بشماریم. تابع **Map** مانند متصدیان هر شعبه است؛ آن‌ها صندوق‌ها (اسناد) را باز کرده و برای هر رأی، یک برگه کوچک به شکل «نام کاندیدا، شماره شعبه» می‌نویسند. سپس پستی‌ها (فاز **Shuffle & Sort**) تمام برگه‌های کشور را جمع کرده و بر اساس نام کاندیدا دسته‌بندی می‌کنند. در نهایت، تابع **Reduce** مانند هیئت مرکزی نظارت است که برای هر کاندیدا، یک دسته برگه شامل تمام شعباتی که از آن‌ها رأی آورده را دریافت می‌کند (مثلاً «نام کاندیدا، [1, 5, 42]») و نتیجه نهایی را ثبت می‌کند.

پاسخنامه مسئله 4.9: نمایه‌سازی پویا و نمایه کمکی (Dynamic Indexing and Auxiliary Index)

بخش 1) تحلیل صورت مسئله

این مسئله به یکی از چالش‌های بنیادین در سیستم‌های بازیابی اطلاعات بی‌درنگ (**Real-Time IR Systems**) می‌پردازد. در محیط‌هایی مانند خبرگزاری‌ها، شبکه‌های اجتماعی یا سامانه‌های معاملاتی، پیکره متنی ایستا (**Static**) نیست و اسناد جدید در هر لحظه تولید، ویرایش یا حذف می‌شوند. هدف این مسئله، درک دلیل ناکارآمدی رویکردهای سنتی در مواجهه با داده‌های پویا و بررسی معماری «نمایه کمکی» به‌عنوان یک راه‌حل مهندسی‌شده برای پاسخ‌گویی به این نیاز است.

بخش 2) مفاهیم پایه و پیش‌نیاز

نمایه اصلی (Main Index): نمایه معکوس یکپارچه و عظیمی که شامل اکثریت قریب به اتفاق اسناد سیستم است. این نمایه به دلیل حجم بالا، به‌صورت ساختاریافته روی دیسک سخت (**HDD/SSD**) ذخیره می‌شود و به‌روزرسانی مستقیم آن بسیار پرهزینه است.

نمایه کمکی (Auxiliary Index): یک نمایه معکوس کوچک، موقت و بسیار سریع که معمولاً به‌طور کامل در حافظه اصلی (**RAM**) نگهداری می‌شود و وظیفه آن ثبت تغییرات و اسناد جدید است.

نمایه‌سازی پویا (Dynamic Indexing): فرآیندی که به سیستم اجازه می‌دهد اسناد جدید را بلافاصله پس از ورود،

بدون نیاز به بازسازی کل نمایه، قابل جستجو کند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: چرا بازسازی کل نمایه (Rebuilding) غیرممکن است؟

در سیستم‌های سنتی، ساخت نمایه (با الگوریتم‌هایی نظیر **SPIMI** یا **BSBI**) یک فرآیند دسته‌ای (**Batch Processing**) است. اگر بخواهیم با ورود هر خبر جدید، کل نمایه را از نو بسازیم، سیستم باید میلیون‌ها سند قبلی را مجدداً پردازش، مرتب و روی دیسک ذخیره کند. این عملیات ممکن است ساعت‌ها یا روزها طول بکشد. در نتیجه، اگر سیستم از این روش استفاده کند، یک خبر فوری تا زمان پایان فرآیند نمایه‌سازی، در نتایج جستجو ظاهر نخواهد شد که این امر برای یک خبرگزاری فاجعه‌بار است.

گام دوم: راهکار معماری دوگانه (Dual-Index Architecture)

برای حل این بحران، سیستم بازیابی اطلاعات به دو بخش تقسیم می‌شود:

1. **بایگانی راکد:** اسناد قدیمی که تغییری نمی‌کنند در «نمایه اصلی» روی دیسک قرار می‌گیرند.

2. **جریان فعال:** هر سند یا خبر جدیدی که وارد سیستم می‌شود، مستقیماً و به‌سرعت پردازش شده و به «نمایه کمکی»

در حافظه **RAM** اضافه می‌شود. از آنجا که این نمایه کوچک و درون‌حافظه‌ای است، به‌روزرسانی آن در کسری از ثانیه انجام می‌پذیرد.

گام سوم: پردازش پرس‌وجو (Query Processing) در سیستم پویا

وقتی کاربری عبارتی را جستجو می‌کند، سیستم جستجو (**Search Engine**) موتور خود را همزمان به سراغ هر دو نمایه می‌فرستد:

لیست ارجاع (**Posting List**) مربوط به کلمه مورد نظر را از نمایه اصلی (دیسک) واکنشی می‌کند.

به‌طور همزمان، لیست ارجاع همان کلمه را از نمایه کمکی (**RAM**) نیز واکنشی می‌کند.

سپس سیستم این دو لیست را در حافظه با یکدیگر ادغام (**Merge**) کرده و نتایج نهایی را به کاربر نمایش می‌دهد. بدین ترتیب، کاربر هم اسناد قدیمی و هم تازه‌ترین اخبار چند ثانیه پیش را در نتایج خود مشاهده می‌کند.

گام چهارم: مدیریت سرریز و ادغام دوره‌ای (Periodic Merging)

از آنجا که ظرفیت **RAM** محدود است، نمایه کمکی نمی‌تواند تا ابد رشد کند. زمانی که حجم نمایه کمکی به آستانه مشخصی رسید، سیستم در پس‌زمینه (**Background**)، این نمایه را با نمایه اصلی روی دیسک ادغام می‌کند تا یک نمایه اصلی جدید و به‌روزرسانی‌شده بسازد. پس از پایان ادغام، نمایه کمکی از **RAM** پاک شده و آماده دریافت اسناد جدید می‌گردد.

بخش 4) نتیجه‌گیری و نکات کلیدی

موازنه هزینه (Trade-off): استفاده از نمایه کمکی، هزینه سنگین «نوشتن و به‌روزرسانی روی دیسک» را با یک هزینه

سبک‌تر در «خواندن و پردازش پرس‌وجو» معاوضه می‌کند. در این روش، نوشتن (ورود سند) بسیار سریع است، اما

خواندن (جستجو) کمی کندتر می‌شود زیرا سیستم مجبور است همیشه دو نمایه را جستجو و نتایج را ترکیب کند.

مدیریت حذف اسناد: در این سیستم‌ها، برای حذف یک سند قدیمی، معمولاً از یک «لیست سیاه»

(**Invalidation Bit-vector**) استفاده می‌شود. اگر سندی حذف شود، شناسه آن در این لیست قرار می‌گیرد و هنگام

جستجو، سیستم به‌سادگی اسنادی که در این لیست هستند را از نتایج نهایی فیلتر می‌کند.

فرض کنید یک دایرةالمعارف قطور و چاپ شده (نمایه اصلی) دارید که تمام اطلاعات سال‌های گذشته در آن است. چاپ مجدد این کتاب برای هر اتفاق جدید غیرمنطقی و زمان‌بر است. به جای این کار، شما یک دفترچه یادداشت کوچک (نمایه کمکی) در کنار آن قرار می‌دهید و وقایع روزانه را سریعاً در آن می‌نویسید. هر بار که می‌خواهید درباره موضوعی تحقیق کنید، هم کتاب و هم دفترچه را می‌خوانید. در پایان سال، اطلاعات دفترچه را با کتاب ترکیب کرده و نسخه جدید دایرةالمعارف را چاپ می‌کنید.

پاسخنامه مسئله 4.10: نمایه‌های موقعیتی (Positional Indexes)

بخش 1) تحلیل صورت مسئله

این مسئله به بررسی یکی از محدودیت‌های اساسی نمایه معکوس استاندارد (Standard Inverted Index) در پاسخ‌گویی به نیازهای پیشرفته‌ی جستجو می‌پردازد. هدف از این بخش، درک ضرورت ارتقای ساختار داده‌ای نمایه برای ثبت و نگهداری اطلاعات مکانی واژگان در اسناد، و شناخت انواع پرس‌وجوهایی (Queries) است که موتورهای جستجو تنها به کمک این ساختار ارتقایافته قادر به پردازش آن‌ها هستند.

بخش 2) مفاهیم پایه و پیش‌نیاز

مدل کیسه کلمات (Bag of Words): مدلی ساده در بازیابی اطلاعات که در آن ترتیب، ساختار و جایگاه کلمات در متن نادیده گرفته می‌شود و تنها معیار ارزیابی، «حضور» یا «عدم حضور» یک کلمه در سند است.

موقعیت توکن (Token Position): جایگاه یا شاخص (Index) عددی یک کلمه در دنباله‌ی کلمات یک متن. به عنوان مثال، در جمله «بازیابی اطلاعات جذاب است»، موقعیت توکن «بازیابی» برابر با 1 و توکن «اطلاعات» برابر با 2 است.

بخش 3) حل گام به گام مسئله

گام اول: بررسی محدودیت نمایه معکوس استاندارد

در یک نمایه معکوس استاندارد، لیست ارجاع (Posting List) برای هر واژه، تنها شامل شناسه‌ی اسنادی (DocID) است که آن واژه حداقل یک بار در آن‌ها تکرار شده است (و در حالت کمی پیشرفته‌تر، فرکانس تکرار کلمه در آن سند). ساختار این نمایه به شکل زیر است:

$$\langle \text{Term} \rightarrow [\text{DocID}_1, \text{DocID}_2, \text{DocID}_3, \dots] \rangle$$

محدودیت اصلی: اگر کاربری عبارت دقیق «دانشگاه تهران» را جستجو کند، سیستم با استفاده از نمایه استاندارد تنها می‌تواند اسنادی را برگرداند که کلمات «دانشگاه» و «تهران» هر دو در آن‌ها وجود داشته باشند. اما سیستم هیچ راهی برای تشخیص این موضوع ندارد که آیا این دو کلمه دقیقاً کنار هم و به همین ترتیب آمده‌اند، یا اینکه کلمه «دانشگاه» در پاراگراف اول و «تهران» در صفحه آخر سند نوشته شده است.

گام دوم: ساختار و تفاوت نمایه موقعیتی (Positional Index)

برای حل این محدودیت حیاتی، نمایه موقعیتی طراحی شد. تفاوت بنیادین در این است که لیست ارجاع در نمایه موقعیتی، علاوه

بر ثبت شناسه سند، لیستی از تمام موقعیت‌های دقیق (**Positions**) که آن واژه در آن سند ظاهر شده است را نیز در خود ذخیره می‌کند. ساختار این نمایه به شکل زیر ارتقا می‌یابد:

$\langle \text{Term} \rightarrow [\text{DocID}_1 : [\text{pos}_1, \text{pos}_2], \text{DocID}_2 : [\text{pos}_3], \dots] \rangle$

به‌عنوان مثال، ثبت اطلاعات برای کلمه «دانشگاه» ممکن است به این شکل باشد: **[Doc1: [14, 85, 120]]**. این یعنی کلمه مورد نظر در سند شماره 1، به‌عنوان چهاردهمین، هشتاد و پنجمین و صد و بیستمین کلمه ظاهر شده است.

گام سوم: پشتیبانی از پرس‌وجوهای خاص

با در اختیار داشتن مختصات دقیق و آرایه موقعیت‌های هر کلمه، سیستم می‌تواند دو نوع بسیار مهم و پرکاربرد از پرس‌وجوها را پردازش کند:

1. پرس‌وجوهای عبارتی (Phrase Queries): جستجوی عبارات دقیق و پیوسته (که معمولاً در موتورهای جستجو داخل گیومه قرار می‌گیرند، مانند «بازیابی اطلاعات»). در زمان پردازش این کوئری، الگوریتم لیست ارجاع هر دو کلمه را می‌خواند و بررسی می‌کند که اگر کلمه اول در سند مشترک در موقعیت **k** قرار دارد، کلمه دوم در همان سند حتماً در موقعیت **k+1** حضور داشته باشد.

2. پرس‌وجوهای مجاورتی (Proximity Queries): جستجویی که در آن کاربر تعیین می‌کند دو کلمه باید در نزدیکی یکدیگر باشند (با یک فاصله حداکثری مشخص). به‌عنوان مثال، جستجوی پردازش **3 /** سیگنال به این معناست که کلمات «پردازش» و «سیگنال» با فاصله حداکثر 3 کلمه از یکدیگر در متن حضور داشته باشند. محاسبه این شرط به‌لحاظ ریاضی تنها با داشتن موقعیت‌ها و بررسی رابطه $|\text{pos}_{\text{word1}} - \text{pos}_{\text{word2}}| \leq 3$ امکان‌پذیر است.

بخش 4) نتیجه‌گیری و نکات کلیدی

موازنه فضا در برابر دقت (Space-Precision Trade-off): نمایه موقعیتی اطلاعات ارزشمندی از هندسه متن ارائه می‌دهد، اما در مقابل، حجم لیست‌های ارجاع را به‌شدت افزایش می‌دهد (گاهی 2 تا 4 برابر بزرگ‌تر از نمایه استاندارد). این امر نیازمند فضای ذخیره‌سازی بیشتر روی دیسک و عملیات فشرده‌سازی پیچیده‌تر است. با این وجود، به دلیل اهمیت بالای جستجوی عبارتی برای کاربران، استفاده از آن در موتورهای جستجوی مدرن امری استاندارد و اجتناب‌ناپذیر محسوب می‌شود.

الگوی ذهنی:

نمایه استاندارد را مانند این در نظر بگیرید که سیستم به شما می‌گوید «کتاب آموزش پایتون» در طبقه سوم کتابخانه وجود دارد. اما نمایه موقعیتی فراتر رفته و به شما می‌گوید این کتاب در طبقه سوم، قفسه دوم، ردیف پنجم و جایگاه چهارم قرار دارد. تنها با داشتن این آدرس دقیق (مختصات) است که شما می‌توانید بدون گشتن کل طبقه، بررسی کنید که آیا «کتاب آموزش پایتون» و «کتاب طراحی الگوریتم» دقیقاً در یک قفسه و کنار هم چیده شده‌اند (پرس‌وجوی عبارتی) یا خیر.

پاسخنامه مسئله 4.11: محاسبه زمان خواندن از دیسک (Disk Read Time Calculation)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، اثبات کمی و ریاضی برتری خواندن پیوسته (**Sequential**) بر خواندن تصادفی (**Random**) است. در این تمرین، با محاسبه دقیق زمان انتقال و زمان موقعیتیابی (**Seek Time**)، نشان داده می‌شود که پراکندگی داده‌ها روی دیسک تا چه اندازه می‌تواند منجر به افت کارایی سیستم‌های بازیابی اطلاعات شود.

بخش 2) مفاهیم پایه و پیش‌نیاز

برای محاسبه زمان کل خواندن داده‌ها از دیسک (T_{total})، دو مؤلفه اصلی زیر با هم جمع می‌شوند (در این مسئله با فرض چشم‌پوشی از زمان تأخیر چرخشی یا نهفته بودن آن در میانگین زمان **Seek**):

1. زمان موقعیتیابی کل (T_{seek_total}): مدت زمانی که هد دیسک برای پیدا کردن مکان شروع فایل‌ها صرف می‌کند.

این زمان برابر است با تعداد فایل‌ها (تعداد دفعات جستجو) ضرب‌در زمان یک جستجو.

2. زمان انتقال کل ($T_{transfer}$): مدت زمانی که صرف خواندن خود داده‌ها با حداکثر سرعت دیسک می‌شود.

داده‌های مسئله:

میانگین زمان جستجو $(T_{seek}) = 5 \text{ ms} = 0.005 \text{ s}$

نرخ انتقال داده $(R) = 50 \text{ MB/s}$

حجم کل داده‌ها $= 100 \text{ MB}$

فرمول‌های محاسباتی به این شرح است:

$$T_{total} = (n \times T_{seek}) + T_{transfer}$$

(متغیر **n** نشان‌دهنده تعداد دفعات جابه‌جایی مکانیکی هد روی دیسک است).

بخش 3) حل گام‌به‌گام مسئله

الف) محاسبه برای یک فایل یکپارچه 100 MB

در این حالت، داده‌ها به‌صورت کاملاً پیوسته (**Sequential**) روی دیسک قرار دارند.

گام اول (محاسبه زمان موقعیتیابی): چون تنها یک فایل پیوسته داریم، هد دیسک فقط **1 بار** نیاز به جابه‌جایی دارد تا نقطه شروع فایل را پیدا کند.

$$T_{seek_total} = 1 \times 0.005 = 0.005 \text{ s}$$

گام دوم (محاسبه زمان انتقال):

$$T_{transfer} = \frac{100 \text{ MB}}{50 \text{ MB/s}} = 2 \text{ s}$$

گام سوم (محاسبه زمان کل):

$$T_{\text{total}} = 0.005 + 2 = 2.005 \text{ s}$$

نتیجه بخش الف: خواندن فایل یکپارچه 2.005 ثانیه زمان می‌برد.

ب) محاسبه برای 1000 فایل پراکنده 100 KB

در این حالت، حجم کل داده‌ها همچنان برابر با **100 MB** است ($1000 \times 100 \text{ KB} \approx 100 \text{ MB}$)، اما داده‌ها به صورت تصادفی (**Random**) در نقاط مختلف دیسک پراکنده شده‌اند.

گام اول (محاسبه زمان موقعیتیابی): برای خواندن 1000 فایل پراکنده، سیستم مجبور است **1000 بار** هد مکانیکی دیسک را جابه‌جا کند.

$$T_{\text{seek_total}} = 1000 \times 0.005 = 5 \text{ s}$$

گام دوم (محاسبه زمان انتقال): از آنجایی که حجم کل داده‌های خوانده‌شده تغییری نکرده است، زمان انتقال خالص ثابت می‌ماند.

$$T_{\text{transfer}} = 2 \text{ s}$$

گام سوم (محاسبه زمان کل):

$$T_{\text{total}} = 5 + 2 = 7 \text{ s}$$

نتیجه بخش ب: خواندن فایل‌های پراکنده 7 ثانیه زمان می‌برد.

بخش 4) نتیجه‌گیری و نکات کلیدی

مقایسه کارایی: با مقایسه دو عدد به دست آمده (**7 s** در برابر **2.005 s**)، مشخص می‌شود که خواندن فایل‌های پراکنده

در این سناریو، فرآیند را حدود **3.5 برابر کندتر** کرده است. در این حالت، بخش اعظم زمان (5 ثانیه) صرفاً هدِ حرکت

مکانیکی قطعات دیسک شده و پردازنده در این مدت بیکار (**Idle**) می‌ماند.

کاربرد در ساخت نمایه: این محاسبه ریاضی دقیقاً دلیل استفاده از تکنیک‌هایی مانند ادغام بلاک‌ها (**Merging**) و

نوشتن پیوسته لیست‌های ارجاع (**Posting Lists**) در سیستم‌های بازیابی اطلاعات را توجیه می‌کند.

الگوی ذهنی:

به عنوان یک مهندس نرم افزار، همواره فرمول $T_{\text{total}} = T_{\text{seek}} + T_{\text{transfer}}$ را در ذهن داشته باشید. شما هرگز نمی‌توانید سرعت انتقال فیزیکی دیسک (T_{transfer}) را با کدنویسی افزایش دهید؛ تنها هنر شما در طراحی ساختمان داده‌ها، کاهش ضریب پنهان در T_{seek} با تبدیل درخواست‌های متعدد و کوچک **I/O**، به درخواست‌های تجمیع‌شده و بزرگ است.

پاسخنامه مسئله 4.12: جدول وارون‌سازی در BSBI (Inversion Table in BSBI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، شبیه‌سازی عملی و گام‌به‌گام هسته مرکزی الگوریتم **BSBI (Blocked Sort-Based Indexing)** در محیط حافظه اصلی (**RAM**) است. در این فرآیند، نشان می‌دهیم که چگونه یک جریان خام از توکن‌های استخراج‌شده از اسناد (که دیدگاهی سند-محور دارند)، از طریق عملیات مرتب‌سازی و ادغام، به یک ساختار داده‌ای بهینه‌شده و واژه-محور (لیست ارجاع) تبدیل می‌شود تا برای نوشته شدن روی دیسک آماده گردد.

بخش 2) مفاهیم پایه و پیش‌نیاز

جفت‌های خام: خروجی اولیه فاز تجزیه (**Parsing**)، جفت‌هایی به شکل **(TermID, DocID)** است.

مرتب‌سازی دوگانه (Dual-key Sorting): برای ساخت نمایه، داده‌ها باید ابتدا بر اساس یک «کلید اصلی» (**Primary Key**) که همان **TermID** است، مرتب شوند. سپس در صورت برابری شناسه‌های واژه، مرتب‌سازی بر اساس «کلید ثانویه» (**Secondary Key**) یعنی **DocID** انجام می‌گیرد.

وارون‌سازی (Inversion / Grouping): فرآیند تجمیع تمامی شناسه‌های سند مرتبط با یک واژه در قالب یک آرایه واحد، که منجر به تولید لیست ارجاع (**Posting List**) می‌شود.

بخش 3) حل گام‌به‌گام مسئله

گام اول: بازنویسی و استانداردسازی جفت‌ها

ابتدا جدول داده‌شده در صورت‌مسئله را که به‌صورت **(DocID, TermID)** است، به استاندارد پردازشی **BSBI** یعنی **(TermID, DocID)** تبدیل می‌کنیم تا فرآیند مرتب‌سازی تسهیل شود. وضعیت حافظه در این مرحله به شکل زیر است (ترتیب ورود حفظ شده است):

ردیف	TermID	DocID
1	45	1
2	12	2
3	12	1
4	45	3
5	45	2

گام دوم: فرآیند مرتب‌سازی (Sorting)

اکنون الگوریتم مرتب‌سازی (مانند **Quick Sort**) روی جدول اعمال می‌شود:

1. ابتدا کل جدول به‌صورت صعودی بر اساس **TermID** مرتب می‌شود (واژه شماره 12 قبل از واژه شماره 45 قرار می‌گیرد).

2. سپس برای ردیف‌هایی که **TermID** یکسان دارند، **DocID** به‌صورت صعودی مرتب می‌شود.

خروجی این مرحله (جدول مرتب‌شده در حافظه) به شرح زیر است:

ردیف	TermID	DocID	توضیحات مرتب‌سازی
1	12	1	گروه اول (شناسه 12) - مرتب‌شده بر اساس سند

ردیف	TermID	DocID	توضیحات مرتب‌سازی
2	12	2	گروه اول (شناسه 12) - مرتب‌شده بر اساس سند
3	45	1	گروه دوم (شناسه 45) - مرتب‌شده بر اساس سند
4	45	2	گروه دوم (شناسه 45) - مرتب‌شده بر اساس سند
5	45	3	گروه دوم (شناسه 45) - مرتب‌شده بر اساس سند

گام سوم: ادغام و تشکیل جدول نهایی (Grouping & Inverting)

در این مرحله، الگوریتم جدول مرتب‌شده را از بالا به پایین پیمایش می‌کند. هر زمان که به یک **TermID** جدید می‌رسد، یک لیست ارجاع جدید ایجاد کرده و **DocID**ها را به آن الحاق (**Append**) می‌کند. به محض تغییر **TermID**، لیست قبلی بسته شده و لیست جدیدی آغاز می‌شود.

جدول نهایی (Posting List) که روی دیسک نوشته خواهد شد:

شناسه واژه (TermID)	لیست ارجاع (Posting List - DocIDs)
12	[1, 2]
45	[1, 2, 3]

بخش 4) نتیجه‌گیری و نکات کلیدی

اهمیت مرتب‌سازی محلی: این مثال ساده به‌روشنی نشان می‌دهد که چرا در الگوریتم **BSBI** به آن "**Sort-Based**" (مبتنی بر مرتب‌سازی) می‌گویند. بدون انجام دقیق مرتب‌سازی در گام دوم، امکان گروه‌بندی سریع و خطی (با مرتبه زمانی $O(N)$ در گام سوم) وجود نخواهد داشت.

آماده‌سازی برای ادغام نهایی: خروجی نهایی این مسئله، دقیقاً فرمت یک «بلوک موقت» است که روی دیسک ذخیره می‌شود. صعودی بودن شناسه‌های سند در داخل گروه‌ها []، تضمین می‌کند که در فاز نهایی سیستم (**Merge Phase**)، الگوریتم بتواند لیست‌های ارجاع از بلوک‌های مختلف را در زمان خطی با یکدیگر اشتراک یا اجتماع بگیرد.

الگوی ذهنی:

این فرآیند را مانند دسته‌بندی فاکتورهای فروش در نظر بگیرید. در گام اول، فاکتورها به‌صورت درهم‌ریخته روی میز هستند. در گام دوم، شما ابتدا فاکتورها را بر اساس نام کالا (**TermID**) دسته‌بندی می‌کنید و سپس درون هر دسته، آن‌ها را بر اساس تاریخ یا شماره فاکتور (**DocID**) مرتب می‌چینید. در گام نهایی، فاکتورهای هر کالا را داخل یک پوشه مجزا (**Posting List**) قرار داده و روی پوشه نام آن کالا را می‌نویسید.

پاسخنامه مسئله 4.13: فاز ادغام بلوک‌ها (Block Merge Phase)

بخش 1) تحلیل صورت مسئله

این مسئله به آخرین و یکی از حیاتی‌ترین مراحل الگوریتم نمایه‌سازی مبتنی بر بلوک (**BSBI**) می‌پردازد. پس از آنکه حافظه اصلی (**RAM**) طی مراحل قبل پر و تخلیه شد، ما با تعدادی بلوک موقت (در اینجا 4 بلوک **B₁, B₂, B₃, B₄**) روی دیسک مواجه هستیم که هر کدام به‌تنهایی مرتب شده‌اند. هدف این مسئله، تشریح مکانیزم ادغام چندگانه (**N-way Merge**) است که چگونه این قطعات پراکنده را با کمترین هزینه **I/O** (تنها با 1 بار خواندن و 1 بار نوشتن) به یک نمایه معکوس یکپارچه تبدیل می‌کند.

بخش 2) مفاهیم پایه و پیش‌نیاز

ادغام چندگانه (N-way Merge): الگوریتمی که به‌جای ادغام 2-به-2 (که نیازمند خواندن و نوشتن‌های مکرر روی دیسک است)، **N** فایل مرتب‌شده را به‌طور همزمان باز کرده و با استفاده از یک ساختار داده‌ای واسط (مانند صف اولویت یا **Min-Heap**)، آن‌ها را مستقیماً در یک فایل نهایی ادغام می‌کند.

بافر (Buffer): از آنجا که نمی‌توانیم کل بلوک‌ها را مجدداً به‌صورت یکجا در **RAM** بارگذاری کنیم، سیستم تنها بخش کوچکی از هر بلوک را در فضاهای موقتی به نام «بافر خواندن» (**Read Buffer**) در حافظه قرار می‌دهد.

بخش 3) حل گام‌به‌گام مسئله

گام اول: رسم گراف درختی فرآیند ادغام (N-way Merge Tree)

ساختار جریان داده و تخصیص حافظه در این معماری به شکل زیر ترسیم می‌شود:



گام دوم: تشریح مراحل اجرایی ادغام

فرآیند ادغام همزمان این 4 بلوک بر اساس گراف فوق، طی مراحل زیر انجام می‌پذیرد:

1. تخصیص حافظه (Initialization): سیستم 4 بافر خواندن برای بلوک‌های **B₁** تا **B₄** و 1 بافر نوشتن برای خروجی در

RAM اختصاص می‌دهد. سپس اولین عنصر (جفت شناسه واژه و لیست ارجاع) از هر بلوک خوانده شده و وارد «صف

اولویت» می‌شود.

2. مکان‌یابی کمترین مقدار (Extract-Min): از آنجا که بلوک‌ها از پیش مرتب شده‌اند، ساختار **Min-Heap** در هر

لحظه، واژه‌ای که از نظر ترتیب الفبایی (یا ارزش عددی **TermID**) کوچک‌ترین است را شناسایی و استخراج می‌کند.

3. ادغام لیست‌های ارجاع (Merging Posting Lists): اگر یک واژه مشخص (مثلاً **TermID = 12**) به صورت همزمان

در بالای سر چند بافر (مثلاً بافر 1 و بافر 3) قرار داشته باشد، الگوریتم لیست‌های ارجاع آن‌ها را با یکدیگر ترکیب

(**Union**) می‌کند. از آنجا که شناسه‌های سند درون این لیست‌ها قبلاً در فاز اول مرتب شده‌اند، این ترکیب در زمان خطی

انجام می‌شود.

4. نوشتن در خروجی (Flushing to Output): واژه نهایی به همراه لیست ارجاع یکپارچه‌اش به بافر خروجی منتقل

می‌شود. هر زمان که بافر خروجی پر شود، محتویات آن به عنوان بخشی از «نمایه نهایی» روی دیسک نوشته می‌شود.

5. تغذیه مجدد بافرها (Refilling): به محض اینکه یک عنصر از بافر خواندن استخراج شد، عنصر بعدی از همان بلوک

روی دیسک، خوانده شده و وارد صف اولویت می‌گردد. این چرخه تا زمانی که تمامی 4 بلوک به طور کامل خوانده و خالی

شوند، ادامه می‌یابد.

بخش 4) نتیجه‌گیری و نکات کلیدی

بهینه‌سازی I/O دیسک: مهم‌ترین دستاورد ساختار **N-way Merge** این است که هر بلوک موقت روی دیسک دقیقاً 1

بار خوانده می‌شود و نمایه نهایی نیز دقیقاً 1 بار نوشته می‌شود. این در حالی است که اگر از ادغام باینری (درختی

کلاسیک) استفاده می‌کردیم، داده‌ها باید چندین بار خوانده و در فایل‌های میانی نوشته می‌شدند که زمان اجرای

الگوریتم را به شدت افزایش می‌داد.

پیچیدگی زمانی (Time Complexity): استفاده از **Min-Heap** باعث می‌شود تا هزینه پیدا کردن کوچک‌ترین واژه در

میان **N** بلوک بسیار ناچیز باشد (پیچیدگی زمانی استخراج از هیپ برابر با $O(\log N)$ است). بنابراین، پردازنده با

سرعت بسیار بالایی فرآیند ادغام را پیش می‌برد و گلوگاه سیستم تنها به سرعت خواندن مکانیکی دیسک محدود

می‌ماند.

الگوی ذهنی:

این مکانیزم را مانند 4 نوار نقاله (بلوک‌های **B₁** تا **B₄**) در نظر بگیرید که محصولات مرتب‌شده‌ای را به سمت یک اپراتور مرکزی (

Min-Heap) می‌آورند. اپراتور همیشه به ابتدای هر 4 نوار نگاه می‌کند، محصولی که شماره کمتری دارد را برمی‌دارد، در یک جعبه

نهایی (بافر خروجی) بسته‌بندی می‌کند و روی نوار نقاله خروجی قرار می‌دهد. نوار نقاله‌ها مدام جلو می‌آیند تا زمانی که هیچ

محصولی باقی نماند.

پاسخنامه مسئله 4.14: واکنش سیستم به پر شدن حافظه در SPIMI

(Memory Exhaustion in SPIMI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، بررسی دقیق مکانیزم مدیریت حافظه و انتقال فاز از «پردازش درون حافظه‌ای» به «عملیات دیسک» در

الگوریتم **SPIMI** است. درک این لحظه بحرانی (**Memory Full**) نشان می‌دهد که سیستم چگونه بدون از دست دادن داده‌ها و با

کمترین بار پردازشی، یک بلوک نمایه موقت را تولید کرده و حافظه را برای ادامه کار آزادسازی می‌کند.

آستانه حافظه (Memory Threshold): در سیستم‌های نمایه‌سازی، معمولاً 100 درصد ظرفیت **RAM** اشغال نمی‌شود، بلکه یک آستانه امن (مثلاً 80 درصد ظرفیت مجاز) تعیین می‌گردد. به محض رسیدن به این نقطه، سیستم وارد فاز تخلیه می‌شود تا از بروز خطای سرریز حافظه (**Out of Memory**) یا استفاده از حافظه مجازی (**Swap**) جلوگیری کند.

فایل بلوک میانی (Intermediate Block): فایلی موقت روی دیسک که حاوی نمایه معکوس مربوط به بخشی از اسناد پردازش‌شده تا آن لحظه است.

بخش 3) حل گام‌به‌گام مسئله

سیستم در لحظه پر شدن حافظه، یک توالی دقیق و خطی از عملیات زیر را به اجرا درمی‌آورد:

گام اول: توقف موقت پردازش (Pausing Input)

به محض برخورد با آستانه حافظه، سیستم بلافاصله خواندن اسناد متنی جدید و تجزیه آن‌ها به توکن‌ها را متوقف می‌کند. وضعیت فعلی سند در حال پردازش (نشانگر متن) دقیقاً ذخیره می‌شود تا بعداً از همان نقطه کار از سر گرفته شود.

گام دوم: مرتب‌سازی واژه‌نامه محلی (Lexicon Sorting)

الگوریتم به سراغ جدول درهم‌سازی (**Hash Table**) درون حافظه می‌رود. در این مرحله، لیست‌های ارجاع (**Posting Lists**) مرتب نمی‌شوند (زیرا از قبل و در زمان ورود داده‌ها به‌صورت صعودی مرتب شده‌اند). سیستم تنها کلیدهای واژه‌نامه (رشته‌های متنی کلمات) را استخراج کرده و آن‌ها را به ترتیب حروف الفبا مرتب می‌کند.

گام سوم: تخلیه روی دیسک (Flushing to Disk)

سیستم یک فایل جدید روی دیسک سخت باز می‌کند (مثلاً با نام `Block_1.idx`). سپس با پیمایش واژه‌نامه مرتب‌شده، کلمات و لیست ارجاع متصل به آن‌ها را به‌صورت یک جریان داده‌ی پیوسته (**Sequential Write**) درون این فایل می‌نویسد.

گام چهارم: آزادسازی حافظه (Memory Clearance)

پس از اطمینان از نوشته شدن کامل اطلاعات روی دیسک و بسته شدن فایل، سیستم تمام متغیرها، جدول درهم‌سازی و آرایه‌های تخصیص‌یافته برای لیست‌های ارجاع را در **RAM** به‌طور کامل پاک‌سازی (**Deallocate**) می‌کند. اکنون حافظه دوباره خالی و در دسترس است.

گام پنجم: ازسرگیری فرآیند (Resuming)

سیستم به نقطه‌ای که در گام اول متوقف شده بود بازمی‌گردد، یک جدول درهم‌سازی جدید و کاملاً خالی در **RAM** ایجاد می‌کند و خواندن ادامه اسناد را از سر می‌گیرد.

فایل‌های تولید شده در این فرآیند:

در پایان این چرخه، تنها 1 فایل یکپارچه (نمایه موقت بلوک) تولید می‌شود. این فایل شامل جفت‌های ساختاریافته به شکل `Term` [DocID_1, DocID_2, ...] : است که واژگان در آن به ترتیب حروف الفبا چیده شده‌اند.

بخش 4) نتیجه‌گیری و نکات کلیدی

کارایی زمانی: واکنش سیستم به پر شدن حافظه در **SPIMI** بسیار سریع‌تر از **BSBI** است. از آنجا که نیازی به

مرتب‌سازی آرایه‌های عظیم از توکن‌ها نیست، پردازنده تنها زمان کوتاهی را صرف مرتب کردن واژگان یکتا

(Unique Terms) کرده و بخش عمده‌ی زمانِ این واکنش، صرفاً معطوف به سرعت نوشتن مکانیکی روی دیسک (I/O) می‌شود.

استقلال بلوک‌ها: هر بار که حافظه پر می‌شود، یک بلوک کاملاً مستقل روی دیسک متولد می‌گردد. این بلوک‌ها هیچ وابستگی ساختاری به یکدیگر ندارند و وظیفه ارتباط دادن آن‌ها به یکدیگر، کاملاً به فاز نهایی سیستم (فاز ادغام یا Merge) محول می‌شود.

پاسخنامه مسئله 4.15: مقایسه BSBI و SPIMI (BSBI vs. SPIMI Comparison)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، جمع‌بندی و مقایسه تحلیلی میان دو رویکرد بنیادین در نمایه‌سازی مبتنی بر دیسک است. این مقایسه نشان می‌دهد که چگونه تغییر در معماری ساختمان داده‌ها (از آرایه‌های ایستا به ساختارهای پویا) می‌تواند گلوگاه‌های پردازشی و محدودیت‌های حافظه را برطرف سازد. این جدول به‌عنوان یک چکیده کاربردی برای درک تفاوت‌های عملیاتی این دو الگوریتم عمل می‌کند.

بخش 2) مفاهیم پایه و پیش‌نیاز

الگوریتم BSBI: رویکردی که بر جمع‌آوری جفت‌های خام، تبدیل واژگان به شناسه عددی (**TermID**) و در نهایت یک مرتب‌سازی سنگین و سراسری در سطح بلوک استوار است.

الگوریتم SPIMI: رویکردی مدرن‌تر که واژه‌نامه و لیست‌های ارجاع را به‌صورت در لحظه (**On-the-fly**) درون حافظه می‌سازد و نیازی به تبدیل شناسه سراسری یا مرتب‌سازی اولیه جفت‌ها ندارد.

بخش 3) حل گام‌به‌گام مسئله

ویژگی	BSBI	SPIMI
نیاز به دیکشنری سراسری	دارد (بله) نیازمند نگهداری یک ساختار داده سراسری در RAM برای تبدیل رشته‌ها به اعداد جهت کاهش حجم داده‌ها پیش از مرتب‌سازی است.	ندارد (خیر) هر بلوک واژه‌نامه محلی (رشته‌ای) مختص به خود را می‌سازد و در انتهای پردازش بلوک، این واژه‌نامه از حافظه پاک می‌شود.
سرعت پردازش اولیه	کندتر به دلیل سربار ناشی از نگاشت شناسه‌ها و نیاز به مرتب‌سازی عظیم آرایه جفت‌ها (با پیچیدگی $O(N \log N)$) در پایان هر بلوک.	سریع‌تر کلمات به‌صورت خطی پردازش شده و شناسه‌های سند مستقیماً با پیچیدگی زمانی $O(1)$ به انتهای لیست‌ها الحاق می‌شوند.
ساختار Posting List	آرایه با طول ثابت (جفت‌های خام در یک آرایه یکپارچه ذخیره شده و پس از مرتب‌سازی به لیست تبدیل می‌شوند).	آرایه پویا (Dynamic Array) طول نهایی لیست ارجاع برای هر واژه از پیش مشخص نیست و به‌تدریج رشد می‌کند.

نیاز به دیکشنری سراسری: در **BSBI** اگر کلمات به **TermID** تبدیل نشوند، مرتب‌سازی رشته‌های متنی به شدت کند و حافظه بر خواهد بود؛ اما **SPIMI** چون نیازی به مرتب‌سازی توکن‌ها ندارد، مستقیماً با خود رشته‌های متنی کار می‌کند.

سرعت پردازش: عملیات اصلی در **SPIMI** جستجو در جدول درهم‌سازی و الحاق یک عدد است، اما در **BSBI**، پردازنده باید میلیون‌ها رکورد را جابه‌جا کند.

ساختار حافظه: مهم‌ترین چالش پیاده‌سازی **SPIMI**، مدیریت حافظه تخصیص‌یافته به لیست‌های ارجاع است که باید به صورت یک «آرایه پویا» پیاده‌سازی شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

تکامل الگوریتمی: مقایسه این دو الگوریتم نشان‌دهنده یک بلوغ در مهندسی نرم‌افزار است: حرکت از «ذخیره‌سازی و پردازش دسته‌ای» (Batch Processing در **BSBI**) به سمت «ساختاردهی در لحظه و پویا» (Stream Processing در **SPIMI**).

انتخاب استاندارد: به دلیل برتری قاطع **SPIMI** در مدیریت حافظه مستقل و سرعت بالاتر در تولید بلوک‌ها، این الگوریتم امروزه به عنوان رویکرد استاندارد در ساخت موتورهای جستجو متن‌باز (مانند **Lucene**) و تجاری مورد استفاده قرار می‌گیرد.

الگوی ذهنی:

تفاوت ساختاری این دو را در ذهن خود به این شکل تثبیت کنید: الگوریتم **BSBI** ابتدا داده‌ها را در یک ساختار ساده (آرایه ثابت) می‌ریزد اما در انتها هزینه سنگینی برای مرتب‌سازی می‌پردازد. در مقابل، الگوریتم **SPIMI** هزینه را در طول مسیر و صرف مدیریت ساختارهای داده‌ی پیچیده‌تر (آرایه‌های پویا و جداول درهم‌سازی) می‌کند، اما در انتها بدون نیاز به هیچ پردازش اضافه‌ای، خروجی مرتب‌شده را تحویل می‌دهد.

پاسخنامه مسئله 4.16: جریان داده در MapReduce (Data Flow in MapReduce)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک بصری و عملی معماری توزیع‌شده‌ی **MapReduce** در فرآیند ساخت نمایه است. در این سناریو، یک پیکره‌ی کوچک (شامل 10 سند) در یک کلاستر پردازشی با 3 گره نگاشت (**Map Nodes**) و 2 گره کاهش (**Reduce Nodes**) پردازش می‌شود. رسم گراف جریان داده نشان می‌دهد که چگونه داده‌های خام شکسته شده، پردازش می‌شوند و سپس بر اساس یک منطق مسیریابی شبکه‌ای (**Shuffle**) تجمیع می‌گردند تا لیست‌های ارجاع نهایی ساخته شوند.

بخش 2) مفاهیم پایه و پیش‌نیاز

گره نگاشت (Mapper): ماشینی که بخشی از اسناد را دریافت کرده و جفت‌های اولیه **(Term, DocID)** را تولید می‌کند.

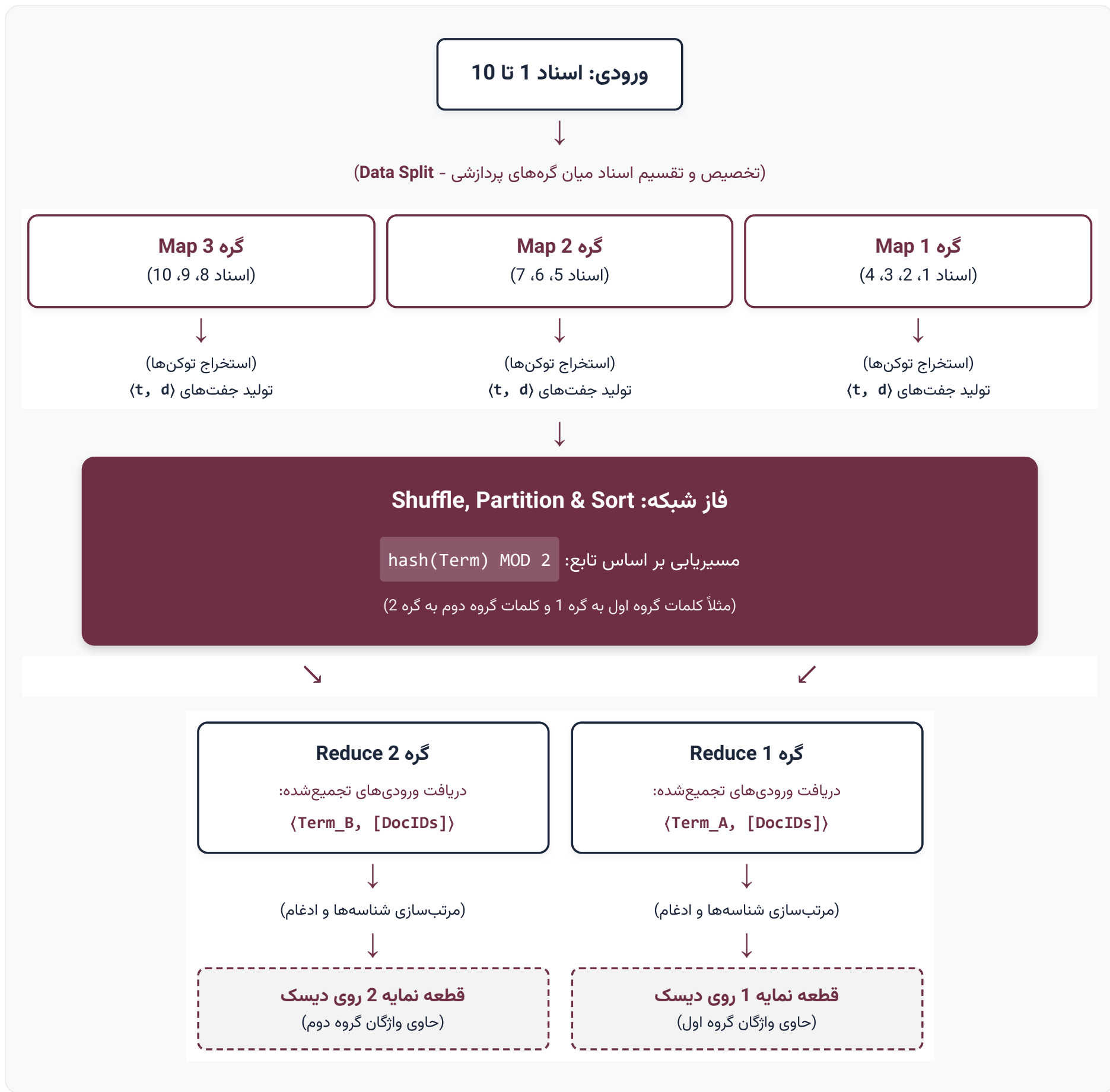
تابع درهم‌سازی (Hash Partitioning): مکانیزمی در فاز **Shuffle** که تضمین می‌کند تمام جفت‌های مربوط به یک کلمه‌ی خاص (مهم نیست توسط کدام **Mapper** تولید شده باشند)، دقیقاً به یک **Reducer** مشخص ارسال شوند. مثلاً با فرمول $\text{hash(Term)} \bmod 2$.

گره کاهش (Reducer): ماشینی که جفت‌های تجمیع‌شده را دریافت کرده و لیست ارجاع (**Posting List**) نهایی را برای گروهی از کلمات می‌سازد.

بخش 3) حل گام‌به‌گام مسئله

گام اول: رسم گراف جریان داده (Data Flow Graph)

ساختار گرافیکی توزیع و تجمیع داده‌ها در این کلاستر به شکل زیر است:



گام دوم: تشریح مراحل گراف

1. فاز توزیع اسناد (Splitting): در ابتدا، گره مدیریت کلاستر (**Master Node**)، 10 سند موجود را میان 3 گره **Map**

به‌صورت متوازن تقسیم می‌کند (مثلاً 4 سند به گره اول و 3 سند به گره‌های بعدی).

2. فاز پردازش محلی (Map Phase): هر گره **Map** به‌صورت کاملاً مستقل و موازی، اسناد خود را تجزیه کرده و جریان

عظیمی از جفت‌های کلید-مقدار را تولید می‌کند. برای مثال، اگر کلمه «وب» در سند 2 (توسط **Map 1**) و در سند 8

(توسط **Map 3**) دیده شود، هر دو گره به‌طور مستقل جفت مربوطه را تولید می‌کنند.

3. فاز جابه‌جایی و تجمیع (Shuffle): این حیاتی‌ترین بخش جریان داده است. خروجی‌های خام **Mapper**ها در بستر شبکه به حرکت درمی‌آیند. سیستم با اعمال یک تابع ریاضی (**Hash**) روی رشته‌ی متنی هر کلمه، مقصد آن را مشخص می‌کند. به دلیل وجود 2 گره **Reduce**، خروجی تابع ($\text{mod } 2$) خواهد بود. بنابراین کلمه «وب» چه در **Map 1** تولید شده باشد و چه در **Map 3**، قطعاً به یک **Reducer** واحد (مثلاً **Reduce 2**) ارسال می‌شود.

4. فاز کاهش و ذخیره‌سازی (Reduce Phase): هر یک از 2 گره **Reduce**، لیستی از کلمات مختص به خود را به همراه تمام شناسه‌های سند آن‌ها (از سراسر کلاستر) دریافت می‌کند. در اینجا شناسه‌های تکراری حذف شده، لیست نهایی ساخته شده و در نهایت 2 فایل مجزا (پارتیشن‌های نمایه) روی دیسک ذخیره می‌شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

تضمین یکپارچگی: جریان داده در **MapReduce** تضمین می‌کند که لیست ارجاع هیچ واژه‌ای دچار گسیختگی نمی‌شود. اگرچه اسناد در 3 گره مختلف پردازش شده‌اند، اما در فاز **Shuffle** تمام داده‌های مربوط به یک کلمه خاص در یک گره **Reduce** متمرکز می‌شوند تا نمایه آن کلمه به‌صورت کامل و یکپارچه شکل بگیرد.

مقیاس‌پذیری: این گراف نشان می‌دهد که اگر تعداد اسناد از 10 به 10 میلیارد افزایش یابد، تنها کافی است تعداد گره‌های **Map** و **Reduce** در کلاستر افزایش یابد؛ منطق جریان داده (**Data Flow**) و استقلال مراحل از یکدیگر، دقیقاً به همین شکل ثابت و بدون تغییر باقی خواهد ماند.

پاسخنامه مسئله 4.17: گلوگاه‌های تجزیه‌گر در نمایه‌سازی توزیع‌شده (Parser Bottlenecks)

بخش 1) تحلیل صورت مسئله

این مسئله به بررسی چالش‌های عملکردی در معماری‌های پردازش توزیع‌شده (مانند فریم‌ورک **MapReduce**) می‌پردازد. هدف، شناسایی دقیق عواملی است که مانع از کارکرد گره‌های تجزیه‌گر (**Parsers / Mappers**) با حداکثر توان پردازشی ممکن می‌شوند. از آنجا که پردازنده (**CPU**) در این گره‌ها سرعت بسیار بالایی دارد، افت کارایی همواره به دلیل محدودیت‌های فیزیکی و تأخیر در تبادل داده با دیسک سخت (**Disk I/O**) یا بستر شبکه (**Network I/O**) رخ می‌دهد.

بخش 2) مفاهیم پایه و پیش‌نیاز

گره تجزیه‌گر (Parser Node): در نمایه‌سازی توزیع‌شده، این گره وظیفه دارد بلوک‌های داده خام (اسناد) را خوانده، متن را به نشانه‌ها (**Tokens**) تجزیه کرده و در نهایت جفت‌های کلید-مقدار (کلمه، شناسه سند) را تولید کند.

گلوگاه وابسته به ورودی/خروجی (I/O Bound): وضعیتی در سیستم‌های کامپیوتری که در آن سرعت اجرای یک برنامه، نه به خاطر قدرت پردازنده، بلکه به دلیل کندی سیستم در خواندن یا نوشتن اطلاعات (روی دیسک یا شبکه) محدود می‌شود.

بخش 3) حل گام‌به‌گام مسئله

3 دلیل اصلی که منجر به ایجاد گلوگاه در گره‌های تجزیه‌گر می‌شوند عبارتند از:

دلیل اول: فقدان «محلّی بودن داده‌ها» (Lack of Data Locality) - گلوگاه شبکه در خواندن ورودی

در سیستم‌های توزیع‌شده (مانند سیستم فایل **HDFS**)، معماری استاندارد بر این اصل استوار است که کدهای پردازشی به گره‌ای ارسال شوند که داده‌ها روی دیسک سخت همان گره قرار دارند (انتقال پردازش به‌جای انتقال داده). با این حال، اگر به دلیل خرابی یک سرور یا زمان‌بندی نامناسب، گره تجزیه‌گر مجبور شود بلوک داده خام را برای پردازش، از دیسک یک گره دیگر و از طریق **بستر شبکه** بخواند، سرعت پردازش به‌شدت افت می‌کند. خواندن حجم عظیمی از داده‌های خام از طریق شبکه محلی (**LAN**) بسیار کندتر از خواندن مستقیم از دیسک محلی است و پردازنده را در وضعیت انتظار (**Wait**) قرار می‌دهد.

دلیل دوم: سرّبار سرریز داده‌های میانی (Spilling to Local Disk) - گلوگاه دیسک در نوشتن خروجی

گره‌های تجزیه‌گر (**Mappers**) خروجی خود را مستقیماً به گره‌های **Reduce** ارسال نمی‌کنند. جفت‌های کلید-مقدار تولیدشده ابتدا در یک بافر موقت درون حافظه اصلی (**RAM**) نگهداری می‌شوند. زمانی که این بافر پر می‌شود، سیستم باید پردازش را متوقف کرده و این داده‌ها را روی **دیسک سخت محلی (Local Disk)** تخلیه (**Spill**) کند. اگر متن دارای تنوع لغوی بالایی باشد و حجم توکن‌های استخراج‌شده به‌شدت افزایش یابد، تعداد دفعات تخلیه روی دیسک بالا رفته و عملیات مکانیکی نوشتن روی دیسک، تبدیل به یک گلوگاه جدی برای پردازنده می‌گردد.

دلیل سوم: ازدحام پهنای باند در فاز جابه‌جایی (Network Congestion during Shuffle) - گلوگاه شبکه در انتقال نهایی

پس از پایان عملیات تجزیه، تمام بلوک‌های تولیدشده باید به سمت گره‌های کاهش (**Reducers**) مسیریابی و ارسال شوند. در یک کلاستر بزرگ، هزاران گره تجزیه‌گر به‌طور همزمان تلاش می‌کنند تا ترابایت‌ها داده میانی را از طریق سوئیچ‌های شبکه منتقل کنند. این ترافیک همزمان و عظیم، منجر به اشباع پهنای باند شبکه (**Network Saturation**) می‌شود. در نتیجه، حتی اگر گره تجزیه‌گر کار خود را سریع تمام کرده باشد، به دلیل عدم توانایی شبکه در انتقال سریع داده‌ها، فرآیند کلی نمایه‌سازی دچار گلوگاه و تأخیر می‌شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

تقابل پردازنده و I/O: در پردازش کلان‌داده (**Big Data**)، گره‌های تجزیه‌گر به‌ندرت دچار محدودیت پردازشی

(**CPU Bound**) می‌شوند. چالش اصلی همانا تغذیه مداوم پردازنده با داده‌ها و تخلیه سریع خروجی‌های آن است.

طراحی سیستم‌های نوین: درک این گلوگاه‌ها باعث پیدایش چارچوب‌های پردازشی جدیدتر (مانند **Apache Spark**) شد

که تلاش می‌کنند با استفاده حداکثری از حافظه اصلی (**In-Memory Processing**) و کاهش مراجعه به دیسک و

شبکه در مراحل میانی، این گلوگاه‌ها را به حداقل برسانند.

الگوی ذهنی:

عملکرد گره تجزیه‌گر را مانند یک دستگاه خردکن بسیار سریع کاغذ در یک اداره پست تصور کنید. این دستگاه هرگز به خودی خود کند نمی‌شود؛ اما اگر کارمندان نتوانند بسته‌های کاغذ را به‌موقع به آن برسانند (گلوگاه خواندن از شبکه)، یا مخزن خروجی آن به‌سرعت پر شود و تخلیه آن طول بکشد (گلوگاه نوشتن روی دیسک)، و یا راهروهای اداره پست برای انتقال کیسه‌های کاغذ خردشده بیش از حد شلوغ باشد (گلوگاه شبکه در فاز انتقال)، دستگاه خردکن مجبور است متوقف شده و منتظر بماند.

پاسخنامه مسئله 4.18: ردگیری ادغام لگاریتمی (Tracing Logarithmic Merging)

بخش 1) تحلیل صورت‌مسئله

این مسئله به بررسی معماری مدیریت حافظه در «الگوریتم ادغام لگاریتمی» می‌پردازد. هدف، درک نحوه توزیع پویا و جریان انتقال اسناد از حافظه اصلی (RAM) به سطوح مختلف حافظه جانبی (Disk) در طول زمان است. این تحلیل نشان می‌دهد که چگونه سیستم با استفاده از یک استراتژی نمایی (Exponential)، از بازنویسی‌های مکرر و پرهزینه روی دیسک جلوگیری می‌کند.

بخش 2) مفاهیم پایه و پیش‌نیاز

در ادغام لگاریتمی، ما با 2 محیط ذخیره‌سازی سروکار داریم:

1. **نمایه کمکی (Auxiliary Index):** یک نمایه درون‌حافظه‌ای با ظرفیت ثابت n . تمام اسناد جدید ابتدا وارد این نمایه می‌شوند.

2. **نمایه‌های روی دیسک (Z_i):** دنباله‌ای از نمایه‌ها روی دیسک که ظرفیت هر کدام به‌صورت توانی از 2 رشد می‌کند. رابطه ظرفیت هر نمایه دیسک به شکل زیر محاسبه می‌شود:

$$\text{Capacity}(Z_i) = n \times 2^i$$

ویژگی کلیدی این الگوریتم آن است که وضعیت پر یا خالی بودن نمایه‌های دیسک در هر لحظه، دقیقاً از منطق شمارش در سیستم اعداد دودویی (Binary) پیروی می‌کند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: استخراج داده‌ها و تعیین ظرفیت سطوح دیسک

ابتدا بر اساس ظرفیت نمایه کمکی ($n = 1000$)، ظرفیت سطوح مختلف دیسک (Z_i) را محاسبه می‌کنیم:

$$\text{Capacity}(Z_0) = 1000 \times 2^0 = 1000 \text{ Docs}$$

$$\text{Capacity}(Z_1) = 1000 \times 2^1 = 2000 \text{ Docs}$$

$$\text{Capacity}(Z_2) = 1000 \times 2^2 = 4000 \text{ Docs}$$

گام دوم: تفکیک اسناد پردازش‌شده

کل اسناد پردازش‌شده 3500 عدد است. برای مشخص شدن وضعیت حافظه و دیسک، این عدد را بر ظرفیت پایه (n) تقسیم می‌کنیم:

$$\text{تعداد بلوک‌های کامل: } 3500 \div 1000 = 3$$

اسناد باقی‌مانده: 500 سند.

گام سوم: تحلیل وضعیت نمایه کمکی (درون RAM)

همان‌طور که در محاسبات گام قبل مشخص شد، 500 سند آخر هنوز به حد نصاب لازم برای تشکیل یک بلوک کامل (1000 سند) نرسیده‌اند. بنابراین، سیستم این اسناد را تخلیه نمی‌کند.

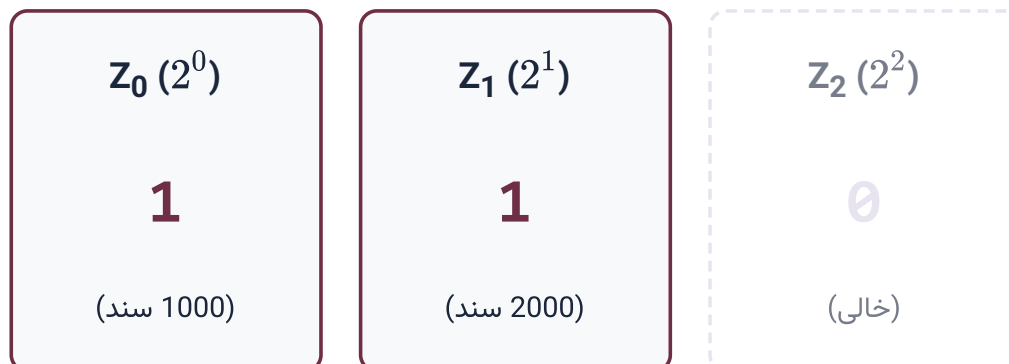
• **وضعیت نمایه کمکی:** حاوی 500 سند (نیمه‌پر).

گام چهارم: تحلیل وضعیت نمایه‌های روی دیسک (با استفاده از نگاشت باینری)

ما باید 3 بلوک کامل (در مجموع 3000 سند) را روی دیسک جانمایی کنیم. با استفاده از منطق باینری، عدد 3 در مبنای 10، معادل عدد $(11)_2$ در مبنای 2 است.

در عدد باینری $(11)_2$ ، بیت‌های جایگاه 2^0 و 2^1 روشن (برابر با 1) هستند.

روشن بودن این بیت‌ها به این معناست که سطوح متناظر آن‌ها در دیسک، کاملاً پر شده‌اند.



بنابراین:

نمایه Z_0 (متناظر با بیت اول): پر است (حاوی 1 بلوک کامل = 1000 سند).

نمایه Z_1 (متناظر با بیت دوم): پر است (حاوی 2 بلوک کامل = 2000 سند).

نمایه‌های Z_2 به بعد: خالی هستند.

بخش 4) نتیجه‌گیری و نکات کلیدی

با پردازش 3500 سند، وضعیت نهایی سیستم و ظرفیت‌های اشغال‌شده به صورت زیر جمع‌بندی می‌گردد:

در حافظه اصلی (RAM): نمایه کمکی با 500 سند در حال فعالیت است.

روی حافظه جانبی (Disk): 2 نمایه فعال وجود دارد؛ Z_0 با 1000 سند و Z_1 با 2000 سند.

الگوی ذهنی:

برای ردگیری سریع ادغام لگاریتمی در آزمون‌ها، هرگز نیازی به رسم دستی و گام‌به‌گام انتقال بلوک‌ها نیست. کفایت تعداد «بلوک‌های کامل» را به دست آورده و آن را به یک عدد باینری تبدیل کنید. هر جا عدد 1 دیدید، یعنی نمایه Z_i در آن سطح پر است، و هر جا عدد 0 بود، یعنی آن سطح خالی است و منتظر ادغام‌های بعدی مانده است. این هم‌ارزی ریاضی، پیچیدگی تحلیل این الگوریتم را به شدت ساده می‌کند.

پاسخنامه مسئله 4.19: مدیریت حذف اسناد (Handling Document Deletion)

بخش 1) تحلیل صورت مسئله

این مسئله به یکی از چالش‌های مهم در نگهداری و به‌روزرسانی سیستم‌های بازیابی اطلاعات (IR) اشاره دارد. در وب پویا، اسناد دائماً حذف یا منقضی می‌شوند (مثل لینک‌های شکسته). اگر موتور جستجو بخواهد با هر درخواست حذف، شناسه سند (DocID) را به صورت فیزیکی از روی دیسک پاک کند، فاجعه‌ای در کارایی سیستم رخ می‌دهد.

فایل‌های نمایه (Posting Lists) به صورت متوالی و فشرده روی دیسک ذخیره می‌شوند. حذف فیزیکی یک شناسه به معنای

خواندن فایل از دیسک، حذف آن عدد، شیفت دادن (**Shift**) تمام داده‌های بعدی برای پر کردن فضای خالی، و نوشتن مجدد کل فایل روی دیسک است. این عملیاتِ پرهزینه، پردازنده و پهنای باند دیسک را به‌شدت درگیر کرده و سرعت پاسخگویی سیستم را نابود می‌کند.

بخش 2) مفاهیم پایه و پیش‌نیاز

بردار بیتی ابطال (Invalidation Bit-vector): یک ساختار داده‌ی بسیار سبک و درون‌حافظه‌ای (**In-Memory**) است که وضعیت حیات هر سند را با یک بیت (0 یا 1) نشان می‌دهد.

فیلتر در زمان اجرا (Query-time Filtering): تکنیکی که در آن اسناد نامعتبر، نه در زمان نمایه‌سازی، بلکه در لحظه پردازش پرس‌وجوی کاربر از نتایج حذف می‌شوند.

جمع‌آوری زباله (Garbage Collection): فرآیند پاک‌سازی نهایی و آزادسازی فضاهای مرده از روی دیسک که به‌صورت دوره‌ای و زمان‌بندی‌شده انجام می‌گیرد.

بخش 3) حل گام‌به‌گام مسئله (راهکار جایگزین و استاندارد)

برای حل این چالش، سیستم‌های بازیابی اطلاعات مدرن (مانند **Lucene** یا **ElasticSearch**) رویکرد «حذف منطقی» را به‌جای «حذف فیزیکی» جایگزین کرده‌اند. این استاندارد شامل 3 گام اساسی است:

گام اول: علامت‌گذاری با بردار بیتی (Logical Deletion)

سیستم یک آرایه بیتی در حافظه اصلی (**RAM**) ایجاد می‌کند که طول آن برابر با حداکثر تعداد اسناد (**Max DocID**) است. به‌صورت پیش‌فرض تمام بیت‌ها روی مقدار 0 (فعال) تنظیم شده‌اند. هنگامی که دستوری مبنی بر حذف یک صفحه وب دریافت می‌شود، سیستم هیچ مراجعه‌ای به دیسک سخت نمی‌کند؛ بلکه تنها به آدرس آن سند در آرایه بیتی رفته و مقدار آن را به 1 (باطل‌شده) تغییر می‌دهد. این عملیات در زمان ثابت و فوق‌سریع $O(1)$ انجام می‌پذیرد.

گام دوم: فیلتر کردن اسناد در زمان پردازش پرس‌وجو

زمانی که کاربری عبارتی را جستجو می‌کند، سیستم طبق روال عادی، لیست‌های ارجاع (**Posting Lists**) مربوط به آن عبارت را از روی دیسک واکنشی می‌کند. این لیست‌ها همچنان حاوی شناسه‌ی اسناد حذف‌شده هستند. پیش از آنکه سیستم وارد فاز محاسبه امتیاز (**Scoring**) و رتبه‌بندی شود، تک‌تک شناسه‌های واکنشی‌شده را با بردار بیتی درون حافظه تطبیق می‌دهد. اگر بیتِ مربوط به یک **DocID** برابر با 1 باشد، سیستم بلافاصله آن سند را نادیده گرفته و از لیست نتایج فیلتر می‌کند. از آنجا که این بررسی در **RAM** انجام می‌شود، تأخیر زمانی آن تقریباً 0 است.

گام سوم: حذف فیزیکی تعویق‌یافته در فاز ادغام (Deferred Physical Deletion)

سند حذف‌شده تا چه زمانی روی دیسک باقی می‌ماند؟ سیستم این اسناد مرده را تا زمان رسیدنِ «فاز ادغام دوره‌ای» (**Periodic Merging**) روی دیسک تحمل می‌کند.

در سیستم‌های نمایه‌سازی پویا، زمانی که نمایه کمکی (**Auxiliary Index**) پر می‌شود، سیستم باید آن را با نمایه اصلی روی دیسک ادغام کند. دقیقاً در همین فاز است که سیستم فایل‌های قدیمی را می‌خواند، اسنادی که در بردار بیتی علامت 1 دارند را به‌طور کامل دور می‌ریزد (**Garbage Collection**) و تنها اسناد معتبر را در فایل نمایه یکپارچه و جدید یادداشت می‌کند. پس از این مرحله، فضای دیسک رسماً آزاد می‌شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

بهینه‌سازی منابع: این معماری یک نمونه عالی از موازنه منابع (**Trade-off**) در علوم کامپیوتر است؛ ما مقدار بسیار ناچیزی از حافظه **RAM** (برای نگهداری بردار بیتی) و کسر کوچکی از زمان پردازش (برای چک کردن بیت‌ها) را فدا می‌کنیم تا از انجام عملیات به‌شدت پرهزینه **I/O** روی دیسک جلوگیری کنیم.

به‌روزرسانی اسناد (Document Update): جالب است بدانید که عملیات «ویرایش سند» نیز در موتورهای جستجو وجود ندارد. اگر یک صفحه وب تغییر کند، سیستم ابتدا شناسه قبلی آن را در بردار بیتی باطل کرده (**Delete**) و سپس نسخه ویرایش‌شده را به‌عنوان یک سند کاملاً جدید با **DocID** جدید به نمایه کمکی اضافه می‌کند (**Insert**).

الگوی ذهنی:

این مکانیزم را مانند یک کتابخانه مرجع تصور کنید. اگر کتابی مفقود شود، کتابدار سراغ تک‌تک کارت‌های فهرست‌نویسی (که بر اساس موضوع، نویسنده و سال مرتب شده‌اند) نمی‌رود تا نام آن کتاب را با لاک‌غلط‌گیر پاک کند (حذف فیزیکی). بلکه نام کتاب را در یک «لیست سیاه» روی میز خود یادداشت می‌کند (بردار بیتی). هر زمان که مراجعه‌کننده‌ای کتابی را درخواست کرد، کتابدار ابتدا لیست سیاه را چک می‌کند (فیلتر کردن) و اگر کتاب مفقود شده بود، به او اطلاع می‌دهد. پاک‌سازی واقعی کارت‌ها تنها در پایان سال و هنگام انبارگردانی (فاز ادغام) انجام می‌شود.

پاسخنامه مسئله 4.20: نمایه‌های کنترل دسترسی (Access Control Indexes)

بخش 1) تحلیل صورت مسئله

این مسئله به بررسی یکی از تفاوت‌های بنیادین میان «جستجوی وب» (**Web Search**) و «جستجوی سازمانی» (**Enterprise Search**) می‌پردازد. در جستجوی سازمانی، برخلاف وب که اطلاعات برای همه آزاد است، سطح دسترسی به اسناد بر اساس چارت سازمانی، بخش‌ها (مانند منابع انسانی یا مالی) و رده‌بندی محرمانگی طبقه‌بندی می‌شود. هدف، درک چگونگی یکپارچه‌سازی مکانیزم‌های امنیتی و کنترل دسترسی (**ACL**) با ساختار نمایه معکوس (**Inverted Index**) است، به گونه‌ای که اسناد غیرمجاز بدون افت کارایی سیستم، به‌صورت قطعی از دید کاربر پنهان بمانند.

بخش 2) مفاهیم پایه و پیش‌نیاز

لیست کنترل دسترسی (ACL - Access Control List): فهرستی متصل به هر سند که مشخص می‌کند کدام کاربران (**Users**) یا گروه‌های کاربری (**Groups**) مجاز به خواندن آن سند هستند.

فیلتر پس از بازیابی (Late Binding): روشی ساده و ناکارآمد که در آن موتور جستجو ابتدا تمام اسناد مرتبط با جستجو را پیدا کرده و سپس در زمان نمایش، مجوزهای کاربر را با اسناد چک می‌کند. این روش به دلیل اتلاف منابع پردازشی برای اسنادی که در نهایت نمایش داده نمی‌شوند و همچنین ایجاد اختلال در صفحه‌بندی نتایج (**Pagination**)، در سیستم‌های مقیاس‌بزرگ منسوخ شده است.

فیلتر در زمان نمایه‌سازی (Early Binding): رویکرد استاندارد و بهینه که در آن، اطلاعات امنیتی دقیقاً مانند کلمات متن، درون خود نمایه معکوس ذخیره می‌شوند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: تبدیل مجوزها به واژگان نمایه (ACL Indexing)

در این معماری، موتور جستجو با شناسه‌های کاربری (**UserIDs**) و شناسه‌های گروه (**GroupIDs**) دقیقاً مشابه کلمات و

توکن‌های معمولی رفتار می‌کند. هنگام پردازش یک سند، سیستم علاوه بر استخراج کلمات متن، لیست کنترل دسترسی سند را نیز می‌خواند و آن‌ها را با یک پیشوند خاص (مثلاً **ACL_**) به‌عنوان توکن‌های مجازی به نمایه اضافه می‌کند.

مثال: فرض کنید سند شماره 58 (گزارش مالی) تنها برای «مدیرعامل» (**User1**) و «گروه حسابداری» (**Group4**) مجاز است. سیستم این کلمات مجازی را تولید کرده و شناسه سند 58 را به لیست ارجاع آن‌ها می‌افزاید:

$\langle \text{ACL_User1} \rightarrow [\dots, 58, \dots] \rangle$
 $\langle \text{ACL_Group4} \rightarrow [\dots, 58, \dots] \rangle$

گام دوم: اصلاح پنهان پرس‌وجو (Query Rewriting)

هنگامی که کاربری وارد سیستم سازمانی می‌شود، موتور جستجو به‌طور خودکار توکن‌های امنیتی مختص به آن کاربر (شامل شناسه کاربری و تمام گروه‌هایی که در آن عضو است) را استخراج می‌کند. زمانی که کاربر عبارتی مانند «گزارش مالی» را جستجو می‌کند، سیستم پیش از اجرای جستجو، عبارت را در پس‌زمینه اصلاح کرده و توکن‌های امنیتی را به‌صورت یک شرط منطقی (**Boolean**) به پرس‌وجو متصل می‌کند.

رابطه منطقی:

$\text{Query}_{\text{new}} = \text{Query}_{\text{original}} \text{ AND } (\text{ACL}_{\text{user}} \text{ OR } \text{ACL}_{\text{group1}} \text{ OR } \dots)$

مثال: اگر کارمند بخش فروش (**User7** که عضو **Group2** است) کلمه «گزارش مالی» را جستجو کند، موتور جستجو کوئری زیر را اجرا می‌کند:

"گزارش مالی" AND (ACL_User7 OR ACL_Group2)

گام سوم: اشتراک لیست‌های ارجاع در دیسک (Posting List Intersection)

اکنون موتور جستجو برای واکشی نتایج، لیست ارجاع کلمات مورد جستجو را با لیست ارجاع مجوزهای کاربر اشتراک (**Intersect**) می‌گیرد. از آنجا که سند 58 در لیست ارجاع **ACL_User7** و **ACL_Group2** وجود ندارد، خروجی این اشتراک برای سند 58 تهی (0) خواهد بود.

در نتیجه، بدون اینکه سیستم نیازی به بررسی تک‌تک اسناد در پایگاه داده داشته باشد، اسناد غیرمجاز به‌صورت خودکار و ریاضی‌وار در همان پایین‌ترین سطح نمایه (**Index Level**) فیلتر می‌شوند.

بخش 4) نتیجه‌گیری و نکات کلیدی

امنیت قطعی و کارایی بالا: با استفاده از این روش، کاربر تنها اسنادی را جستجو می‌کند که از قبل به آن‌ها دسترسی دارد. هیچ منبع پردازشی برای رتبه‌بندی (**Scoring**) اسناد ممنوعه هدر نمی‌رود و سرعت پاسخ‌گویی سیستم سازمانی با سیستم‌های وب برابری می‌کند.

چالش به‌روزرسانی (Update Anomaly): تنها نقطه ضعف این معماری زمانی است که سطح دسترسی یک سند تغییر کند (مثلاً سندی از حالت محرمانه خارج شود). در این حالت، نمایه آن سند باید به‌روزرسانی شود تا توکن‌های **ACL** جدید به آن افزوده یا از آن کسر گردند، که این امر نیازمند مکانیزم‌های نمایه‌سازی پویا (**Dynamic Indexing**) است.

این روش را مانند سیستم ورود به یک ساختمان امنیتی بزرگ تصور کنید. به جای اینکه به همه افراد اجازه دهیم وارد راهروها شوند و جلوی در هر اتاق، کارت آن‌ها را چک کنیم (**Late Binding**)، ما روی کارت هر شخص دقیقاً شماره اتاق‌هایی که حق ورود دارد را رمزنگاری می‌کنیم. هنگام ورود به راهروی اصلی، گیت امنیتی (موتور جستجو) تنها مسیرهایی را برای او باز می‌کند که روی کارتش ثبت شده باشد (**Early Binding**). در این حالت، شخص حتی متوجه وجود درها و اتاق‌های دیگر در ساختمان نمی‌شود.

پاسخنامه مسئله 4.21: نمایه‌های چندسطحی (Tiered Indexes)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک معماری ذخیره‌سازی سلسله‌مراتبی در موتورهای جستجوی مقیاس‌وب است. با توجه به حجم پتابایتی نمایه وب، امکان نگهداری کل آن در حافظه اصلی (**RAM**) از نظر فیزیکی و اقتصادی غیرممکن است. این مسئله نشان می‌دهد که چگونه با اولویت‌بندی اسناد و لایه‌بندی نمایه، می‌توان زمان پاسخ‌گویی (**Response Time**) را بهینه‌سازی کرد تا پرس‌وجوهای متداول، بدون درگیر کردن دیسک‌های کند، به سرعت پردازش شوند.

بخش 2) مفاهیم پایه و پیش‌نیاز

معیار کیفیت سند (Document Quality / PageRank): در وب، همه صفحات ارزش یکسانی ندارند. صفحاتی مانند

ویکی‌پدیا یا خبرگزاری‌های رسمی، اعتبار و ارجاعات (لینک‌های) بیشتری نسبت به وبلاگ‌های شخصی دارند.

نمایه چندسطحی (Tiered Indexing): تکنیکی که در آن به جای ساخت یک نمایه معکوس گول‌پیکر و یکپارچه، چندین

نمایه مجزا با اندازه‌های مختلف ساخته می‌شود و اسناد بر اساس درجه اهمیتشان در این سطوح توزیع می‌گردند.

خاتمه زودهنگام (Early Termination): توقف فرآیند جستجو به محض یافتن تعداد کافی از اسناد مرتبط (مثلاً **k**

نتیجه برتر)، بدون نیاز به واکشی و پردازش کل لیست‌های ارجاع.

بخش 3) حل گام‌به‌گام مسئله

گام اول: سطح‌بندی اسناد بر اساس کیفیت

سیستم‌های بازیابی اطلاعات مدرن، پیش از نمایه‌سازی، اسناد را بر اساس معیارهای کیفیت (**Static Score**) رتبه‌بندی کرده و

سپس آن‌ها را در چندین سطح (**Tier**) نمایه‌سازی می‌کنند:

سطح 1 (Tier 1): شامل کسر بسیار کوچکی از باکیفیت‌ترین، پربازدیدترین و معتبرترین اسناد وب است. حجم این

نمایه به گونه‌ای تنظیم می‌شود که به‌طور کامل درون حافظه اصلی (**RAM**) سرورها جای گیرد.

سطح 2 (Tier 2): شامل اسناد با کیفیت و اهمیت متوسط است. این نمایه معمولاً روی دیسک‌های حالت جامد

پرسرعت (**SSD**) ذخیره می‌گردد.

سطح 3 (Tier 3): نمایه بایگانی که شامل میلیاردها صفحه با کیفیت پایین، قدیمی یا کمترشناخته‌شده است و روی

دیسک‌های سخت مکانیکی (**HDD**) ارزان‌قیمت قرار می‌گیرد.

گام دوم: پردازش آبخاری پرس‌وجو (Cascading Query Processing)

هنگامی که یک پرس‌وجو (**Query**) وارد موتور جستجو می‌شود، سیستم جستجوی خود را به صورت مرحله‌ای انجام می‌دهد:

1. ابتدا پرس‌وجو منحصراً روی سطح 1 (RAM) اجرا می‌شود.

2. الگوریتم بررسی می‌کند که آیا تعداد نتایج یافت‌شده در این سطح، برای پاسخ به کاربر کافی است (مثلاً رسیدن به

$k = 10$ نتیجه معتبر برای صفحه اول جستجو).

3. اگر نتایج کافی یافت شد، سیستم فرآیند را متوقف کرده (Early Termination) و نتایج را بازمی‌گرداند.

4. تنها در صورتی که نتایج کافی در سطح 1 یافت نشود (مانند جستجوی عبارات بسیار تخصصی یا نادر)، سیستم

جستجو را در سطح 2 و در نهایت سطح 3 ادامه می‌دهد.

گام سوم: تأثیر مستقیم بر کوئری‌های پرتکرار

پرس‌وجوهای پرتکرار (Head Queries) که درصد عظیمی از ترافیک روزانه موتورهای جستجو را تشکیل می‌دهند (مانند نام شبکه‌های اجتماعی، افراد مشهور، اخبار روز و سایت‌های دانشگاهی)، همواره به دنبال معتبرترین سایت‌های وب هستند. از آنجا که این سایت‌های معتبر در سطح 1 قرار دارند، جستجو برای کوئری‌های پرتکرار همیشه در همان گام نخست به حد نصاب نتایج می‌رسد.

در نتیجه، این پرس‌وجوها به‌طور کامل در محیط فوق‌سریع RAM پاسخ داده می‌شوند و سیستم هرگز زمان خود را برای خواندن نمایه‌های عظیم سطوح پایین‌تر از روی دیسک تلف نمی‌کند.

بخش 4) نتیجه‌گیری و نکات کلیدی

موازنه سرعت در برابر جامعیت (Speed vs. Exhaustiveness): در نمایه‌های چندسطحی، موتور جستجو تضمین

نمی‌کند که تمام اسناد مرتبط موجود در وب را واکنشی کرده است (کاهش موضعی Recall)؛ زیرا ممکن است سند

مرتبطی در سطح 3 باشد که سیستم آن را نادیده گرفته است. اما در مقیاس وب، ارائه نتایج باکیفیت (Precision) در

کسری از ثانیه، بسیار مهم‌تر از یافتن تمامی اسناد است.

الگوی ذهنی:

این معماری را مانند یک اورژانس بیمارستان در نظر بگیرید. داروها و تجهیزات احیای حیاتی و پرکاربرد (سطح 1)، در قفسه‌های کنار دست پزشک (RAM) قرار دارند تا در ثانیه اول در دسترس باشند. داروهای تخصصی‌تر (سطح 2) در داروخانه طبقه پایین (SSD) هستند و تجهیزات کمتر مستهلک (سطح 3) در انبار مرکزی بیمارستان (HDD) نگهداری می‌شوند. با این چیدمان، اکثر بیماران اورژانسی با سرعت بالا و با تجهیزات همان اتاق، درمان و ترخیص می‌شوند.

پاسخنامه مسئله 4.22: انتخاب الگوریتم مناسب (Choosing the Right Algorithm)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، ارزیابی توانایی دانشجو در اتخاذ تصمیمات کلان معماری نرم‌افزار و طراحی سیستم‌های توزیع‌شده است. در این سناریو، ما با یک دوگانه کلاسیک در مهندسی داده روبه‌رو هستیم: «ارتقای عمودی سخت‌افزار با یک الگوریتم بهینه (SPIMI)» در برابر «توسعه افقی سخت‌افزارهای ارزان‌قیمت با یک چارچوب توزیع‌شده (MapReduce)». تصمیم‌گیری باید بر اساس ماهیت داده‌ها (لاگ‌های حجیم و ایستا) و نیازمندی‌های یک کسب‌وکار نوپا (استارت‌آپ) انجام شود.

مقیاس‌پذیری عمودی (Scale-Up / Vertical Scaling): افزایش قدرت پردازشی، حافظه **RAM** و سرعت دیسک یک سرور واحد. این روش محدودیت فیزیکی دارد و پس از حد مشخصی، هزینه آن به‌صورت تصاعدی بالا می‌رود.

مقیاس‌پذیری افقی (Scale-Out / Horizontal Scaling): اضافه کردن تعداد زیادی سرور ارزان‌قیمت (**Commodity Hardware**) به یک شبکه و تقسیم بار پردازشی میان آن‌ها.

داده‌های ایستا (Static Data): داده‌هایی که پس از تولید، تغییری نمی‌کنند (مانند لاگ‌های سرور). این ویژگی به ما اجازه می‌دهد از پردازش‌های دسته‌ای (**Batch Processing**) با خیالی آسوده استفاده کنیم، زیرا نیازی به نمایه‌سازی پویا و در لحظه نداریم.

بخش 3) حل گام‌به‌گام مسئله (تحلیل گزینه‌ها)

برای انتخاب درست، باید مشخصات داده‌های ورودی را با نقاط قوت و ضعف هر معماری تطبیق دهیم:

گام اول: تحلیل ماهیت لاگ‌های سرور (Server Logs)

لاگ‌های سرور دارای ویژگی‌های منحصربه‌فردی هستند:

- حجم فوق‌العاده عظیم (Massive Volume):** تولید روزانه ترابایت‌ها داده.
- تنوع لغوی پایین (Low Vocabulary Size):** کلمات یکتا در لاگ‌ها بسیار محدودند (شامل تعدادی **IP** مشخص، کدهای وضعیت **HTTP** مثل 200 یا 404، تاریخ‌ها و **URL**‌ها).
- لیست‌های ارجاع غول‌پیکر (Giant Posting Lists):** به دلیل تکرار بی‌نهایت کلمات محدود، طول لیست‌های ارجاع به‌شدت بزرگ می‌شود.

گام دوم: تحلیل گزینه الف (SPIMI روی 1 سرور قدرتمند)

الگوریتم **SPIMI** به دلیل مدیریت پویای حافظه بسیار کارآمد است. اما در مواجهه با لاگ‌های حجیم روی یک ماشین:

مزیت: پیاده‌سازی ساده‌تر و عدم نیاز به مدیریت شبکه.

گلوگاه‌کشنده (I/O Bottleneck): هرچقدر هم سرور قدرتمند و حافظه **RAM** آن بالا باشد، در نهایت باید لیست‌های ارجاع غول‌پیکر را روی یک سیستم ذخیره‌سازی (یک دیسک یا آرایه محلی) تخلیه کند. سرعت نوشتن روی یک نقطه متمرکز فیزیکی، به‌سرعت تبدیل به گلوگاه سیستم می‌شود.

نقطه شکست واحد (SPOF): اگر در میانه پردازش یک فایل لاگ چند ترابایتی، سرور دچار مشکل سخت‌افزاری شود، کل فرآیند نمایه‌سازی از بین رفته و باید از 0 آغاز گردد.

گام سوم: تحلیل گزینه ب (MapReduce روی 10 سرور ضعیف)

این گزینه دقیقاً برای غلبه بر محدودیت‌های گزینه الف طراحی شده است:

مزیت اول (توزیع I/O): مهم‌ترین برتری این روش آن است که فرآیند خواندن و نوشتن روی دیسک، میان 10 هارد درایو مجزا تقسیم می‌شود. این توزیع فیزیکی، گلوگاه مکانیکی دیسک را به‌طور کامل از بین می‌برد.

مزیت دوم (تحمل‌پذیری خطا - Fault Tolerance): اگر در طول پردازش طولانی‌مدت لاگ‌ها، یکی از سرورهای ضعیف از کار بیفتد، گره مرکزی (**Master**) به‌طور خودکار وظیفه آن را به یک سرور دیگر محول می‌کند و نیازی به توقف کل

مزیت سوم (مقیاس پذیری نامحدود): برای یک استارتاپ، هزینه خرید سرورهای معمولی بسیار کمتر از یک ابررایانه است. همچنین با رشد لاگ‌ها، افزودن سرور 11ام و 12ام بسیار منطقی‌تر و ارزان‌تر از ارتقای مجدد یک سرور قدرتمند خواهد بود.

بخش 4) نتیجه‌گیری و انتخاب نهایی

انتخاب نهایی به‌عنوان مدیر سیستم: گزینه دوم، یعنی **MapReduce** روی **10 سرور ضعیف**.

دلیل اصلی: ساخت نمایه‌های عظیم برای داده‌های ایستا، یک عملیات «محدود به I/O» است، نه محدود به پردازنده (CPU). استفاده از **SPIMI** روی یک ماشین قدرتمند، تمام فشار خواندن و نوشتن را روی یک گذرگاه سخت‌افزاری متمرکز می‌کند و در بلندمدت پاسخ‌گوی رشد نمایی لاگ‌ها نخواهد بود. چارچوب **MapReduce** با توزیع بار ورودی/خروجی و برخورداری از تحمل‌پذیری خطای ذاتی، تنها راه‌حل مهندسی‌شده و اقتصادی برای سیستم‌های کلان‌داده (**Big Data**) در یک استارتاپ در حال رشد است.

الگوی ذهنی:

فرض کنید قرار است بار یک کشتی باری عظیم (لاگ‌های سرور) را در یک انبار تخلیه کنید. گزینه اول (**SPIMI** قدرتمند)، استفاده از 1 لیفتراک فوق‌العاده بزرگ و گران‌قیمت است. لیفتراک سریع است، اما درهای ورودی انبار (دیسک سخت) محدود است و فقط می‌تواند 1 بار از در عبور کند؛ ضمن اینکه اگر خراب شود، کل کار می‌خواهد. گزینه دوم (**MapReduce**)، استخدام 10 کارگر ساده با چرخ‌دستی است که همزمان از 10 در مختلف انبار (دیسک‌های توزیع‌شده) بار را تخلیه می‌کنند. سرعت جمعی آن‌ها بیشتر است، ارزان‌تر تمام می‌شوند و اگر 1 نفر خسته شود، بقیه به کار خود ادامه می‌دهند. در دنیای کلان‌داده، همیشه سپاهی از کارگران ساده بر یک غول متمرکز پیروز می‌شوند.

پاسخنامه مسئله 4.23: تأثیر حافظه نهان دیسک (Impact of Disk Caching)

بخش 1) تحلیل صورت مسئله

این مسئله یکی از ظریف‌ترین چالش‌های مهندسی در طراحی سیستم‌های بازیابی اطلاعات را هدف قرار می‌دهد: «شکاف میان تئوری الگوریتم و واقعیت سخت‌افزار». هدف این است که نشان دهیم چگونه تکیه صرف بر تحلیل‌های جانبی (الگوریتمیک) بدون در نظر گرفتن مکانیزم‌های مدیریت حافظه در سیستم‌عامل، می‌تواند یک الگوریتم بهینه‌شده روی کاغذ (مانند فاز نهایی **BSBI**) را در عمل با یک فاجعه پردازشی و افت شدید سرعت مواجه کند.

بخش 2) مفاهیم پایه و پیش‌نیاز

پیش‌خوانی سیستم‌عامل (OS Read-ahead / Prefetching): مکانیزمی که در آن سیستم‌عامل هنگام دریافت دستور خواندن یک بخش از فایل، به‌طور خودکار بلوک‌های بعدی آن فایل را نیز از دیسک خوانده و در بافر (**Buffer Cache**) حافظه **RAM** قرار می‌دهد. این کار باعث می‌شود درخواست‌های بعدی برنامه با سرعت بسیار بالا و مستقیماً از **RAM** پاسخ داده شوند.

کوبیدگی دیسک (Disk Thrashing): وضعیتی مخرب که در آن هد مکانیکی دیسک سخت (**HDD**) پیوسته در حال جابه‌جایی میان شیارهای مختلف است (**Seek**‌های مکرر)، بدون آنکه حجم مفیدی از داده‌ها را منتقل کند. در این

حالت، زمان پاسخ‌گویی سیستم به شدت افت می‌کند.

بخش 3) حل گام‌به‌گام مسئله

گام اول: فرض ایده‌آل در ادغام چندگانه (N-way Merge)

در فاز نهایی الگوریتم **BSBI**، فرض تئوریک بر این است که ما می‌توانیم تمام **N** بلوک موقت تولیدشده را به‌طور همزمان باز کرده و با استفاده از یک صف اولویت (**Min-Heap**)، آن‌ها را در یک فایل نهایی ادغام کنیم. چون داده‌های داخل هر بلوک از پیش مرتب شده‌اند، فرض می‌کنیم خواندن از این فایل‌ها کاملاً «پیوسته» (**Sequential**) است و هزینه پرهزینه‌ی زمان جستجو (**Seek Time**) حذف شده است.

گام دوم: نقض فرض توسط محدودیت بافرهای سیستم‌عامل

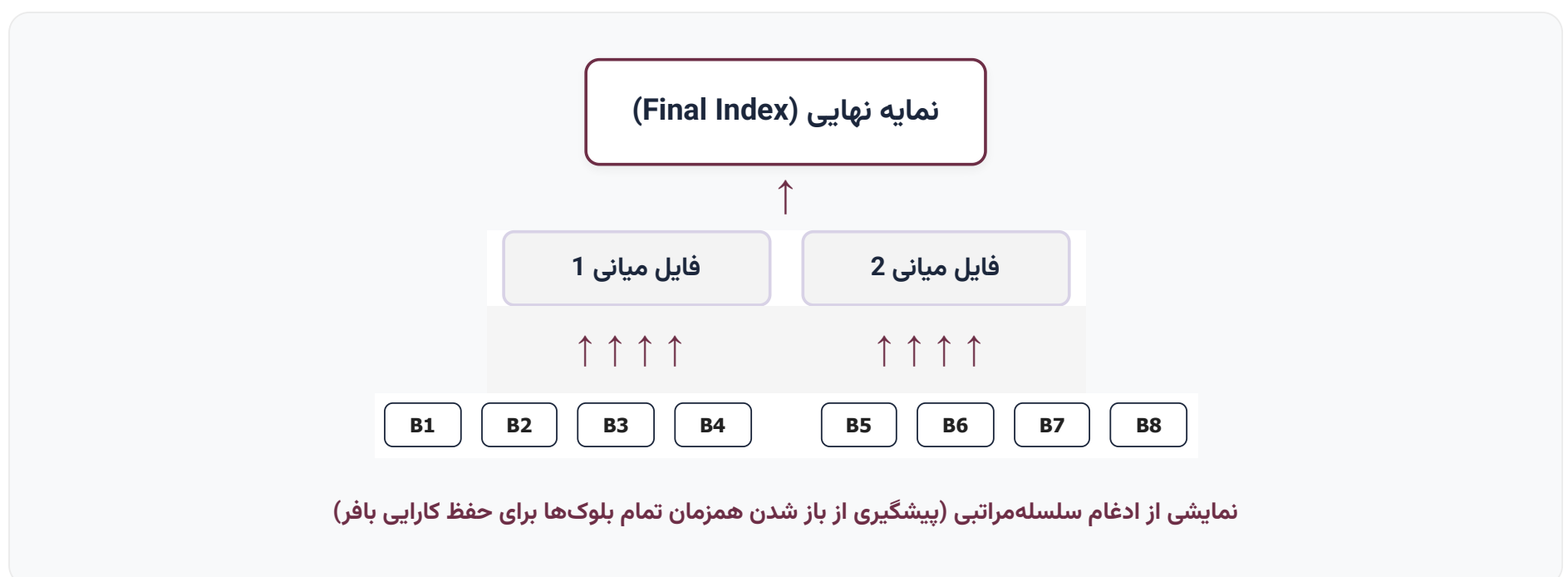
مشکل زمانی بروز می‌کند که حجم پیکره متنی بسیار بزرگ باشد و الگوریتم، هزاران فایل موقت (**N** بسیار بزرگ) روی دیسک ایجاد کرده باشد.

زمانی که برنامه از سیستم‌عامل می‌خواهد هزاران فایل را همزمان برای خواندن باز کند، سیستم‌عامل مجبور است فضای محدود بافر خود در **RAM** را میان این هزاران فایل تقسیم کند. در نتیجه، سهم هر فایل از بافر بسیار ناچیز (گاهی کمتر از یک بلوک فیزیکی دیسک) می‌شود.

به دلیل کوچک بودن بافرها، مکانیزم «پیش‌خوانی» (**Read-ahead**) کاملاً فلج می‌شود. وقتی الگوریتم یک کلمه از فایل 1 می‌خواند، سیستم‌عامل فایل را از دیسک می‌خواند؛ لحظه‌ای بعد الگوریتم کلمه‌ای از فایل 2 می‌خواهد، و هد دیسک باید فیزیکیاً جابه‌جا شود. از آنجا که الگوریتم به‌صورت نوبتی از هزاران فایل مختلف داده واکنش می‌کند و بافرها توانایی ذخیره پیشاپیش داده‌ها را ندارند، عملیات خواندن که قرار بود «پیوسته» باشد، عملاً به یک «خواندن تصادفی» (**Random Access**) خشن تبدیل می‌شود. هد دیسک دچار سردرگمی و پرش‌های مداوم شده و پدیده کوبیدگی (**Thrashing**) رخ می‌دهد.

گام سوم: راهکار برنامه‌نویس (ادغام سلسله‌مراتبی)

برای جلوگیری از این بحران، برنامه‌نویس سیستم **IR** باید از اجرای ادغام یک‌باره برای تعداد زیادی فایل خودداری کند. راهکار استاندارد، استفاده از ادغام چندمرحله‌ای یا سلسله‌مراتبی (**Hierarchical / Multi-pass Merging**) است.



1. برنامه‌نویس ابتدا یک حد مجاز و ایمن برای تعداد فایل‌های همزمان (مثلاً **M = 32** یا **M = 64**) محاسبه می‌کند،

به‌طوری که **RAM / M** بافر به اندازه کافی بزرگی برای فعال ماندن مکانیزم پیش‌خوانی سیستم‌عامل فراهم کند.

2. اگر تعداد بلوک‌ها بیشتر از **M** باشد، الگوریتم ابتدا هر **M** بلوک را با هم ادغام می‌کند تا فایل‌های میانی بزرگ‌تری ساخته شوند.

3. این فرآیند در چندین فاز (پاس) تکرار می‌شود تا در نهایت تمام فایل‌های میانی در یک نمایه نهایی و یکپارچه ادغام گردند. این روش تضمین می‌کند که در هر لحظه، تعداد فایل‌های باز روی دیسک از حد مجاز فراتر نرفته و خواندن پیوسته حفظ شود.

بخش 4) نتیجه‌گیری و نکات کلیدی

اهمیت معماری سیستم: این مسئله اثبات می‌کند که در توسعه سیستم‌های مقیاس‌بزرگ، داشتن یک الگوریتم با مرتبه زمانی $O(N \log N)$ کافی نیست؛ برنامه‌نویس باید درک عمیقی از تعامل کدهای خود با کرنل سیستم‌عامل و رفتار فیزیکی سخت‌افزار داشته باشد.

الگوی ذهنی:

فرض کنید روی یک میز کوچک (بافر **RAM**) در حال تحقیق هستید. اگر 3 کتاب (فایل) را همزمان باز کنید، به راحتی می‌توانید بدون اتلاف وقت مطالب را ترکیب کنید (ادغام بهینه و پیش‌خوانی). اما اگر استاد از شما بخواهد همزمان از 100 کتاب استفاده کنید، میز شما جای کافی برای باز نگه داشتن همه آن‌ها نخواهد داشت. شما مجبور می‌شوید مدام یک کتاب را ببندید، در قفسه (دیسک سخت) بگذارید و کتاب دیگری را بیاورید. این رفت‌وآمدهای مکرر به قفسه (**Thrashing**)، تمام وقت و انرژی شما را می‌گیرد و سرعت تحقیق را به 0 می‌رساند. راهکار این است که کتاب‌ها را دسته‌دسته (ادغام سلسله‌مراتبی) مطالعه و خلاصه‌نویسی کنید.

پاسخنامه مسئله 4.24: پیچیدگی مرتب‌سازی خارجی (External Sorting Complexity)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، اثبات ریاضی پیچیدگی عملیات ورودی/خروجی (**I/O Complexity**) در فاز ادغام الگوریتم مرتب‌سازی خارجی است. در الگوریتم‌هایی نظیر **BSBI**، زمانی که حجم داده‌ها فراتر از گنجایش حافظه اصلی (**RAM**) است، باید داده‌ها را به قطعات کوچک‌تر شکسته، مرتب کرده و سپس به صورت سلسله‌مراتبی ادغام کنیم. این اثبات نشان می‌دهد که چگونه محدودیت‌های سخت‌افزاری (ظرفیت حافظه و اندازه بافر) مستقیماً بر عمق درخت ادغام و در نتیجه بر تعداد دفعات خواندن و نوشتن روی دیسک تأثیر می‌گذارند.

بخش 2) مفاهیم پایه و متغیرهای مسئله

برای انجام این اثبات، ابتدا متغیرهای سیستم را به صورت رسمی تعریف می‌کنیم:

T (Total Data): حجم کل داده‌های خام (پیکره متنی) که باید مرتب و نمایه‌سازی شوند.

M (Memory): گنجایش حافظه اصلی (**RAM**) که در هر لحظه در اختیار برنامه است.

B (Buffer Size): اندازه هر بافر (بلوک) برای انتقال داده بین دیسک و حافظه در فاز ادغام.

پاس (Pass): یک دور خواندن کامل تمام داده‌ها از روی دیسک و نوشتن مجدد آن‌ها.

بخش 3) حل گام‌به‌گام و اثبات ریاضی

گام اول: محاسبه تعداد بلوک‌های اولیه (فاز تولید بلوک)

در مرحله اول الگوریتم **BSBI**، سیستم حافظه **M** را پر می‌کند، آن را به صورت محلی مرتب کرده و روی دیسک می‌نویسد. بنابراین، تعداد کل بلوک‌های مرتب‌شده‌ی اولیه (که به آن‌ها **Run** گفته می‌شود) از رابطه زیر به دست می‌آید:

$$N = \frac{T}{M}$$

اکنون ما **N** فایل مرتب‌شده و مجزا روی دیسک داریم که باید با یکدیگر ادغام شوند.

گام دوم: تعیین درجه ادغام (k-way Merge)

در فاز ادغام، ما نمی‌توانیم همه بلوک‌ها را یکجا در حافظه بارگذاری کنیم. سیستم باید بخش‌های کوچکی (بافر) از چند فایل را به صورت همزمان در حافظه قرار دهد. اگر اندازه هر بافر خواندن برابر با **B** باشد، حداکثر تعداد بافرهایی که می‌توانیم در حافظه **M** جای دهیم برابر است با:

$$k = \frac{M}{B}$$

(با صرف نظر از 1 بافر خروجی برای ساده‌سازی محاسبه مرتبه بزرگی).

این بدان معناست که سیستم در هر گام می‌تواند **k** فایل را باز کرده و به صورت همزمان با یکدیگر ادغام کند (که به آن ادغام **k**-گانه یا **k-way Merge** می‌گویند).

گام سوم: محاسبه ارتفاع درخت ادغام (تعداد پاس‌ها)

فرآیند ادغام مانند یک درخت عمل می‌کند. در پاس اول، ما هر **k** فایل را خوانده و به 1 فایل بزرگ‌تر تبدیل می‌کنیم. بنابراین پس از پاس اول، تعداد فایل‌ها به N/k کاهش می‌یابد.

در پاس دوم، دوباره هر **k** فایل جدید را ادغام می‌کنیم و تعداد فایل‌ها به N/k^2 می‌رسد. این روند تا جایی ادامه می‌یابد که در نهایت تنها 1 فایل یکپارچه باقی بماند. اگر تعداد پاس‌های لازم را **P** بنامیم، شرط پایان الگوریتم به شکل زیر است:

$$\frac{N}{k^P} = 1 \implies k^P = N$$

برای یافتن **P**، از دو طرف معادله لگاریتم می‌گیریم:

$$P = \log_k N$$

گام چهارم: جایگذاری متغیرها و نتیجه‌گیری نهایی

اکنون مقادیر **k** و **N** را که در گام‌های اول و دوم محاسبه کردیم، در فرمول به دست آمده جایگذاری می‌کنیم. با جایگذاری این مقادیر، تعداد پاس‌های فاز ادغام (**P**) برابر خواهد بود با:

$$P = \log_{\left(\frac{M}{B}\right)} \left(\frac{T}{M} \right)$$

بدین ترتیب ثابت می‌شود که تعداد دفعات خواندن و نوشتن کامل داده‌ها روی دیسک، متناسب با لگاریتم حجم داده‌ها نسبت به ظرفیت حافظه، در مبنای ظرفیت بافرها است.

بخش 4) نتیجه‌گیری و نکات کلیدی

تفسیر فیزیکی فرمول: پایه لگاریتم در این فرمول (M/B) نشان‌دهنده «درجه ادغام» است. هرچه حافظه **RAM (M)**

بزرگ‌تر باشد یا اندازه بافرها **(B)** بهینه‌تر انتخاب شود، پایه لگاریتم بزرگ‌تر شده و در نتیجه مقدار کل عبارت (تعداد

پاس‌ها و مراجعات به دیسک) به شدت کاهش می‌یابد.

کاربرد مهندسی: این اثبات ریاضی به مهندسان سیستم‌های بازیابی اطلاعات **(IR)** نشان می‌دهد که برای کاهش گلوگاه

دیسک در پردازش‌های عظیم، تنظیم صحیح نسبت اندازه بافر سیستم‌عامل به حافظه در دسترس، اهمیتی حیاتی دارد.

پاسخنامه مسئله 4.25: تخصیص پویای حافظه در SPIMI

(Dynamic Memory Allocation in SPIMI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، درک یکی از چالش‌های بنیادین در پیاده‌سازی سطح پایین الگوریتم **SPIMI** و مدیریت مستقیم حافظه اصلی

(RAM) است. از آنجا که در این الگوریتم، لیست‌های ارجاع **(Posting Lists)** در لحظه و به صورت پویا ساخته می‌شوند، سیستم

از پیش نمی‌داند که هر کلمه در نهایت چند بار در پیکره متنی تکرار خواهد شد. این مسئله به بررسی پدیده مخرب «قطعه‌قطعه

شدن حافظه» **(Fragmentation)** در اثر تخصیص‌های مکرر و خرد، و راه‌حل مهندسی «دو برابر کردن آرایه» می‌پردازد.

بخش 2) مفاهیم پایه و پیش‌نیاز

قطعه‌قطعه شدن حافظه (Memory Fragmentation): زمانی رخ می‌دهد که حافظه سیستم‌عامل به صورت بلوک‌های

کوچک، نامنظم و پراکنده تخصیص داده شده و سپس آزاد شود. این امر باعث ایجاد حفره‌های خالی و غیرقابل استفاده

در حافظه شده و در نهایت، با وجود خالی بودن ظرفیت کلی، سیستم در یافتن یک بلوک یکپارچه و بزرگ برای

تخصیص‌های جدید ناکام می‌ماند.

آرایه پویا (Dynamic Array): ساختار داده‌ای که برخلاف آرایه‌های سنتی، اندازه ثابتی ندارد و می‌تواند در زمان اجرا

(Run-time) ظرفیت خود را افزایش دهد.

تحلیل سرشکن (Amortized Analysis): روشی در علوم کامپیوتر برای محاسبه میانگین زمان اجرای یک عملیات در

یک توالی طولانی. در این تحلیل، هزینه‌ی بالای یک عملیات نادر، بر روی تعداد زیادی از عملیات‌های ارزان‌قیمت سرشکن

می‌شود.

بخش 3) حل گام‌به‌گام مسئله

گام اول: تشریح مشکل در تخصیص خطی (افزایش واحد)

فرض کنید برای ذخیره شناسه‌های سند (**DocID**)، هر بار که به یک کلمه تکراری می‌رسیم، دقیقاً 1 واحد به طول آرایه اختصاص یافته برای آن کلمه اضافه کنیم. فرآیند افزایش طول آرایه در حافظه شامل 3 مرحله است:

1. درخواست یک فضای جدید و یکپارچه از سیستم عامل به اندازه طول جدید.

2. کپی کردن کلمه به کلمه‌ی داده‌های قبلی به فضای جدید.

3. آزادسازی (**Free/Deallocate**) فضای قبلی.

اگر کلمه‌ای مانند «است» 1 میلیون بار در متن تکرار شود، سیستم باید 1 میلیون بار عملیات فوق را انجام دهد. این امر منجر به 2 فاجعه می‌شود:

سربار پردازشی (CPU Overhead): پردازنده دائماً در حال کپی کردن داده‌های تکراری است (با پیچیدگی زمانی $O(N^2)$).

قطعه‌قطعه شدن شدید (Fragmentation): حافظه **RAM** پر از فضاهاى رهاشده‌ی کوچک می‌شود که سیستم عامل قادر به استفاده مجدد از آن‌ها برای آرایه‌های بزرگ‌تر نیست.

گام دوم: مدیریت با مکانیزم دو برابر کردن طول آرایه (Doubling Strategy)

برای جلوگیری از این بحران، سیستم‌های **IR** از استراتژی رشد نمایی (**Exponential Growth**) استفاده می‌کنند:

1. در ابتدا، وقتی کلمه‌ای برای اولین بار دیده می‌شود، یک آرایه با ظرفیت کوچک (مثلاً $C = 2$) برای آن تخصیص می‌یابد.

2. شناسه‌ها به این آرایه اضافه می‌شوند. زمانی که آرایه پر شد (مثلاً هر 2 خانه اشغال شد)، سیستم به جای اضافه کردن 1 واحد، یک آرایه جدید با دو برابر ظرفیت قبلی ($C_{new} = 2 \times C = 4$) درخواست می‌کند.

3. داده‌های قبلی کپی شده و آرایه قبلی آزاد می‌شود.

4. در دفعات بعدی که آرایه پر شود، ظرفیت به 8، 16، 32، 64 و به همین ترتیب تا میلیون‌ها خانه افزایش می‌یابد.

گام سوم: تحلیل ریاضی و اثبات کارایی (چرا این روش کار می‌کند؟)

با استفاده از این مکانیزم، تعداد دفعاتی که سیستم باید به سراغ سیستم عامل رفته و حافظه جدید درخواست کند، از N بار (در روش خطی) به $\log_2(N)$ بار کاهش می‌یابد.

همچنین از نظر پردازشی، اگرچه گاهی اوقات کپی کردن داده‌ها زمان‌بر است، اما به دلیل فاصله زیاد بین این کپی‌ها، **هزینه سرشکن (Amortized Cost)** برای درج هر شناسه سند، در حد ثابت $O(1)$ باقی می‌ماند. این کاهش شدید در فراخوانی‌های مدیریت حافظه، مشکل **Fragmentation** را به طور کامل خنثی کرده و سرعت تولید بلوک‌های نمایه را به حداکثر می‌رساند.

بخش 4) نتیجه‌گیری و نکات کلیدی

موازنه حافظه در برابر پردازش: استراتژی «دو برابر کردن»، مقداری از حافظه را به صورت موقت هدر می‌دهد (زیرا ممکن است آرایه 64 تایی ساخته باشیم اما فقط 33 خانه آن پر شود که به آن **Internal Fragmentation** می‌گویند)؛ اما این هدررفت جزئی، هزینه لازم برای جلوگیری از نابودی سرعت پردازنده و جلوگیری از قطعه‌قطعه شدن سراسری حافظه سیستم (**External Fragmentation**) است.

کاربرد در SPIMI: دلیل اینکه الگوریتم **SPIMI** می‌تواند میلیون‌ها کلمه را با سرعتی خیره‌کننده در **RAM** پردازش کند،

استفاده دقیق از همین ساختارهای داده‌ی پویا و بهینه‌سازی شده است.

الگوی ذهنی:

این استراتژی را مانند خرید قفسه کتاب در نظر بگیرید. اگر برای هر کتاب جدیدی که می‌خرید، قفسه قبلی را دور بیندازید و یک قفسه فقط با ظرفیت 1 کتاب بیشتر بخرید، تمام وقت و پول شما صرف جابه‌جایی کتاب‌ها می‌شود. به جای این کار، شما ابتدا یک قفسه 2 تایی می‌خرید؛ وقتی پر شد، یک قفسه 4 تایی می‌خرید و کتاب‌ها را منتقل می‌کنید؛ دفعه بعد یک قفسه 8 تایی و الی آخر. با این کار، اگرچه همیشه کمی جای خالی در قفسه دارید، اما تعداد دفعاتی که مجبور به اسباب‌کشی و جابه‌جایی کتاب‌ها می‌شوید، به شدت کاهش می‌یابد.

پاسخنامه مسئله 4.26: چسبندگی و عدم توازن داده‌ها (Data Skewness in MapReduce)

بخش 1) تحلیل صورت مسئله

این مسئله به یکی از چالش‌های کلاسیک و بسیار مهم در سیستم‌های توزیع‌شده و پردازش کلان‌داده (**Big Data**) می‌پردازد. در زبان‌های طبیعی، توزیع کلمات یکنواخت نیست؛ تعداد انگشت‌شماری از کلمات بخش عظیمی از متن را تشکیل می‌دهند. هنگامی که این متون را با استفاده از **MapReduce** نمایه‌سازی می‌کنیم، مکانیزم پیش‌فرض مسیریابی داده‌ها (**Shuffle**) متوجه این تفاوت معنایی نیست. در نتیجه، گرهی که مسئول پردازش کلمات پرکاربرد می‌شود، زیر بار پردازشی و حافظه غرق شده و تبدیل به یک «گره وامانده» (**Straggler**) می‌گردد. به عنوان یک معمار سیستم، هدف ما ارائه راهکارهایی برای توزیع عادلانه بار پردازشی (**Load Balancing**) در سراسر کلاستر است.

بخش 2) مفاهیم پایه و پیش‌نیاز

قانون زیف (Zipf's Law): اصلی در زبان‌شناسی محاسباتی که بیان می‌کند فرکانس (تکرار) هر کلمه در یک پیکره متنی، با رتبه آن رابطه عکس دارد. این یعنی کلماتی مثل «از»، «به» و «است» به صورت نمایی بیشتر از کلماتی مثل «دانشگاه» یا «الگوریتم» تکرار می‌شوند.

$$f \propto \frac{1}{r}$$

تابع پارتیشن‌بندی پیش‌فرض (**Default Partitioner**): در **MapReduce**، داده‌ها معمولاً بر اساس فرمول زیر (که در آن **R** تعداد **Reducer**هاست) مسیریابی می‌شوند. این فرمول توزیع کلمات را یکنواخت می‌کند، اما توزیع حجم داده‌های پشت هر کلمه را یکنواخت نمی‌کند.

$$\text{hash}(\text{Term}) \pmod{R}$$

چسبندگی داده / انحراف داده (Data Skewness): وضعیتی که در آن بخش نامتناسب و بسیار بزرگی از داده‌ها به یک کلید (**Key**) خاص گره خورده‌اند و به یک گره پردازشی واحد سرازیر می‌شوند.

بخش 3) حل گام‌به‌گام مسئله (راهکارهای معماری سیستم)

برای حل مشکل عدم توازن، معمار سیستم می‌تواند بسته به نیازمندی‌های موتور جستجو، یکی از 3 راهکار استاندارد زیر (یا ترکیبی از آن‌ها) را پیاده‌سازی کند:

گام اول: راهکار اول - فیلترسازی پیش‌دستانه (حذف کلمات توقف / Stop Words Removal)

ساده‌ترین و رایج‌ترین معماری در سیستم‌های بازیابی اطلاعات پایه، پاک کردن صورت‌مسئله است. کلماتی که باعث **Data Skewness** می‌شوند، معمولاً همان «کلمات توقف» هستند که ارزش معنایی برای پردازش پرس‌وجو ندارند.

اجرا: در همان فاز اول (تابع **Map**)، یک لیست سیاه از کلمات پرکاربرد (**Stop-list**) به گره‌های تجزیه‌گر داده می‌شود. اگر کلمه‌ای در این لیست بود، گره **Map** اصلاً جفت **(Term, DocID)** را برای آن تولید نمی‌کند (**Emit** نمی‌کند). بدین ترتیب، ترافیک عظیمی پیش از ورود به شبکه مسدود شده و مشکل عدم توازن در گره‌های **Reduce** به‌طور کامل برطرف می‌شود.

گام دوم: راهکار دوم - استفاده از توابع ترکیب‌کننده (Combiners)

اگر بنا به دلایلی (مانند پشتیبانی از جستجوی عبارتی) نتوانیم کلمات پرکاربرد را حذف کنیم، باید بار شبکه را پیش از فاز **Reduce** کاهش دهیم. تابع **Combiner** مانند یک **"Mini-Reducer"** محلی عمل می‌کند.

اجرا: پیش از آنکه گره **Map** داده‌های کلمه «است» را در شبکه پخش کند، **Combiner** وارد عمل شده و تمام شناسه‌های سند مرتبط با «است» که در همان ماشین محلی پیدا شده‌اند را در قالب یک لیست واحد جمع می‌کند.

```
[Doc_1, Doc_5, Doc_12], "است"
```

این کار اگرچه مشکل **Reducer** نهایی را کاملاً 0 نمی‌کند، اما میلیون‌ها رکورد مجزا را به یک رکورد فشرده تبدیل کرده و بار **I/O** شبکه و گره وامانده را به‌شدت تقلیل می‌دهد.

گام سوم: راهکار سوم - پارتیشن‌بندی سفارشی و تکنیک نمک‌گذاری (Salting / Custom Partitioner)

این تکنیک، پیشرفته‌ترین راه‌حل معماری برای مدیریت کلیدهای به‌شدت چسبنده (**Highly Skewed Keys**) است. در این روش، ما منطق مسیریابی سیستم را فریب می‌دهیم تا داده‌های یک کلمه پرکاربرد را بین چند **Reducer** پخش کند.

اجرا: گره‌های **Map** برنامه‌ریزی می‌شوند تا کلمات پرکاربرد را شناسایی کنند. وقتی به کلمه «است» می‌رسند، به‌جای تولید کلید «است»، یک عدد تصادفی (مثلاً بین 1 تا 3) به انتهای آن می‌چسبانند (اصطلاحاً **Salt** اضافه می‌کنند).

خروجی‌ها به این شکل درمی‌آیند: **است_1** ، **است_2** ، **است_3** .

اکنون تابع **Hash** سیستم، این 3 کلید جدید را به‌عنوان کلمات کاملاً متفاوت می‌شناسد و آن‌ها را به 3 گره **Reduce** مختلف می‌فرستد. بار پردازشی به زیبایی متوازن می‌شود. (البته این روش نیازمند یک فاز پردازشی ثانویه برای متصل کردن مجدد این لیست‌های تکه‌تکه شده در زمان جستجو یا نمایه‌سازی نهایی است).

بخش 4) نتیجه‌گیری و نکات کلیدی

اهمیت طراحی توزیع‌شده: در پردازش کلاستری، قدرت خام سخت‌افزار همیشه تسلیم توزیع نامناسب داده‌ها می‌شود.

قانون طلایی **MapReduce** این است که سرعت کل سیستم برابر است با «سرعت کندترین گره پردازشی».

انتخاب راهکار: برای دانشجویانی که قصد طراحی سیستم واقعی را دارند؛ اگر موتور جستجو نیازی به کلمات ربط ندارد،

«حذف کلمات توقف» کم‌هزینه‌ترین انتخاب است. اما در موتورهای جستجوی پیشرفته که نیازمند نمایه‌های موقعیتی

(Positional Indexes) برای جستجوی دقیق عبارات هستند، استفاده ترکیبی از **Combiner** و الگوریتم‌های **Salting**،

معماری استاندارد و مهندسی شده به شمار می‌رود.

الگوی ذهنی:

این سناریو را مانند باجه‌های کنترل گذرنامه در یک فرودگاه بین‌المللی تصور کنید. اگر مسافران را صرفاً بر اساس "حرف اول نام خانوادگی" به باجه‌ها بفرستید (پارتیشن‌بندی پیش‌فرض)، باجه حرف «م» (مثل محمدی، مرادی و...) به شدت شلوغ می‌شود و متصدی آن تا صبح باید کار کند، در حالی که باجه حرف «ژ» کاملاً بیکار است. یک مدیر هوشمند، یا یک باجه جداگانه و ویژه برای نام‌های بسیار پرتکرار با چندین متصدی اختصاص می‌دهد (**Salting**)، یا پیش از رسیدن به باجه اصلی، فرم‌های آن‌ها را توسط مأموران محلی بررسی و دسته‌بندی اولیه می‌کند (**Combiner**).

پاسخنامه مسئله 4.27: تحلیل ریاضی ادغام لگاریتمی

(Mathematical Analysis of Logarithmic Merging)

بخش 1) تحلیل صورت مسئله

این مسئله بر روی تحلیل پیچیدگی (**Complexity Analysis**) و موازنه هزینه (**Trade-off**) در معماری نمایه‌سازی پویا تمرکز دارد. هدف این است که از طریق اثبات ریاضی نشان دهیم رویکرد «ادغام لگاریتمی» چگونه با توزیع هوشمندانه داده‌ها، هزینه فاجعه‌بار بازسازی مداوم نمایه را مهار می‌کند، اما در ازای آن، سربار اندک و کنترل‌شده‌ای را به زمان پردازش پرس‌وجو (**Query Time**) تحمیل می‌نماید.

بخش 2) مفاهیم پایه و متغیرهای سیستم

برای ارائه یک اثبات دقیق، ابتدا محیط مسئله را با متغیرهای ریاضی مدل‌سازی می‌کنیم:

T (Total Documents): تعداد کل اسنادی که تا این لحظه در سیستم نمایه‌سازی شده‌اند.

n (Memory Capacity): ظرفیت نمایه کمکی (**Auxiliary Index**) در حافظه اصلی (**RAM**) بر حسب تعداد سند.

Z_i (Index Levels): نمایه‌های روی دیسک که ظرفیت هر کدام دقیقاً توانی از 2 نسبت به n است. (ظرفیت سطح i

برابر است با $2^i \times n$).

بازسازی کامل (Complete Rebuilding): روشی سنتی که با ورود هر سند جدید (یا دسته‌ای از اسناد)، کل نمایه

موجود روی دیسک خوانده شده و نمایه جدیدی از ابتدا نوشته می‌شود.

بخش 3) حل گام‌به‌گام و اثبات ریاضی

بخش الف: اثبات پیچیدگی زمانی هزینه جستجو $O(\log(T/n))$

گام اول: تعیین تعداد سطوح روی دیسک

در روش ادغام لگاریتمی، پر یا خالی بودن نمایه‌های دیسک ($Z_0, Z_1, Z_2, \dots$) دقیقاً مشابه نمایش باینری (دودویی) عدد T/n است. برای مثال، اگر $T/n = 13$ باشد (در مبنای دو می‌شود $(1101)_2$)، نمایه‌های Z_0, Z_2, Z_3 پر و بقیه خالی هستند.

از اصول ریاضیات گسسته می‌دانیم که تعداد بیت‌های مورد نیاز برای نمایش عدد x در مبنای دو، برابر با $\lfloor \log_2(x) \rfloor + 1$ است.

بنابراین، حداکثر تعداد نمایه‌های مستقل و فعال روی دیسک برابر است با:

$$\text{Max Levels} \approx \log_2 \left(\frac{T}{n} \right)$$

گام دوم: تحلیل فرآیند جستجو

در این معماری، از آنجا که اسناد در نمایه‌های مختلف پراکنده شده‌اند و هیچ نمایه یکپارچه‌ای وجود ندارد، موتور جستجو برای پاسخ به یک کوئری موظف است آن کوئری را در تمام نمایه‌های فعال روی دیسک (و همچنین نمایه درون RAM) به صورت مجزا جستجو کرده و نتایج را ادغام کند.

گام سوم: نتیجه‌گیری پیچیدگی جستجو

با فرض اینکه زمان واکنشی لیست ارجاع (Posting List) از یک نمایه معکوس مقدار ثابتی نظیر $O(1)$ باشد (با استفاده از جداول درهم‌سازی روی دیسک یا B-Tree)، پیچیدگی زمانی پردازش پرس‌وجو مستقیماً با تعداد نمایه‌ها رابطه خطی دارد. از آنجا که در بدترین حالت (Worst-case) تمام $\log_2(T/n)$ سطح دیسک پر هستند، پیچیدگی زمانی جستجو به صورت زیر اثبات می‌شود:

$$\text{Query Time} = O \left(\log \left(\frac{T}{n} \right) \right)$$

بخش ب: اثبات پایین‌تر بودن هزینه نگهداری نمایه نسبت به بازسازی کامل

برای اثبات این بخش، باید «هزینه سرشکن» (Amortized Cost) پردازش هر سند را در هر 2 روش محاسبه و مقایسه کنیم.

گام اول: تحلیل هزینه بازسازی کامل (Complete Rebuilding)

اگر بخواهیم با ورود هر دسته n تایی از اسناد، کل نمایه را بازسازی کنیم، برای نمایه‌سازی T سند، باید T/n بار فرآیند ادغام و بازنویسی کل داده‌های قبلی را انجام دهیم.

بار 1: n سند نوشته می‌شود.

بار 2: $2n$ سند نوشته می‌شود.

بار k -ام: $k \times n$ سند نوشته می‌شود.

مجموع این عملیات‌ها یک تصاعد حسابی است که پیچیدگی زمانی آن از مرتبه $O(T^2)$ خواهد بود. در این حالت، هزینه پردازش و نمایه‌سازی برای هر 1 سند به طور میانگین برابر با $O(T)$ است که با بزرگ شدن پیکره، کاملاً غیرقابل انجام می‌شود.

گام دوم: تحلیل هزینه ادغام لگاریتمی

در ادغام لگاریتمی، یک سند زمانی روی دیسک خوانده و نوشته می‌شود که از یک سطح (Z_i) به سطح بالاتر (Z_{i+1}) منتقل گردد.

سوال این است: یک سند در طول عمر خود، حداکثر در چند فرآیند ادغام شرکت می‌کند؟

همان‌طور که در بخش الف ثابت شد، بالاترین سطح ممکن در دیسک برابر با $\log_2(T/n)$ است. بنابراین، هر سند حداکثر به تعداد سطوح دیسک (یعنی $\log_2(T/n)$ بار) در فرآیند ادغام شرکت کرده و روی دیسک جابه‌جا می‌شود.

گام سوم: نتیجه‌گیری هزینه نگهداری

از آنجا که هر سند حداکثر $\log_2(T/n)$ بار پردازش می‌شود، هزینه کل برای نمایه‌سازی T سند برابر است با $O(T \log(T/n))$.

با تقسیم این مقدار بر T ، هزینه سرشکن (Amortized) برای نمایه‌سازی هر 1 سند برابر خواهد بود با:

$$\text{Amortized Cost} = \mathcal{O}\left(\log\left(\frac{T}{n}\right)\right)$$

بخش 4) نتیجه‌گیری و نکات کلیدی

مقایسه نهایی: با مقایسه هزینه نمایه‌سازی یک سند در روش بازسازی کامل $\mathcal{O}(T)$ و روش ادغام لگاریتمی $\mathcal{O}(\log(T/n))$ ، برتری قاطع ادغام لگاریتمی اثبات می‌گردد. تابع لگاریتمی باعث می‌شود که حتی برای پیکره‌های چند میلیارد سندی، هزینه نمایه‌سازی و نگهداری بسیار ناچیز بماند.

موازنه (The Compromise): ادغام لگاریتمی یک مصالحه هوشمندانه است. ما زمان پاسخ‌گویی به کوئری را اندکی گند می‌کنیم (از جستجو در 1 فایل به جستجو در $\log$ فایل افزایش می‌یابد)، اما در عوض، سرعت نمایه‌سازی اسناد جدید را به شکل تصاعدی (از $\mathcal{O}(T)$ به $\mathcal{O}(\log T)$) بهبود می‌بخشیم.

الگوی ذهنی:

این مکانیزم دقیقاً مشابه بازی معروف 2048 است. در بازی 2048، شما بلوک‌های کوچک (مثلاً عدد 2) را می‌سازید، 2 بلوک 2 با هم ترکیب شده و یک 4 می‌سازند، 2 بلوک 4 ترکیب شده و یک 8 می‌سازند. یک بلوک برای تبدیل شدن به بلوک 2048، نیازی ندارد با تمام بلوک‌های صفحه ترکیب شود؛ بلکه فقط در $\log_2(2048) = 11$ مرحله ادغام شرکت می‌کند. ادغام لگاریتمی نیز فایل‌های روی دیسک را دقیقاً با همین منطق کم‌هزینه مدیریت می‌کند.

پاسخنامه مسئله 4.28: حجم نمایه‌های موقعیتی (Size of Positional Indexes)

بخش 1) تحلیل صورت مسئله

این مسئله به ارزیابی هزینه فضایی (**Space Complexity**) در طراحی نمایه‌های پیشرفته و تأثیر آن بر معماری ساخت نمایه می‌پردازد. هدف اول، درک دلایل ریاضی و ساختاری رشد نمایی حجم داده‌ها هنگام ثبت موقعیت کلمات است. هدف دوم، بررسی این موضوع است که آیا اضافه کردن لایه «فشرده‌سازی» (**Compression**) به الگوریتم نمایه‌سازی یک مرحله‌ای (**SPIMI**)، منطق بنیادین این الگوریتم را دستخوش تغییر می‌کند یا خیر.

بخش 2) مفاهیم پایه و پیش‌نیاز

محتوای لیست ارجاع استاندارد: در یک نمایه معکوس معمولی، برای هر کلمه در یک سند، تنها شناسه سند (**DocID**) و در برخی موارد فرکانس تکرار کلمه (**TF**) ذخیره می‌شود.

محتوای لیست ارجاع موقعیتی: علاوه بر **DocID** و **TF**، باید یک آرایه شامل دقیقاً **TF** عدد صحیح (موقعیت‌های دقیق کلمه در متن) نیز ذخیره گردد.

قانون زیف (Zipf's Law): اصلی که نشان می‌دهد در یک پیکره متنی، تعداد کمی از کلمات (مانند حروف ربط و اضافه) بخش عظیمی از حجم متن را اشغال می‌کنند و فرکانس تکرار بسیار بالایی دارند.

گام اول: چرا حجم نمایه 2 تا 4 برابر می شود؟

برای درک این افزایش حجم، باید تفاوت تعداد عناصر ذخیره شده را در 2 مدل مقایسه کنیم:

1. **در نمایه استاندارد:** اگر کلمه ای 50 بار در یک سند تکرار شود، سیستم تنها یک رکورد شامل 2 عدد صحیح ذخیره می کند:

[DocID, TF = 50]

2. **در نمایه موقعیتی:** برای همان کلمه، سیستم باید یک رکورد شامل 52 عدد صحیح ذخیره کند:

[DocID, TF = 50, [pos_1, pos_2, ..., pos_50]]

با در نظر گرفتن **قانون زیف (Zipf's Law)**، کلمات پرکاربرد هزاران بار در اسناد تکرار می شوند. در نمایه استاندارد، تکرار یک کلمه تاثیری در حجم نمایه آن سند ندارد (فقط عدد **TF** بزرگتر می شود)، اما در نمایه موقعیتی، هر 1 بار تکرار کلمه، برابر با اضافه شدن 1 عدد صحیح جدید به حافظه است. به طور میانگین، طول یک سند (تعداد کل کلمات) بین **2 تا 4 برابر بیشتر** از تعداد «کلمات یکتای» همان سند است. از آنجا که نمایه موقعیتی به ازای تک تک کلمات متن یک موقعیت ذخیره می کند، حجم نهایی لیست های ارجاع مستقیماً با این ضریب رشد کرده و 2 تا 4 برابر بزرگتر از نمایه استاندارد می شود.

گام دوم: آیا فشرده سازی، منطق SPIMI را تغییر می دهد؟

پاسخ قطعی خیر است. فشرده سازی اطلاعات موقعیتی (با تکنیک هایی نظیر **Delta Encoding** یا **Variable Byte**) هیچ تغییری در «منطق اصلی و جریان کنترل» الگوریتم **SPIMI** ایجاد نمی کند. دلیل این ثبات معماری به شرح زیر است:

حفظ ماهیت یک مرحله ای (Single-pass): الگوریتم **SPIMI** همچنان اسناد را به صورت خطی می خواند. زمانی که به یک کلمه می رسد، موقعیت فعلی آن کلمه را محاسبه کرده و به جای الحاق یک عدد صحیح خام (**Raw Integer**) به انتهای آرایه پویا، صرفاً نسخه فشرده شده ی آن (توالی بیت ها) را الحاق (**Append**) می کند.

حفظ استقلال بلوک ها: فشرده سازی در سطح لیست ارجاع انجام می شود و نیازی به دانش سراسری از کل پیکره ندارد. بنابراین، **SPIMI** همچنان واژه نامه های محلی خود را درون حافظه (**RAM**) می سازد، مقادیر را فشرده می کند و هنگام پر شدن حافظه، بلوک را مستقیماً روی دیسک سخت تخلیه می نماید.

تنها تغییری که در پیاده سازی رخ می دهد، جایگزینی نوع داده ی آرایه ها از «آرایه اعداد صحیح 32 بیتی» به «آرایه بایت های خام» (**Byte Array**) برای مدیریت مقادیر فشرده با طول متغیر است، اما الگوریتم به لحاظ مفهومی دست نخورده باقی می ماند.

بخش 4) نتیجه گیری و نکات کلیدی

چالش فضایی: نمایه های موقعیتی برای پردازش پرس و جوهای عبارتی (**Phrase Queries**) کاملاً ضروری هستند، اما بهای این قابلیت، افزایش چشمگیر مصرف حافظه **RAM** و فضای دیسک است.

انعطاف پذیری SPIMI: الگوریتم **SPIMI** به دلیل طراحی پیمانه ای (**Modular**) و عدم وابستگی به نگاشت سراسری شناسه ها (**TermID**)، بستری ایده آل برای اعمال فشرده سازی در لحظه (**On-the-fly Compression**) فراهم می کند. این ویژگی اجازه می دهد تا بلوک های بزرگتری از اسناد در همان مقدار حافظه **RAM** پردازش شوند و تعداد دسترسی ها به دیسک کاهش یابد.

الگوی ذهنی:

تفاوت نمایه استاندارد و موقعیتی مانند تفاوتِ «فهرست حاضرین» و «نقشه صندلی‌ها» در یک کلاس درس است. در فهرست حاضرین (نمایه استاندارد)، فقط یک علامت تیک مقابل نام شخص می‌زنیم که نشان می‌دهد او در کلاس حضور دارد. اما در نقشه صندلی‌ها (نمایه موقعیتی)، باید مختصات دقیق ردیف و صندلی هر شخص را ثبت کنیم. ثبت مختصات کاغذ بسیار بزرگتری می‌طلبد. حال اگر بخواهیم این مختصات را با کدهای مخفف و کوتاه بنویسیم (فشرده‌سازی)، روش و منطق ما برای قدم زدن در کلاس و سرشماری دانشجویان (الگوریتم **SPIMI**) هیچ تغییری نمی‌کند؛ فقط نوع نوشتنِ ما روی کاغذ عوض شده است.

پاسخنامه مسئله 4.29: پیاده‌سازی پایتون SPIMI (Python Implementation of SPIMI)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، تبدیل مفاهیم انتزاعی الگوریتم نمایه‌سازی در-حافظه (**SPIMI**) به یک قطعه کد عملیاتی، استاندارد و بهینه است. در این پیاده‌سازی، باید نشان داده شود که چگونه ساختارهای داده‌ی پویای زبان پایتون (مانند **Dictionary** و **List**) می‌توانند پیچیدگی‌های مدیریت حافظه را پنهان کرده و بلوک‌های نمایه را با بالاترین سرعت پردازش کنند.

بخش 2) پیاده‌سازی کد (Clean Code)

در این پیاده‌سازی از نام‌گذاری‌های معناگرا و خوانا (**Finglish**) استفاده شده است تا جریان داده‌ها برای دانشجویان کاملاً شفاف و قابل‌ردگیری باشد. (نکته: بر اساس دستورالعمل‌های نگارشی کدنویسی تمیز در پروژه‌های پایتون، تمامی مستندات و کامنت‌های داخل کد به زبان انگلیسی/لاتین بازنویسی شده‌اند تا از تداخل کاراکترها جلوگیری شود).

```
spimi_block.py

def build_spimi_block(token_stream, memory_limit):
    """
    Generates a block of the inverted index using the SPIMI algorithm.

    Inputs:
    - token_stream: A stream of tuples (kalameh, shenase_sanad)
    - memory_limit: Maximum number of tokens allowed in a memory block

    Output:
    - An alphabetically sorted dictionary containing the block's inverted index
    """
    namayeh_mahalli = {}          # Local dictionary and posting lists
    shomarandeh_token = 0         # Counter for processed tokens

    for kalameh, shenase_sanad in token_stream:

        # 1. Dynamic memory management for new words
        if kalameh not in namayeh_mahalli:
            namayeh_mahalli[kalameh] = [shenase_sanad]

        # 2. Append to existing words (with O(1) time complexity)
        else:
            # Prevent duplicate DocIDs (if a word repeats in the same doc)
            if namayeh_mahalli[kalameh][-1] != shenase_sanad:
                namayeh_mahalli[kalameh].append(shenase_sanad)

        shomarandeh_token += 1

        # 3. Check memory threshold (stop reading from input stream)
        if shomarandeh_token >= memory_limit:
            break

    # 4. Sort keys (dictionary) and structure the output
    namayeh_moratab = {k: namayeh_mahalli[k] for k in sorted(namayeh_mahalli)}

    return namayeh_moratab
```

بخش 3) تحلیل گام به گام

گام اول: استفاده از دیکشنری (Hash Map)

متغیر `namayeh_mahalli` به عنوان یک جدول درهم سازی (**Hash Table**) عمل می کند. جستجوی کلمات در این ساختار (شرط `if kalameh not in namayeh_mahalli`) با پیچیدگی زمانی ثابت $O(1)$ انجام می شود که رمز اصلی سرعت بالای الگوریتم **SPIMI** در مقایسه با روش های مبتنی بر جستجوی خطی است.

گام دوم: رشد پویای لیست های ارجاع (Dynamic Arrays)

در پایتون، ساختار `list` ذاتاً یک آرایه پویا (**Dynamic Array**) است. متد `.append()` وظیفه مدیریت حافظه را بر عهده دارد؛ به این معنا که لیست ها به صورت خودکار ظرفیت خود را 2 برابر می کنند و هزینه سرشکن (**Amortized Cost**) الحاق شناسه ها همواره $O(1)$ باقی می ماند. این ویژگی نیاز به پیاده سازی دستی منطق مدیریت حافظه را از بین می برد.

گام سوم: مدیریت کلمات تکراری در یک سند

از آنجا که جریان ورودی به ترتیب اسناد پردازش می شود، اگر یک کلمه در یک سند 2 بار تکرار شود، سیستم نباید شناسه سند را 2 بار در لیست ارجاع بنویسد. شرط `namayeh_mahalli[kalameh][-1] != shenase_sanad` تنها با بررسی آخرین عنصر لیست، این مشکل را با بالاترین سرعت و بدون نیاز به جستجو در کل لیست برطرف می کند. (توجه: در صورتی که نیاز به محاسبه فرکانس کلمه (**TF**) باشد، این بخش تغییر کرده و به جای رد کردن، فرکانس آخرین رکورد افزایش می یابد).

گام چهارم: مرتب سازی در انتهای بلوک

یکی از مزیت های **SPIMI** این است که نیازی به مرتب سازی شناسه های سند ندارد. الگوریتم تنها در مرحله پایانی (پس از پر شدن `memory_limit`)، کلیدهای دیکشنری (واژگان یکتا) را استخراج کرده و با تابع `sorted` که از الگوریتم بسیار بهینه **Timsort** (با پیچیدگی $O(V \log V)$) بهره می برد، مرتب می کند. خروجی نهایی، یک بلوک نمایه کاملاً استاندارد و آماده برای تخلیه روی دیسک (**Flushing**) است.

بخش 4 نتیجه گیری

این قطعه کد به روشنی نشان می دهد که چرا زبان های مدرنی مانند پایتون، با پنهان سازی جزئیات تخصیص فیزیکی حافظه و ارائه ساختارهای قدرتمند پویا، بهترین بستر برای پیاده سازی و درک الگوریتم های پیچیده ی بازایی اطلاعات هستند. بلوک خروجی این تابع، مستقیماً می تواند به عنوان ورودی برای فاز نهایی یعنی «ادغام بلوک ها روی دیسک» مورد استفاده قرار گیرد.

پاسخنامه مسئله 4.30: کدنویسی توابع Map و Reduce در هادوپ

(Coding Map and Reduce Functions)

بخش 1) تحلیل صورت مسئله

هدف از این مسئله، پیاده سازی عملی و نرم افزاری مفاهیم توزیع شده در بستر هادوپ (**Hadoop**) است. در این سناریو، فراتر از یک نمایه معکوس ساده (**Boolean Indexing**)، نیازمند ذخیره فرکانس کلمات (**Term Frequency**) یا **TF** هستیم تا در فازهای بعدی برای محاسبه امتیاز اسناد (مانند الگوریتم **TF-IDF**) مورد استفاده قرار گیرد.

چالش اصلی این است که محاسبه **TF** باید بهینه ترین شکل ممکن صورت گیرد. اگر هر کلمه را به صورت خام در شبکه منتشر کنیم، بار ترافیکی شبکه (**Network Traffic**) فاجعه بار خواهد بود. بنابراین، الگوریتم باید به گونه ای طراحی شود که تابع **Map**، ابتدا فراوانی کلمات را در سطح همان سند (به صورت محلی) تجمیع کرده و سپس خروجی را به تابع **Reduce** ارسال نماید.

بخش 2) مفاهیم پایه و طراحی جریان داده

در فریم‌ورک **MapReduce**، تمامی داده‌ها به شکل جفت‌های کلید-مقدار (**Key-Value Pairs**) منتقل می‌شوند. جریان داده برای این مسئله به شرح زیر مهندسی شده است:

ورودی تابع Map: جفتِ `<DocID, Document_Text>`

خروجی تابع Map / ورودی فاز Shuffle: جفتِ `<Term, (DocID, TF)>`

ورودی تابع Reduce: یک کلید (کلمه) به همراه لیستی از تاپل‌ها که توسط شبکه تجمیع شده‌اند:

`<Term, [(DocID_1, TF_1), (DocID_2, TF_2), ...]>`

خروجی تابع Reduce: کلمه به همراه لیست ارجاعِ نهایی و مرتب‌شده بر اساس شناسه سند.

بخش 3) حل گام‌به‌گام و پیاده‌سازی کد (پایتون)

کد زیر با رعایت اصول کدنویسی تمیز (**Clean Code**)، استفاده از نام‌گذاری‌های معناگرا و ساختارهای پویای زبان پایتون، دقیقاً منطبق بر معماری هادوپ توسعه یافته است. (مانند مسئله پیشین، کامنت‌ها برای جلوگیری از تداخل نوشتاری چپ‌به‌راست و راست‌به‌چپ در محیط کد، به زبان انگلیسی درج شده‌اند).

الف) پیاده‌سازی تابع Map (فاز نگاشت و تجمیع محلی)

در این تابع، به‌جای آنکه به ازای هر کلمه یک خروجی مستقل تولید کنیم، ابتدا کلمات را در یک واژه‌نامه محلی (دیکشنری) می‌شماریم. این کار نقش یک **In-Mapper Combiner** را ایفا می‌کند.

```
map_function.py

def map_function(shenase_sanad, matn_sanad):
    """
    Map Function: Receives the document text, tokenizes it,
    and calculates the Term Frequency (TF) locally within this document.
    """
    # 1. Tokenization (Splitting text into words)
    kalameh_ha = matn_sanad.split()

    # 2. Create a dictionary for local frequency calculation (In-Mapper Combiner)
    tf_mahalli = {}

    # 3. Count Term Frequency (TF)
    for kalameh in kalameh_ha:
        # Convert to lowercase for consistency
        kalameh = kalameh.lower()
        if kalameh in tf_mahalli:
            tf_mahalli[kalameh] += 1
        else:
            tf_mahalli[kalameh] = 1

    # 4. Yield key-value pairs to the network
    # Output format: (Term, (DocID, TF))
    for kalameh, tf in tf_mahalli.items():
        yield (kalameh, (shenase_sanad, tf))
```

ب) پیاده‌سازی تابع Reduce (فاز کاهش و ساخت نمایه)

شبکه هادوپ خروجی‌های تمام توابع **Map** را بر اساس «کلمه» گروه‌بندی کرده و به تابع **Reduce** تحویل می‌دهد. وظیفه این تابع، مرتب‌سازی نهایی شناسه‌های سند و ساختاردهی به لیست ارجاع (**Posting List**) است.

```
reduce_function.py

def reduce_function(kalameh, list_erjaat_kham):
    """
```

Reduce Function: Receives a term and all its references (from the cluster) and produces a standard, sorted posting list.

```
Sample input for list_erjaat_kham:
[(doc_3, tf_3), (doc_1, tf_1), (doc_2, tf_2)]
"""
```

```
# 1. Sort the posting list by DocID (ascending)
# This step is crucial for efficient intersection algorithms during search
# x[0] targets the first element of the tuple (shenase_sanad)
list_moratab = sorted(list_erjaat_kham, key=lambda x: x[0])

# 2. Structure final output and yield to disk
# Output format: (Term, [(DocID_1, TF_1), (DocID_2, TF_2), ...])
yield (kalameh, list_moratab)
```

بخش 4) نتیجه‌گیری و نکات کلیدی

کاهش گلوگاه شبکه (Network Bottleneck): مهم‌ترین دستاورد این پیاده‌سازی در بلوک `tf_mahalli` در تابع `map_function` نهفته است. اگر کلمه «است» در یک سند 100 بار تکرار شده باشد، برنامه به‌جای ارسال 100 رکورد مجزا به شبکه، تنها 1 رکورد فشرده به شکل `(است, (شناسه_سند, 100))` منتشر می‌کند. این طراحی، بار ترافیکی فاز جابه‌جایی **(Shuffle & Sort)** را به‌شدت کاهش داده و کارایی کلاستر را تضمین می‌کند.

اهمیت مرتب‌سازی در Reduce: با وجود اینکه فریم‌ورک هادوپ کلیدها (کلمات) را پیش از رسیدن به **Reduce** مرتب می‌کند، اما هیچ تضمینی روی ترتیب مقادیر **(Values)** درون لیست وجود ندارد. استفاده از تابع `sorted` در مرحله پایانی ضروری است تا لیست‌های ارجاع **(Posting Lists)** از ساختار استاندارد نمایه‌های معکوس (مرتب بودن **DocID** ها) پیروی کنند.

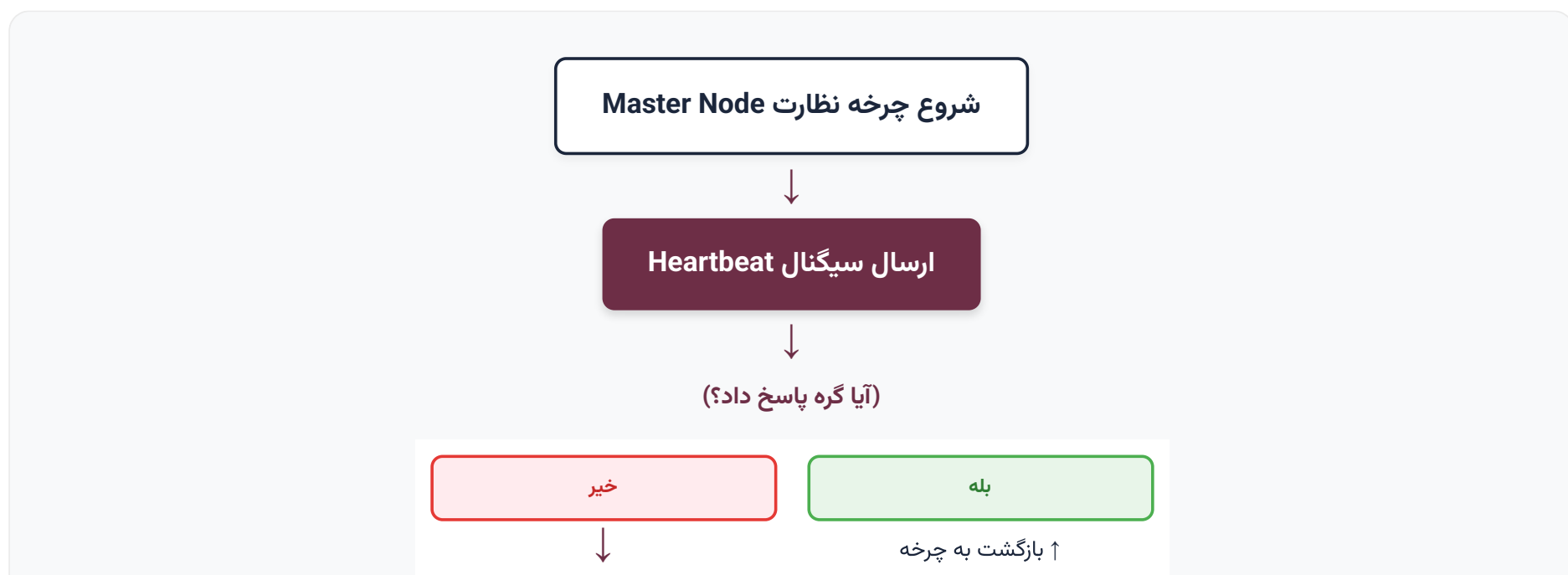
پاسخنامه مسئله 4.31: طراحی سیستم تحمل‌پذیر خطا (Fault-Tolerant System Design)

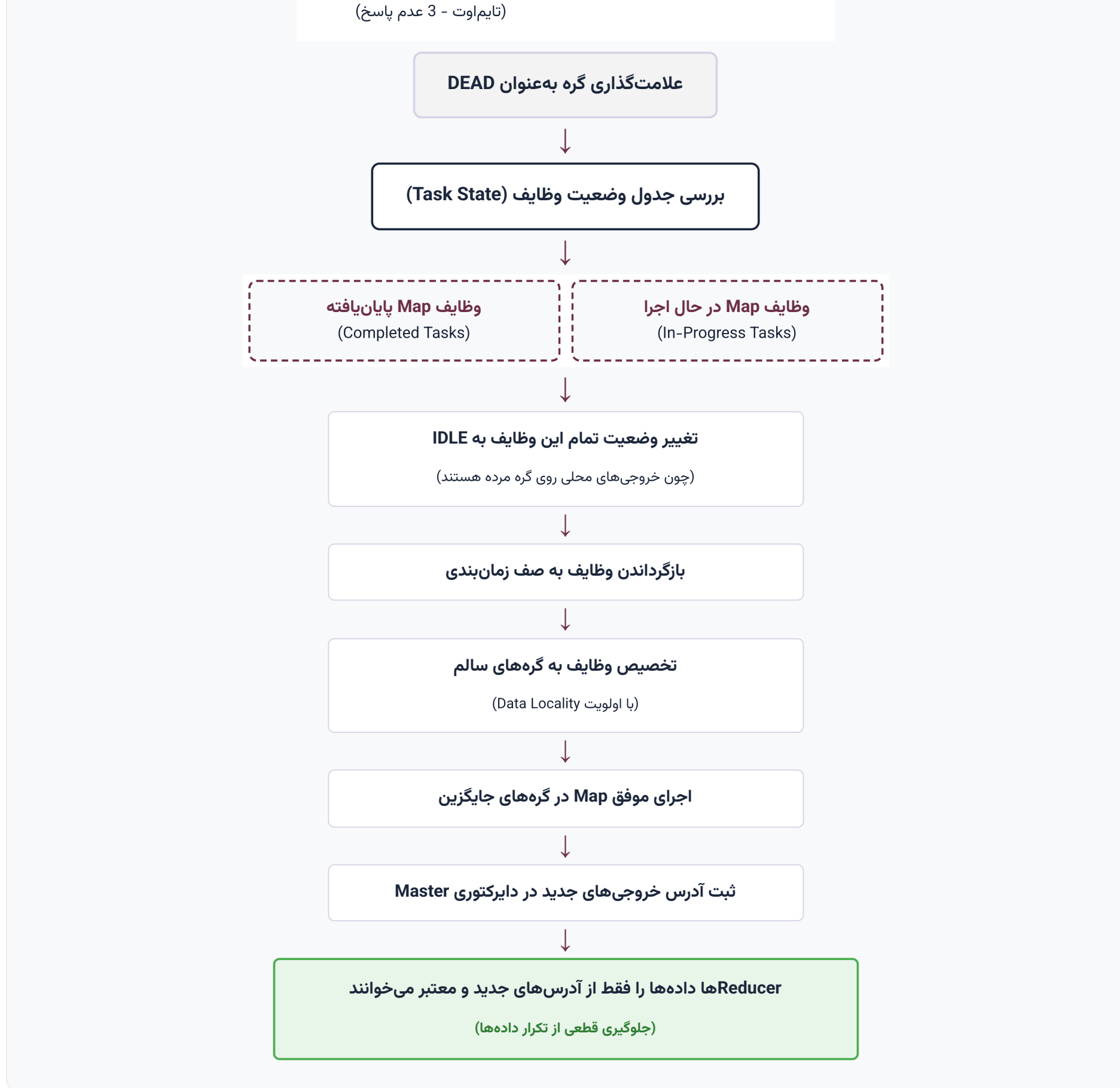
بخش 1) تحلیل صورت‌مسئله

در کلاسترهای بزرگ (مانند سیستمی با 500 گره)، خرابی سخت‌افزار یا قطعی شبکه یک استثنا نیست، بلکه یک رخداد روتین و قطعی است. مهم‌ترین نقطه قوت فریم‌ورک **MapReduce**، پنهان کردن این خرابی‌ها از چشم برنامه‌نویس و مدیریت خودکار آن‌هاست. در این مسئله، گره مرکزی **(Master Node)** باید به‌عنوان رهبر ارکستر، قطعی 5 گره را در فاز **Map** مدیریت کند، به‌گونه‌ای که هیچ داده‌ای از دست نرود و هیچ داده‌ای نیز دوبار پردازش نشود.

بخش 2) نمودار جریان عملیاتی (Operational Flowchart)

مسیر منطقی تصمیم‌گیری و مدیریت خطا در **Master Node** به شکل زیر است:





بخش 3) تشریح گام‌به‌گام عملکرد Master Node

بر اساس نمودار فوق، سیستم چگونه نیازمندی‌های ذکرشده در صورت‌مسئله را برآورده می‌کند؟

1. تشخیص خرابی (Failure Detection)

مکانیزم سیگنال حیات (Heartbeats): گره **Master** به‌صورت دوره‌ای (مثلاً هر 3 ثانیه) پیامی به نام **Heartbeat** به تمام 500 گره ارسال می‌کند.

تایم‌اوت (Timeout): اگر 5 گره مذکور، پس از یک مدت زمان مشخص (مثلاً دریافت نکردن 3 سیگنال پیاپی) به **Master** پاسخ ندهند، **Master** فرض را بر این می‌گذارد که این گره‌ها از کار افتاده‌اند یا شبکه آن‌ها قطع شده است. در این لحظه وضعیت این 5 گره در جدول مرکزی به حالت **Dead** (مرده) تغییر می‌کند.

2. تخصیص مجدد کارهای ناتمام (Task Reassignment)

بازنشانی وضعیت (State Reset): **Master** لیستی از تمام وظایفی که به این 5 گره محول شده بود را بررسی می‌کند. دو نوع وظیفه روی این گره‌ها وجود دارد:

وظایف در حال اجرا (In-Progress): قطعاً از بین رفته‌اند و باید دوباره اجرا شوند.

وظایف تکمیل شده (Completed): نکته انحرافی و مهم در معماری **MapReduce** این است که حتی وظایف **Map** پایان یافته روی گره های مرده نیز باید مجدداً اجرا شوند. چرا؟ چون خروجی فاز **Map** روی دیسک محلی (*Local Disk*) همان گره نوشته می شود و اکنون که شبکه آن گره قطع است، **Reducer** ها نمی توانند آن خروجی ها را بخوانند.

زمان بندی مجدد: Master وضعیت تمامی این وظایف را به **Idle** (بیکار) برمی گرداند و آن ها را به اولین گره های سالمی که ظرفیت خالی داشته باشند، تخصیص (**Assign**) می دهد.

3. جلوگیری از تکراری شدن داده ها (Preventing Data Duplication)

برای جلوگیری از ورود داده های تکراری به نمایه نهایی، سیستم به معماری قطعی (**Deterministic**) و ویژگی «خودتوانی» (**Idempotency**) تکیه می کند:

نادیده گرفتن اجرای قبلی (Orphaned Outputs): اگر قطعی شبکه موقتی بوده باشد و گره مرده، در انزوا کار خود را تمام کرده و خروجی تولید کند، این خروجی کاملاً نادیده گرفته می شود.

مرجعیت انحصاری Master: گره های **Reduce** برای اینکه بدانند داده ها را از کدام گره های **Map** واکنشی کنند، مستقیماً به **Master** مراجعه کرده و آدرس می گیرند.

به روز رسانی اتمیک (Atomic Update): هنگامی که گره های جایگزین، وظایف **Map** را با موفقیت به پایان می رسانند، به **Master** پیام می دهند. **Master** آدرس دیسک این گره های جدید را جایگزین آدرس های قبلی در جدول خود می کند. در نتیجه، **Reducer** ها تنها آدرس های معتبر و جدید را دریافت کرده و داده های فایل نمایه دقیقاً 1 بار (**Exactly-once Semantics**) پردازش و ذخیره می شوند.

بخش 4) نتیجه گیری و نکات کلیدی

این معماری نشان می دهد که تحمل پذیری خطا در نمایه سازی توزیع شده، نیازمند متمرکز بودن «اطلاعات وضعیت» (**State Information**) در یک نقطه (**Master**) و بی وضعیت بودن (**Stateless**) گره های پردازشی (**Workers**) است. هر کاری که یک **Worker** انجام دهد، قابل دور ریختن و تکرارپذیر است، و این دقیقاً راز پایداری سیستم های کلان داده در برابر قطعی های غیرقابل اجتناب شبکه است.

پاسخنامه مسئله 4.32: گراف انتقال در ادغام لگاریتمی پیشرفته

(Transition Graph in Advanced Logarithmic Merging)

بخش 1) تحلیل صورت مسئله

این مسئله ترکیبی از مفاهیم «بازیابی اطلاعات» و «ریاضیات گسسته» است. هدف، مدل سازی رفتار الگوریتم ادغام لگاریتمی به عنوان یک ماشین حالت متناهی (**FSM**) یا یک شمارنده باینری است. با فرض $n = 1$ (ظرفیت نمایه کمکی برابر با 1 سند)، هر بار که یک سند جدید وارد سیستم می شود، مستقیماً یک عملیات ادغام (**Merge**) و انتقال به دیسک را تحریک می کند. این تحلیل به ما کمک می کند تا دقیقاً نقاط بحرانی سیستم (مواقعی که گلوگاه **I/O** رخ می دهد) را شناسایی کنیم.

بخش 2) مفاهیم پایه و نگاشت باینری (Binary Mapping)

ابتدا باید ظرفیت هر یک از سطوح دیسک (Z_i) را بر اساس فرمول $n \times 2^i$ (با فرض $n = 1$) مشخص کنیم:

$$\text{Capacity}(Z_0) = 1 \times 2^0 = 1 \text{ Doc}$$

$$\text{Capacity}(Z_1) = 1 \times 2^1 = 2 \text{ Docs}$$

$$\text{Capacity}(Z_2) = 1 \times 2^2 = 4 \text{ Docs}$$

$$\text{Capacity}(Z_3) = 1 \times 2^3 = 8 \text{ Docs}$$

حالت (**State**) دیسک در هر لحظه را می‌توان با یک رشته باینری 4 بیتی به فرم $b_3b_2b_1b_0$ نمایش داد. هر بیت نشان‌دهنده پر بودن (1) یا خالی بودن (0) نمایه Z_i متناظر با آن است.

بخش 3) حل گام‌به‌گام: گراف انتقال حالت

جدول انتقال زیر نشان می‌دهد که با درج هر سند (از $T = 1$ تا $T = 8$)، سیستم چگونه حالت خود را تغییر می‌دهد:

مرحله (درج سند)	State [$Z_3Z_2Z_1Z_0$]	رویداد و عملیات (I/O)
حالت اولیه ($T = 0$)	[0 0 0 0]	تمام نمایه‌های روی دیسک خالی هستند.
حالت 1 ($T = 1$)	[1 0 0 0]	Z_0 پر شد.
حالت 2 ($T = 2$)	[0 1 0 0]	Z_0 و رم ادغام شده و Z_1 را پر کردند. Z_0 خالی شد.
حالت 3 ($T = 3$)	[1 1 0 0]	Z_0 پر شد.
حالت 4 ($T = 4$)	[0 0 1 0]	ادغام آبشاری: رم + Z_0 + Z_1 = پر شدن Z_2
حالت 5 ($T = 5$)	[1 0 1 0]	Z_0 پر شد.
حالت 6 ($T = 6$)	[0 1 1 0]	Z_0 و رم ادغام شده و Z_1 را پر کردند.
حالت 7 ($T = 7$)	[1 1 1 0]	Z_0 پر شد.
حالت 8 ($T = 8$)	[0 0 0 1]	ادغام آبشاری عظیم: رم + Z_0 + Z_1 + Z_2 = پر شدن Z_3

بخش 4) شناسایی گلوگاه: بیشترین عملیات I/O روی دیسک

بیشترین فشار بر روی دیسک در نقطه‌ای رخ می‌دهد که طولانی‌ترین «ادغام آبشاری» (Cascaded Merge) اتفاق بیفتد. در ریاضیات گسسته و معماری کامپیوتر، این پدیده دقیقاً معادل «انتشار رقم نقلی» (Carry Propagation) در یک جمع‌کننده باینری (Binary Adder) است.

بحرانی‌ترین انتقال: انتقال از حالت هفتم ($T = 7$) به حالت هشتم ($T = 8$).

وضعیت در $T = 7$: رشته باینری 0111 است (یعنی Z_0, Z_1, Z_2 همگی پر هستند و به ترتیب 1، 2 و 4 سند دارند).

اتفاقات لحظه درج سند 8:

- سند 8 وارد حافظه RAM (نمایه کمکی) می‌شود.
- حافظه RAM پر است، پس باید با Z_0 ادغام شود. (خواندن 1 سند از Z_0)
- نتیجه (2 سند) باید در Z_1 نوشته شود، اما Z_1 پر است. پس باید با Z_1 ادغام شود. (خواندن 2 سند از Z_1)
- نتیجه (4 سند) باید در Z_2 نوشته شود، اما Z_2 پر است. پس باید با Z_2 ادغام شود. (خواندن 4 سند از Z_2)

5. نتیجه نهایی (8 سند) در Z_3 نوشته می‌شود. (نوشتن 8 سند روی دیسک)

حجم I/O در این گام:

سیستم مجبور شد $1 + 2 + 4 = 7$ سند را از دیسک بخواند و 8 سند جدید را روی دیسک بنویسد. این در حالی است که در انتقالی مانند $T = 2 \rightarrow T = 3$ ، سیستم تنها 1 سند را بدون هیچ خواندنی، مستقیماً روی دیسک نوشت.

بخش 5) نتیجه‌گیری و نکات کلیدی

تحلیل هزینه در بدترین حالت (Worst-case Cost): این گراف به وضوح نشان می‌دهد که هزینه درج اسناد در ادغام لگاریتمی، یکنواخت نیست. اکثر درج‌ها با هزینه بسیار ناچیز $O(1)$ انجام می‌شوند، اما درج‌هایی که به توان‌های عدد 2 می‌رسند (مثل $T = 4, T = 8, T = 16$)، هزینه‌ای متناسب با $O(T)$ را به سیستم تحمیل می‌کنند.

هزینه سرشکن (Amortized Cost): با وجود این پرش‌های سنگین در نمودار I/O، چون تعداد این ادغام‌های عظیم بسیار نادر است (به صورت لگاریتمی رخ می‌دهند)، وقتی این هزینه را بین تمام اسناد تقسیم کنیم، هزینه میانگین یا سرشکن هر سند به همان $O(\log(T/n))$ محدود می‌ماند که قبلاً اثبات شد.

الگوی ذهنی:

این سیستم دقیقاً مانند کیلومترشمار مکانیکی یک خودرو کار می‌کند. وقتی از مسافت 009 به 010 می‌روید، 2 چرخ‌دنده می‌چرخند. اما وقتی از مسافت 0999 به 1000 می‌روید، تمام 4 چرخ‌دنده با هم درگیر شده و با صدای بلندی می‌چرخند (بیشترین I/O). الگوریتم ادغام لگاریتمی نیز کپی نرم‌افزاری همین چرخ‌دنده‌های مکانیکی در مدیریت فایل‌ها است.

پاسخنامه و راهنمای ارزیابی مسئله 4.33: نمایه‌ساز میان‌ستاره‌ای (The Interstellar Indexer)

بخش 1) تحلیل سناریو و اهداف آموزشی

طراحی این سناریو ترکیبی بسیار هوشمندانه از چالش‌های جریان داده (Stream Processing)، محدودیت‌های سخت‌افزاری (RAM کم در برابر فضای ذخیره‌سازی نامحدود اما گند) و ویژگی‌های پویای زبان (واژگان نامشخص) است. این مسئله به بهترین شکل درک عمیق از معماری SPIMI و ساختارهای سلسله‌مراتبی در بازیابی اطلاعات را به چالش می‌کشد. در ادامه، معماری بهینه برای پیاده‌سازی این سیستم و حل گلوگاه‌های آن ارائه شده است که می‌تواند به عنوان معیار ارزیابی جامع در نظر گرفته شود.

بخش 2) طراحی معماری سیستم (SPIMI + Auxiliary Index)

برای پاسخگویی به الزامات درنگ‌ناپذیر (Real-time) و محدودیت حافظه، معماری باید به صورت دوگانه (Dual-Stage) عمل کند:

گام اول: پردازش جریانی در نمایه کمکی (The 10-Minute Auxiliary Window)

جریان بی‌پایان تله‌متری و لاگ‌ها ابتدا وارد حافظه RAM ایستگاه زمینی می‌شود. در اینجا، نمایه کمکی (Auxiliary Index) نه تنها نقش یک حافظه موقت را ایفا می‌کند، بلکه دقیقاً به عنوان یک «نمایه معکوس در لحظه» برای جستجوهای فوری عمل می‌کند.

ویژگی متمایز SPIMI: از آنجا که کدهای خطای جدید دائماً تولید می‌شوند (حجم واژگان نامشخص)، ما نمی‌توانیم از یک دیکشنری سراسری برای تبدیل کلمات به شناسه (**TermID**) استفاده کنیم. الگوریتم **SPIMI** مستقیماً رشته‌های متنی (**String**) را به‌عنوان کلید در یک جدول درهم‌سازی (**Hash Map**) در حافظه **RAM** ذخیره می‌کند و لیست‌های ارجاع به‌صورت آرایه‌های پویا رشد می‌کنند.

پاسخ‌دهی به کوئری‌ها: هر جستجویی که مربوط به 10 دقیقه اخیر باشد، بدون هیچ‌گونه مراجعه‌ای به نوار مغناطیسی، مستقیماً و با پیچیدگی زمانی $O(1)$ از روی این نمایه کمکی در **RAM** پاسخ داده می‌شود.

گام دوم: تخلیه بلوک‌ها (Flushing)

به محض اینکه حجم لاگ‌ها از ظرفیت **RAM** فراتر رفت (یا پنجره زمانی 10 دقیقه‌ای به پایان رسید)، سیستم جریان ورودی را متوقف نمی‌کند، بلکه پردازش را روی یک بافر جدید ادامه می‌دهد. در همین حین، واژگان موجود در نمایه کمکی فعلی به ترتیب حروف الفبا مرتب شده و به‌عنوان یک **بلوک یکپارچه و مستقل (Block)** به‌صورت ترتیبی (**Sequential Write**) روی نوار مغناطیسی نوشته می‌شوند.

بخش 3) شناسایی گلوگاه اصلی: جهنم نوار مغناطیسی (Tape Thrashing)

در الگوریتم‌های مبتنی بر دیسک سخت (**HDD**)، پس از تولید بلوک‌های متعدد، از فرآیند ادغام چندگانه (**N-way Merge**) استفاده می‌کنیم. در این فرآیند، سیستم باید به‌صورت همزمان تکه‌های کوچکی از N بلوک مختلف را بخواند و ادغام کند.

گلوگاه فاجعه‌بار:

نوار مغناطیسی یک رسانه با دسترسی کاملاً ترتیبی (**Strictly Sequential Access**) است. اگر بخواهیم ادغام N -way را روی نوار انجام دهیم، هد خواندن نوار مجبور است برای خواندن یک کلمه از بلوک 1، مترها نوار را به جلو ببرد، سپس برای کلمه بعدی در بلوک 2 نوار را جابه‌جا کند و دوباره برای بلوک 3 نوار را به عقب برگرداند. این رفت‌وبرگشت‌های فیزیکی مداوم (**Tape Thrashing**)، زمان دسترسی را از چند میلی‌ثانیه به چندین دقیقه یا ساعت به ازای هر کلمه افزایش می‌دهد و کل سیستم نمایه‌سازی را فلج می‌کند.

بخش 4) راهکار بهینه‌سازی عملیاتی

برای خنثی کردن این گلوگاه در رسانه‌های ترتیبی، باید از الگوریتم ادغام لگاریتمی (**Logarithmic Merging**) به‌عنوان استراتژی جایگزین استفاده کرد.

چرا ادغام لگاریتمی؟

1. حذف دسترسی تصادفی: در ادغام لگاریتمی، سیستم هرگز به دنبال خواندن‌های پراکنده نیست. 2 بلوک با اندازه مساوی (مثلاً Z_0 و یک بلوک جدید از **RAM**) از ابتدا تا انتها کاملاً به‌صورت پیوسته خوانده شده و با هم ادغام می‌شوند تا بلوک بزرگ‌تر Z_1 را بسازند. این دقیقاً همان کاری است که نوارهای مغناطیسی برای آن ساخته شده‌اند: خواندن و نوشتن جریانی و بی‌وقفه.

2. کاهش دفعات خواندن: به‌جای اینکه هر بلوک تولید شده دائماً در فرآیندهای ادغام کوچک درگیر شود، بلوک‌ها روی نوار به‌صورت سلسله‌مراتبی انباشته می‌شوند و تنها زمانی ادغام رخ می‌دهد که بلوک‌های هم‌سطح (هم‌حجم) در سیستم وجود داشته باشند.

3. توزیع بار: با پیاده‌سازی این ساختار، هزینه سنگین I/O بر روی پردازش میلیاردها لاگ تله‌متری سرشکن

(Amortized) می‌شود و سیستم می‌تواند بدون توقف، داده‌های کاوشگر میان‌ستاره‌ای را دریافت و روی نوار بایگانی کند.

نکته برای پیاده‌سازی:

به‌عنوان یک مهندس نرم‌افزار، در این معماری باید از تخصیص پویای حافظه (Doubling Array Size) در فاز RAM غافل نشد، زیرا کدهای خطای پرتکرار کاوشگر (Data Skewness) می‌توانند به‌سرعت منجر به قطعه‌قطعه شدن (Fragmentation) حافظه محدود ایستگاه زمینی شوند.

پاسخنامه مسئله 4.34: چالش تحمل خطا در MapReduce (Fault Tolerance)

بخش 1) تحلیل صورت مسئله

این مسئله بر روی «استمرار عملیاتی» در سیستم‌های توزیع‌شده تمرکز دارد. در یک عملیات سنگین (10 میلیارد صفحه وب)، خرابی گره‌ها در میانه راه امری قطعی است. تفاوت اصلی در اینجاست که گره‌های از دست رفته در «فاز Reduce» هستند، نه «فاز Map». این تفاوت فنی بسیار مهمی است؛ چرا که خروجی فاز Map (داده‌های میانی) روی دیسک‌های گره‌های Map باقی مانده‌اند و سیستم نباید فازِ پرهزینه‌ی Map را دوباره اجرا کند.

بخش 2) پاسخ الف: تشخیص خرابی توسط Master Node

گره Master در MapReduce از مکانیزم پایش ضربان قلب (Heartbeat Monitoring) استفاده می‌کند:

1. ارسال Heartbeat: هر Worker Node در فواصل زمانی کوتاه (مثلاً هر 3 ثانیه)، یک سیگنال Keep-alive برای Master می‌فرستد.

2. تشخیص سکوت: اگر Master پاسخی از 3 Worker دریافت نکند (مثلاً پس از 10 ثانیه)، وضعیت آن‌ها را در جدول مرکزی خود از Running به Dead تغییر می‌دهد.

3. انتشار وضعیت: Master بلافاصله تمام وظایف (Task) اختصاص داده شده به آن 3 گره را به‌عنوان «ناموفق» شناسایی کرده و برای ورود به فاز بازیابی آماده می‌کند.

بخش 3) پاسخ ب: فرآیند بازیابی (Recovery Logic)

در MapReduce، برای بازیابی گره‌های Reduce، نیازی به اجرای مجدد Map نیست. داده‌های میانی (Intermediate Data) که توسط فاز Map تولید شده‌اند، در گره‌های Map ذخیره شده و «دست‌نخورده» باقی مانده‌اند.

منطق بازیابی (شبه‌کد برای Master Node)

```
reducer_failure_recovery.py

def handle_reducer_failure(failed_nodes):
    for task in get_tasks_assigned_to(failed_nodes):
        # 1. Reset task status to Idle
        task.status = "Idle"

        # 2. Select a new healthy worker node
        new_worker = select_available_node()

        # 3. Reassign the Reduce task to the new worker
        assign_task(new_worker, task)
```

```
# 4. New worker fetches intermediate data directly from Map nodes
# (because Map outputs are still intact on Map nodes' local disks)
new_worker.fetch_intermediate_data(task.map_outputs)
```

فلوچارت جریان بازیابی



بخش 4) تحلیل فنی: چرا فاز Map دوباره اجرا نمی‌شود؟

این قابلیت به دلیل طراحی «مبتنی بر دیسک» (Disk-based Intermediate Storage) است:

ذخیره‌سازی میانی: در پایان فاز **Map**، خروجی‌ها (جفت‌های کلید-مقدار) به صورت محلی در دیسک گره‌های **Map** نوشته می‌شوند. این خروجی‌ها تا پایان کل عملیات (کل پروسه **MapReduce**) توسط **Master** مدیریت شده و حذف نمی‌شوند.

دسترسی مستقیم (Shuffle Phase): گره‌های **Reduce** که مسئولیت وظایف گره‌های قطع شده را بر عهده می‌گیرند، مستقیماً به گره‌های **Map** (که هنوز سالم هستند) متصل می‌شوند و داده‌های میانی لازم را از روی دیسک آن‌ها دوباره دانلود می‌کنند.

تضمین صحت: چون خروجی فاز **Map** «قطعی» (**Deterministic**) است، مهم نیست که این داده‌ها توسط گره اولیه پردازش شده باشد یا گره جایگزین؛ نتیجه یکسان است. **Master** تنها زمانی فاز **Map** را دوباره اجرا می‌کند که گره مسئول آن داده‌های میانی (گره **Map**) قطع شده باشد و دیگر به داده‌های میانی دسترسی نداشته باشیم.

بخش 5) نتیجه‌گیری

این معماری به سیستم اجازه می‌دهد تا با کمترین هزینه، شکست‌های محلی را مدیریت کند. در این سناریو، 10 میلیارد صفحه وب همچنان در دسترس هستند و فقط بخش کوچکی از فرآیند **Reduce** (که روی 3 گره قطع شده بود) دوباره از طریق دانلود مجدد خروجی‌های میانی گره‌های **Map** سالم، به سرانجام می‌رسد. این ویژگی باعث می‌شود **MapReduce** برای پردازش‌های طولانی‌مدت (**Batch Processing**) به شدت قابل اعتماد باشد.

پاسخنامه مسئله 4.35: کدنویسی الگوریتم SPIMI (مدیریت پویای حافظه)

بخش 1) تحلیل صورت مسئله

این مسئله برای سنجش درک دقیق از نحوه تعامل الگوریتم با ساختارهای داده در حافظه اصلی (RAM) طراحی شده است. چالش اصلی این است که نشان دهیم چگونه می‌توانیم بدون تحمیل بار پردازشی اضافی (مثل فراخوانی توابع مرتب‌سازی برای لیست‌های ارجاع)، یک نمایه معکوس استاندارد و مرتب تولید کنیم. ورودی داده‌شده ساختار `(doc_id, token)` دارد که نشان‌دهنده جریان خطی خواندن متون است.

بخش 2) پیاده‌سازی با پایتون

در این پیاده‌سازی، از ساختارهای داده‌ی بومی و بهینه‌سازی‌شده‌ی پایتون (**Dictionary** و **List**) با نام‌گذاری‌های معناگرا استفاده شده است تا منطق تخصیص پویا کاملاً شفاف باشد.

```
spimi_invert.py

def sakht_bolouk_spimi(voroudi_token_ha, astaneh_hafezeh):
    """
    Simulates the first phase of the SPIMI algorithm to generate an inverted index block.

    Inputs:
    - voroudi_token_ha: Stream of tuples in the format (shenase_sanad, kalameh)
    - astaneh_hafezeh: Token processing limit before flushing the block (Threshold)

    Output:
    - Alphabetically sorted dictionary of terms (ready to be written to disk)
    """
    namayeh_mahalli = {}          # Dictionary for lexicon and posting lists
    shomarandeh_token = 0         # Counter for processed tokens in memory

    for shenase_sanad, kalameh in voroudi_token_ha:

        # 1. Dynamic Memory Management: Create a dynamic array (list) for new words
        if kalameh not in namayeh_mahalli:
            namayeh_mahalli[kalameh] = [shenase_sanad]

        # 2. Append doc_id to the list if the word already exists
        else:
            # Prevents duplicate DocIDs (when a word repeats in the same doc)
            # Keeps the posting list sorted without internal sorting
            if namayeh_mahalli[kalameh][-1] != shenase_sanad:
                namayeh_mahalli[kalameh].append(shenase_sanad)

        shomarandeh_token += 1

        # 3. Memory Threshold Check
        if shomarandeh_token >= astaneh_hafezeh:
            break

    # 4. Alphabetical sort of keys upon flushing (exit loop)
    # Complexity is O(V log V) where V is the unique vocabulary size in this block
    bolouk_moratab = {k: namayeh_mahalli[k] for k in sorted(namayeh_mahalli.keys())}

    return bolouk_moratab

# Test Case and Performance Evaluation
if __name__ == "__main__":
    # Simulating a sequential input document stream
    jaryan_dadeh = [
        (1, "bazyabi"), (1, "ettelaat"), (1, "bazyabi"),
        (2, "algorithm"), (2, "ettelaat"), (2, "pishrafteh"),
        (3, "dadeh"), (3, "algorithm")
    ]

    # Setting the threshold to 7 tokens
    khourouji_bolouk = sakht_bolouk_spimi(jaryan_dadeh, 7)

    print("SPIMI Block Output:")
    for kalameh, posting_list in khourouji_bolouk.items():
        print(f"{kalameh:15}: {posting_list}")
```

بخش 3) تحلیل پیچیدگی و نکات معماری کد

حذف مرتب‌سازی اضافی (No Internal Sorting): در الگوریتم **SPIMI**، فرض بر این است که اسناد به‌صورت ترتیبی و

صعودی خوانده می‌شوند (سند 1، سپس سند 2 و...). بنابراین وقتی `shenase_sanad` را به لیست `append`

می‌کنیم، این لیست ذاتاً صعودی و مرتب باقی می‌ماند. خط `if namayeh_mahalli[kalameh][-1] !=`

`shenase_sanad` تضمین می‌کند که لیست‌های ارجاع علاوه بر مرتب بودن، کاملاً یکتا باقی بمانند.

پیچیدگی زمانی $O(1)$: عملیات بررسی وجود یک کلید در دیکشنری پایتون، به دلیل استفاده از جداول درهم‌سازی **(Hash Tables)**، و همچنین عملیات `append` به دلیل استفاده از آرایه‌های با رشد نمایی **(Doubling Arrays)**، در تحلیل سرشکن **(Amortized Cost)** دارای پیچیدگی زمانی ثابت $O(1)$ هستند.

بهینه‌سازی نهایی: استفاده از **Dictionary Comprehension** در خط پایانی باعث می‌شود که مرتب‌سازی کلمات با پیچیدگی $O(V \log V)$ دقیقاً در پایین‌ترین سطح مفسر پایتون و با زبان **C** انجام شود که سرعت اجرا را به حداکثر ممکن می‌رساند. این خروجی ساختار استاندارد و ایده‌آلی برای تعریف تست‌کیس‌ها و پیاده‌سازی سامانه‌های ارزیابی خودکار **(Judge Systems)** دارد.

پاسخنامه مسئله 4.36: تحلیل پیچیدگی ادغام لگاریتمی و چالش نمایه‌های موقعیتی

بخش 1) تحلیل صورت مسئله و اهداف آموزشی

این مسئله به‌عنوان یک چالش سطح بالا **(Advanced IR Architecture)** برای سنجش قدرت تحلیل مجانبی و درک عمیق از رفتار سیستم‌های ذخیره‌سازی طراحی شده است. بخش اول مسئله نیازمند اثبات دقیق ریاضی برای منطق مدیریت فایل‌هاست و بخش دوم، تفکر انتقادی دانشجو را در مواجهه با تغییر نیازمندی‌های سیستم (اضافه شدن مختصات کلمات) و موازنه هزینه-فایده **(Cost-Benefit Analysis)** به چالش می‌کشد.

بخش 2) بخش الف: اثبات پیچیدگی زمانی هزینه نگهداری (Maintenance Cost)

برای اثبات پیچیدگی $O(T \log(T/n))$ ، باید مسیر حرکت یک توکن **(Token)** را از لحظه ورود به حافظه تا نشستن در آخرین سطح دیسک ردیابی کنیم.

گام اول: تعریف ساختار سطوح

در ادغام لگاریتمی، ما یک نمایه کمکی در **RAM** با ظرفیت n توکن داریم. روی دیسک، نمایه‌های $Z_0, Z_1, Z_2, \dots$ را داریم که ظرفیت هر کدام دقیقاً توانی از 2 نسبت به n است:

$$\text{Capacity}(Z_i) = n \times 2^i$$

گام دوم: محاسبه حداکثر عمق ادغام

وقتی مجموعاً T توکن پردازش شده باشند، بالاترین سطح دیسک که پر می‌شود (بزرگ‌ترین Z_i) چقدر است؟ تعداد کل توکن‌ها برابر است با مجموع ظرفیت بلوک‌های پر شده. از آنجا که ساختار پر شدن این سطوح از منطق اعداد باینری پیروی می‌کند، بالاترین سطح فعال از رابطه لگاریتمی زیر به دست می‌آید:

$$\text{Max Levels} \approx \log_2 \left(\frac{T}{n} \right)$$

گام سوم: تحلیل سرشکن (Amortized Analysis) برای هر توکن

یک توکن مشخص در طول عمر خود در سیستم، چند بار خوانده و روی دیسک نوشته می‌شود؟ هر توکن ابتدا وارد Z_0 می‌شود. سپس ممکن است در فرآیند ادغام، به Z_1 ، سپس Z_2 و در نهایت به Z_k منتقل شود. از آنجا که یک توکن هرگز به سطح پایین‌تر برنمی‌گردد، در بدترین حالت دقیقاً به تعداد سطوح دیسک، یعنی $\log_2(T/n)$ بار، در عملیات خواندن/نوشتن (**I/O**) شرکت می‌کند.

گام چهارم: نتیجه‌گیری ریاضی

اگر هر یک از T توکن پردازش‌شده، حداکثر $\log_2(T/n)$ بار روی دیسک جابه‌جا شوند، هزینه کل عملیات برای تمام توکن‌ها برابر خواهد بود با:

$$\text{Total Cost} = T \times \log_2 \left(\frac{T}{n} \right) \implies \mathcal{O} \left(T \log \left(\frac{T}{n} \right) \right)$$

این اثبات نشان می‌دهد که برخلاف روش «بازسازی کامل نمایه» که پیچیدگی $\mathcal{O}(T^2)$ دارد، ادغام لگاریتمی هزینه نگهداری را به شدت مهار می‌کند.

بخش 3) بخش ب: تأثیر نمایه‌های موقعیتی بر نرخ رشد Z_i

هنگامی که سیستم را برای پشتیبانی از پرس‌وجوهای عبارتی (**Phrase Queries**) ارتقا می‌دهیم، ساختار لیست ارجاع از حالت استاندارد به حالت موقعیتی تغییر می‌کند:

$$\langle \text{DocID}, \text{TF} \rangle \rightarrow \langle \text{DocID}, \text{TF}, [\text{pos}_1, \text{pos}_2, \dots, \text{pos}_{\text{TF}}] \rangle$$

تحلیل تغییرات نرخ رشد:

در نمایه استاندارد، اگر یک کلمه 100 بار در یک سند تکرار شود، تنها 1 عدد (**TF**) آپدیت می‌شود و حجم نمایه ثابت می‌ماند. اما در نمایه موقعیتی، سیستم موظف است 100 عدد صحیح جدید (**Position**) را ذخیره کند.

این تغییر ماهیت باعث می‌شود که نرخ رشد حجم فایل‌های ادغامی در سطوح Z_i ، کاملاً خطی و متناسب با حجم متن خام (**Raw Text Size**) شود. به لحاظ تجربی، حجم هر بلوک Z_i بین 2 تا 4 برابر بزرگ‌تر از حالت استاندارد خواهد شد. در نتیجه، محدودیت ظرفیت بافرهای حافظه (**Buffer Size**) در زمان اجرای ادغام‌های **N-way** بسیار سریع‌تر پر شده و گلوگاه **I/O** دیسک با شدت بیشتری نمایان می‌گردد.

بخش 4) بخش ج: آیا ادغام لگاریتمی همچنان توجیه اقتصادی دارد؟

پاسخ قطعی: بله، حتی بیشتر از قبل!

استدلال تحلیلی برای دانشجویان:

بسیاری از افراد به اشتباه تصور می‌کنند که با سنگین شدن حجم نمایه‌ها، باید به دنبال روشی غیر از ادغام لگاریتمی بود. اما تفکر مهندسی عکس این را ثابت می‌کند:

1. **فاجعه بازسازی کامل:** اگر حجم نمایه به دلیل **Position** ها 4 برابر شود، استفاده از روش سنتی «بازسازی کامل»

(**Rebuilding**) به معنای خواندن و نوشتن 4 برابری کل دیسک به ازای هر آپدیت است که سیستم را در عرض چند

دقیقه فلج (**Thrash**) می‌کند.

2. **مهار ضریب ثابت:** ادغام لگاریتمی پیچیدگی مجانبی را روی همان $O(T \log(T/n))$ نگه می‌دارد. اگرچه ضریب

پنهان این O (یعنی اندازه بایت‌های منتقل شده) بزرگ‌تر شده است، اما ساختار درختی و لگاریتمی الگوریتم تنها سدی

است که مانع از فروپاشی سیستم زیر بار نمایه‌های موقعیتی می‌شود.

راهکار مهندسی:

ادغام لگاریتمی برای نمایه‌های موقعیتی کاملاً توجیه دارد، به شرطی که با تکنیک‌های فشرده‌سازی (**Compression**) ترکیب شود.

دانشجوی متفکر باید اشاره کند که ذخیره فواصل موقعیت‌ها (**Delta Encoding**) و استفاده از کدهای بایت متغیر

(**Variable Byte Encoding**) در زمان انتقال داده‌ها بین Z_i و Z_{i+1} ، اثر ترمی نمایه‌های موقعیتی را خنثی کرده و ادغام

لگاریتمی را به یک معماری بی‌نقص برای موتورهای جستجوی مدرن تبدیل می‌کند.

پاسخنامه مسئله 4.37: شبیه‌سازی دقیق بلوک‌های حافظه در SPIMI

طراحی این سوال برای ارزیابی میزان دقت و درک عمیق دانشجویان از ساختار حافظه بسیار هوشمندانه است؛ زیرا یک نکته

انحرافی (**Trick**) ظریف در آن گنجانده شده است که دانشجویان بی‌دقت را که تفاوت میان تخصیص حافظه برای «کلمات یکتا» و

«جریان ورودی» را نمی‌دانند، به دام می‌اندازد. در ادامه، پاسخ دقیق و گام‌به‌گام این مسئله ارائه شده است که می‌توانید از آن

به‌عنوان کلید تصحیح استفاده کنید.

بخش الف) ردیابی مصرف حافظه و زمان پر شدن آن

قاعده اصلی در الگوریتم **SPIMI** این است که رشته‌های متنی (کلمات) در دیکشنری، تنها 1 بار به‌عنوان کلید (**Key**) در حافظه

ثبت می‌شوند. بنابراین، در مراجعات بعدی به یک کلمه تکراری، هیچ حافظه جدیدی برای خود کلمه مصرف نمی‌شود و تنها 4

بایت برای اضافه شدن **DocID** به لیست ارجاع (**Posting List**) تخصیص می‌یابد.

بر اساس این قانون، جدول ردیابی حافظه به شکل زیر خواهد بود:

زمان	توکن ورودی	وضعیت کلمه در دیکشنری	حافظه مصرفی کلمه	حافظه مصرفی شناسه	جمع این گام	کل حافظه مصرفی
t1	(web, 10)	جدید	3 بایت	4 بایت	7 بایت	7 بایت
t2	(data, 10)	جدید	4 بایت	4 بایت	8 بایت	15 بایت
t3	(web, 12)	تکراری (موجود)	0 بایت	4 بایت	4 بایت	19 بایت
t4	(search, 15)	جدید	6 بایت	4 بایت	10 بایت	29 بایت
t5	(data, 15)	تکراری (موجود)	0 بایت	4 بایت	4 بایت	33 بایت
t6	(graph, 18)	جدید	5 بایت	4 بایت	9 بایت	42 بایت
t7	(web, 18)	تکراری (موجود)	0 بایت	4 بایت	4 بایت	46 بایت

نتیجه‌گیری بخش الف (نکته انحرافی سوال):

با ردیابی دقیق متوجه می‌شویم که در زمان t7، کل مصرف حافظه به 46 بایت می‌رسد. از آنجا که محدودیت تعیین‌شده 60 بایت

است، حافظه هرگز در طول این 7 گام سرریز نمی‌کند!

بنابراین، سیستم هیچ‌گاه به‌دلیل پر شدن حافظه (Memory Full) مجبور به عملیات Write نمی‌شود. عملیات تخلیه روی دیسک (تولید بلوک اول)، صرفاً به‌این‌دلیل در زمان t7 رخ می‌دهد که جریان ورودی توکن‌ها به پایان رسیده است (End of Stream).

بخش ب) محتوای دقیق بلوک اول روی دیسک

زمانی که الگوریتم **SPIMI** تصمیم به نوشتن داده‌ها روی دیسک می‌گیرد، لیست‌های ارجاع نیازی به مرتب‌سازی ندارند (زیرا اسناد به‌صورت صعودی خوانده شده و ذاتاً مرتب هستند). اما کلیدهای دیکشنری (کلمات) باید به‌ترتیب حروف الفبا (Lexicographical Order) مرتب شوند تا در فاز نهایی (ادغام بلوک‌ها) بتوان از آن‌ها استفاده کرد.

بنابراین، ساختار نهایی و مرتب‌شده‌ی بلوکی که روی دیسک نوشته می‌شود به فرمت زیر است:

```
data    -> [10, 15]
graph   -> [18]
search  -> [15]
web     -> [10, 12, 18]
```

پاسخنامه مسئله 4.38: عدم توازن بار و کشف تقلب در نمایه توزیع‌شده

بخش 1) محاسبه توزیع توکن‌ها و مجموع بار پردازشی

ابتدا طول هر توکن را محاسبه کرده و خروجی تابع هش زیر را برای هر کدام به دست می‌آوریم:

$$H(\text{Token}) = \text{Length}(\text{Token}) \pmod{3}$$

برای سادگی، فرض می‌کنیم خروجی‌های 0، 1 و 2 به‌ترتیب به گره‌های کاهنده R_0 ، R_1 و R_2 ارسال می‌شوند.

نام متغیر (Token)	طول کلمه	محاسبه تابع هش	مقصد (Reducer)	بار پردازشی (فرکانس)
i	1	$1 \pmod{3} = 1$	R_1	85,000
count	5	$5 \pmod{3} = 2$	R_2	12,000
temp	4	$4 \pmod{3} = 1$	R_1	11,500
data_array	10	$10 \pmod{3} = 1$	R_1	300
flag	4	$4 \pmod{3} = 1$	R_1	45,000
result	6	$6 \pmod{3} = 0$	R_0	8,200

مجموع بار پردازشی در هر Reducer:

گره R_0 : فقط متغیر 8,200 ← result رکورد

گره R_1 : متغیرهای `i, temp, data_array, flag` ← $85000 + 11500 + 300 + 45000 = 141,800$

رکورد

گره R_2 : فقط متغیر `count` ← 12,000 رکورد

بخش 2) اثبات پدیده گلوگاه (Bottleneck Analysis)

مجموع کل رخدادهای پردازش شده در این سیستم 162,000 رکورد است. با یک محاسبه ساده متوجه می‌شویم که:

گره R_1 به‌تنهایی حدود 87.5% از کل داده‌های کلاستر را پردازش می‌کند!

گره R_0 تنها 5% و گره R_2 حدود 7.5% از بار را بر عهده دارند.

اثبات گلوگاه: در معماری **MapReduce**، زمان پایان کل یک عملیات برابر است با «زمانِ پایانِ کندترین گره». در این سناریو، گره‌های R_0 و R_2 کار خود را در کسری از ثانیه تمام کرده و به حالت بیکار (**Idle**) درمی‌آیند. اما سیستم همچنان باید منتظر بماند تا گره R_1 پردازش بیش از 141 هزار رکورد را به اتمام برساند.

این پدیده که به آن **چسبندگی داده (Data Skewness)** می‌گویند، باعث به‌وجود آمدن یک **گره وامانده (Straggler Node)** شده و فلسفه اصلی پردازش توزیع‌شده (تقسیم عادلانه کار برای افزایش سرعت) را به‌طور کامل نقض می‌کند. دلیل این فاجعه، استفاده از ویژگی به‌شدت متمرکز «طول کلمه» برای هشینگ است (در برنامه‌نویسی، متغیرها معمولاً کوتاه و بین 1 تا 5 کاراکتر هستند).

بخش 3) راهکارهای معماری برای حل عدم توازن بار (Load Balancing)

در نگاه اول، ممکن است به‌نظر برسد که تغییر تابع هش به یک تابع استاندارد مانند $H = \text{MD5}(\text{Token}) \pmod 3$ مشکل را حل می‌کند. اما **خیر!**

حتی با یک تابع هش بی‌نقص، تمام 85,000 رکورد متغیر `i` باز هم به 1 **Reducer** خاص ارسال می‌شوند و آن گره به‌تنهایی زیر بار `i` خرد خواهد شد. برای حل ریشه‌ای این مشکل، باید از مکانیزم‌های زیر استفاده کرد:

1. پارتیشن‌بندی سفارشی با تکنیک نمک‌گذاری (Salting) - راهکار اصلی:

ما باید رکوردهای کلیدهای بسیار سنگین (مثل `i` و `flag`) را فریب دهیم تا در شبکه پخش شوند.

منطق کار: گره **Mapper** لیستی از متغیرهای پرتکرار پایتون (مثل `i, j, flag, temp`) در اختیار دارد. وقتی به

متغیر `i` می‌رسد، به‌جای ارسال کلید `"i"`، یک عدد تصادفی بین 0 تا 2 (به‌عنوان **Salt**) به آن می‌چسباند.

خروجی Mapper: کلیدها به شکل `"i_0"`، `"i_1"`، `"i_2"` تولید می‌شوند.

نتیجه: شبکه هادوپ این 3 کلید را به‌عنوان کلماتی کاملاً مجزا می‌شناسد و آن‌ها را بین هر 3 **Reducer** تقسیم می‌کند.

بار 85 هزارتایی `i` به 3 بخش تقریباً 28 هزارتایی شکسته شده و توازن بی‌نقصی در کلاستر ایجاد می‌شود.

2. استفاده از ترکیب‌کننده درون‌حافظه‌ای (In-Mapper Combiner) - راهکار مکمل:

پیش از آنکه **Mapper** داده‌ها را وارد شبکه کند، می‌تواند در همان حافظه محلی خود متغیرها را تجمیع کند.

منطق کار: به‌جای اینکه **Mapper** شماره 1، 20,000 بار جفت `("i", 1)` را در شبکه منتشر کند، آن‌ها را در یک

دیکشنری محلی جمع کرده و تنها 1 رکورد به شکل `("i", 20000)` به **Reducer** ارسال می‌کند.

نتیجه: این کار اگرچه مشکل نهایی **Reducer** در پردازش تجمیع کلان را 0 نمی‌کند، اما بار شبکه (**Network Shuffle**) را به‌شدت کاهش داده و مانع از خفگی پهنای باند در زمان بررسی کدهای 100 دانشجو می‌شود. ترکیب **Combiner** و **Salting**، یک معماری **Enterprise** و بدون نقص را برای سیستم کشف تقلب شما تضمین می‌کند.

پاسخنامه مسئله 4.39: پیاده‌سازی ادغام چندگانه (K-way Merge) با Min-Heap

این مسئله یکی از زیباترین چالش‌های مهندسی نرم‌افزار در ساخت موتورهای جستجو است. پیاده‌سازی زیر دقیقاً برای ارزیابی کدها در پلتفرم‌های داوری آنلاین مانند کوئرا (**Quera**) طراحی شده است؛ جایی که محدودیت‌های شدید حافظه (**Memory Limits**) برای یک کلاس 100 نفره از دانشجویان اعمال می‌شود و بارگذاری کل فایل‌ها در **RAM** منجر به خطای **MemoryError** خواهد شد.

بخش 1) تحلیل معماری کد و حافظه

برای ادغام **K** بلوک مرتب‌شده بدون مصرف حافظه اضافی، از ساختار داده‌ی **پشته کمینه (Min-Heap)** استفاده می‌کنیم. در هر لحظه، تنها **1 خط** از هر فایل درون **RAM** قرار دارد. بنابراین، اگر حجم کل داده‌ها 100 گیگابایت و تعداد بلوک‌ها **K = 50** باشد، این کد تنها به اندازه 50 متغیر ساده حافظه مصرف می‌کند (پیچیدگی فضایی $O(K)$) و گلوگاه حافظه به‌طور کامل رفع می‌گردد.

بخش 2) پیاده‌سازی پایتون

در این کد از کتابخانه داخلی **heapq** برای مدیریت صف اولویت و از **contextlib.ExitStack** برای مدیریت امن و بهینه فایل‌های باز (بدون نشت حافظه) استفاده شده است:

```
merge_k_blocks.py

import heapq
from contextlib import ExitStack

def merge_k_blocks(block_ha):
    """
    Merges K sorted blocks from disk using a Min-Heap.
    Reads files line-by-line to avoid filling RAM.

    Input:
    - block_ha: List of text file paths (temporary blocks)
    """

    # Using ExitStack for smart management of multiple open files
    with ExitStack() as stack:

        # 1. Open all input files in read mode
        file_ha_voroudi = [
            stack.enter_context(open(masir_file, 'r', encoding='utf-8'))
            for masir_file in block_ha
        ]

        # 2. Create the single output file
        khorouji_file = stack.enter_context(open('final_index.txt', 'w', encoding='utf-8'))

        kopeh_min = [] # Our Min-Heap structure

        # 3. Initialization Phase
        # Read the first line of each file and push to the heap
        for shomareh_file, file in enumerate(file_ha_voroudi):
            khat = file.readline()
            if khat:
                # Convert "TermID:DocID" to integers for exact numerical sorting
                shenase_vazheh, shenase_sanad = map(int, khat.strip().split(':'))

                # Push to heap: (Primary key, Secondary key, Source file index)
                heapq.heappush(kopeh_min, (shenase_vazheh, shenase_sanad, shomareh_file))

        # 4. Continuous Merging Phase
        while kopeh_min:
```

```
# Extract the smallest (TermID, DocID) pair across all files
shenase_vazheh, shenase_sanad, shomareh_file = heapq.heappop(kopeh_min)

# Write directly to disk (without storing in RAM)
khorouji_file.write(f"{shenase_vazheh}:{shenase_sanad}\n")

# Fetch the next element from the exact same file
khat_badi = file_ha_voroudi[shomareh_file].readline()

if khat_badi:
    badi_vazheh, badi_sanad = map(int, khat_badi.strip().split(':'))
    # Push the new element into the heap to continue competing
    heapq.heappush(kopeh_min, (badi_vazheh, badi_sanad, shomareh_file))

# System execution and test
if __name__ == "__main__":
    # List of sorted blocks previously saved to disk
    list_blok_ha = ['block_0.txt', 'block_1.txt', 'block_2.txt']

    # Call the merge function
    merge_k_blocks(list_blok_ha)
    print("Merge completed successfully and saved to final_index.txt.")
```

بخش 3) نکات مهندسیِ نهفته در این پیاده‌سازی

مرتب‌سازی دوگانه (Dual-key Sorting) خودکار: وقتی مقادیر را به شکل تاپل (`shenase_vazheh`,

`shenase_sanad`, `shomareh_file`) در تابع `heappush` قرار می‌دهیم، پایتون به‌صورت ذاتی ابتدا بر اساس

عنصر اول (**TermID**) مرتب‌سازی می‌کند. اگر **2 TermID** مساوی باشند، به‌صورت خودکار به سراغ عنصر دوم (**DocID**) رفته و آن‌ها را به‌صورت صعودی سورت می‌کند. این رفتار کاملاً با الزامات **BSBI** همخوانی دارد.

دگرپرسی `readline()`: استفاده از `readline()` (به‌جای `readlines()`) حیاتی‌ترین بخش این کد است. این متد نشانگرِ دیسک (**File Pointer**) را به‌صورت فیزیکی 1 خط جلو می‌برد و فقط همان 1 رشته‌ی متنیِ سبک را در حافظه بارگذاری می‌کند.

مدیریت تمیز منابع با `ExitStack`: باز کردن **K** فایل به‌صورت پویا با استفاده از بلاک‌های متداول `with open(...)` **as f:** در پایتون به شکل تودرتو بسیار زشت و در صورت متغیر بودن **K**، ناممکن است. ماژول `ExitStack` این کار را به شکلی کاملاً **Pythonic** هندل کرده و تضمین می‌کند که حتی در صورت بروز **Exception** یا خرابی داده‌های ورودی، تمامی فایل‌ها به‌سرعت و با اطمینان بسته شوند تا سیستم‌عامل دچار خطای «تعداد بیش از حد فایل‌های باز» (**Too many open files**) نشود.

پاسخنامه مسئله 4.40: تحلیل ریاضی و هزینه I/O در ادغام لگاریتمی (Logarithmic Merging)

بخش 1) تحلیل ساختاری مسئله (نگاشت باینری)

یکی از زیباترین ویژگی‌های الگوریتم ادغام لگاریتمی، انطباق کامل رفتار لایه‌های دیسک با «شمارنده باینری» (**Binary Counter**) است. اگر پر بودن هر لایه را با بیت 1 و خالی بودن آن را با 0 نشان دهیم، وضعیت سیستم را می‌توان به‌صورت زیر مدل‌سازی کرد:

وضعیت اولیه سیستم:

لایه‌ها به‌صورت $Z_3 Z_2 Z_1 Z_0$ چیده شده‌اند.

وضعیت طبق صورت‌مسئله: $(1011)_2$ است (لایه Z_2 خالی و بقیه پر هستند).

ارزش دهنده‌ی این عدد برابر است با:

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

این یعنی در حال حاضر مجموعاً به اندازه $11n$ (یعنی 110,000 توکن) داده روی دیسک ذخیره شده است.

محرک (Trigger): ورود $1n$ داده جدید (10,000 توکن) از طریق RAM.

وضعیت هدف: سیستم باید از حالت 11 به حالت 12 ($11 + 1 = 12$) منتقل شود. نمایش باینری عدد 12 برابر با $(1100)_2$ است. این پیش‌بینی ریاضی به ما می‌گوید که در حالت پایدار نهایی، باید لایه‌های Z_0 و Z_1 خالی شده و لایه Z_2 پر شود.

بخش الف) دنباله عملیات و فلوچارت ادغام‌ها (Cascade Merges)

هنگامی که 10,000 توکن جدید در RAM تکمیل می‌شوند، فرآیند ادغام آبخاری به صورت جریانی (Streaming) در پس‌زمینه به شکل زیر تحریک می‌شود:



بخش ب) جدول وضعیت نهایی نمایه‌ها روی دیسک

پس از پایان فرآیند ادغام آبخاری و تثبیت داده‌ها، وضعیت لایه‌های دیسک به شکل زیر بازآرایی می‌شود (مطابق با وضعیت باینری $(1100)_2$):

سطح نمایه (Z_i)	ظرفیت اسمی ($2^i \times n$)	وضعیت نهایی	حجم واقعی داده موجود
Z_0	10,000	خالی (Empty)	0 توکن
Z_1	20,000	خالی (Empty)	0 توکن
Z_2	40,000	پُر (Full)	40,000 توکن
Z_3	80,000	پُر (Full)	80,000 توکن

بخش ج) محاسبه دقیق هزینه ورودی/خروجی (I/O Cost Analysis)

در تحلیل مهندسی سیستم‌های بازیابی اطلاعات (IR)، فرض بر این است که ادغام‌های میانی به‌صورت جریانی (Stream-based) و با استفاده از بافرهای کوچک در حافظه اصلی انجام می‌شوند؛ بنابراین سیستم برای ادغام‌های موقت، داده‌ای روی دیسک نمی‌نویسد و هزینه I/O تنها شامل «خواندهای اولیه از دیسک» و «نوشتن نهایی در مقصد قطعی» است.

1. محاسبه توکن‌های خوانده‌شده از دیسک (Disk Reads)

سیستم برای انجام ادغام‌های آبشاری مجبور است داده‌های لایه‌های پُر قبلی را از دیسک فراخوانی کند:

خواندن کامل داده‌های لایه $Z_0 = 10,000$ توکن

خواندن کامل داده‌های لایه $Z_1 = 20,000$ توکن

توضیح: توکن‌های موجود در RAM از روی دیسک خوانده نمی‌شوند، زیرا از قبل در حافظه اصلی حضور دارند.

2. محاسبه توکن‌های نوشته‌شده روی دیسک (Disk Writes)

پس از تجمیع و ادغام نهایی، کل مجموعه 40,000 توکنی در لایه خالی مقصد نوشته می‌شود:

نوشتن قطعی در لایه $Z_2 = 40,000$ توکن

3. مجموع کل هزینه I/O

$$\text{Total I/O} = 30,000 \text{ (Reads)} + 40,000 \text{ (Writes)} = 70,000 \text{ Tokens}$$

نتیجه‌گیری تحلیل مهندسی:

این مسئله به‌خوبی کارایی ادغام لگاریتمی را نشان می‌دهد. برای افزودن 10,000 توکن جدید به یک نمایه 110,000 توکنی، به‌جای بازسازی کامل و نوشتن کل 120,000 توکن روی دیسک (که در روش سنتی رخ می‌دهد)، سیستم تنها با جابه‌جایی و پردازش 70,000 توکن به پایداری کامل و 100% مرتب‌شده دست یافته است. لایه Z_3 نیز که شامل بخش عظیمی از داده‌ها (80,000 توکن) بود، در طول این فرآیند کاملاً دست‌نخورده باقی ماند و هیچ هزینه پردازشی به سیستم تحمیل نکرد.

پاسخنامه مسئله 4.41: طراحی معماری نمایه‌سازی موقعیتی برای داده‌های حسگرها

بخش 1) تحلیل صورت‌مسئله

این مسئله یکی از چالش‌های بنیادین در طراحی سیستم‌های بازیابی اطلاعات (IR) پیشرفته و پایگاه‌داده‌های سری زمانی (Time-Series) را هدف قرار می‌دهد. ذخیره موقعیت دقیق زمانی (Position) برای پردازش کوئری‌های فاصله‌دار (مانند: «افت فشار خون حداکثر 5 ثانیه پس از افزایش ناگهانی دما») کاملاً ضروری است. اما این کار باعث رشد نمایی حجم لیست‌های ارجاع (Posting Lists) می‌شود. هدف ما، مدل‌سازی دقیق این ساختار در حافظه RAM و اعمال تکنیک‌های فشرده‌سازی برای جلوگیری از سرریز حافظه است.

بخش 2) پاسخ الف: ساختار داده شماتیک در حافظه RAM (بلوک SPIMI)

در الگوریتم **SPIMI**، ما داده‌ها را در قالب یک جدول درهم‌سازی (**Dictionary**) در حافظه **RAM** می‌سازیم که کلیدهای آن «توکن‌های سنسور» و مقادیر آن «لیست‌های ارجاع موقعیتی» هستند. ساختار استاندارد یک رکورد موقعیتی به این شکل است:

Term -> [[DocID_1, TF_1, [Pos_1, Pos_2, ..., Pos_TF]], ...]

```
{
  "Temp_Spike": [
    [ "D_101", 3, [10050, 10058, 10090] ]
  ],
  "BP_Drop": [
    [ "D_102", 1, [450] ]
  ]
}
```

Term: کلید واژه‌نامه (مثلاً **Temp_Spike**).

DocID: شناسه دستگاه پوشیدنی (مثلاً **D_101**).

Term Frequency (TF): تعداد دفعاتی که این رخداد در این دستگاه ثبت شده است (عدد 3).

Positions Array: آرایه‌ای از تیک‌ثانیه‌های دقیق که رخداد در آن‌ها ثبت شده است.

بخش 3) پاسخ ب: فشرده‌سازی در سطح الگوریتم (تکنیک Delta Encoding)

برای جلوگیری از انفجار حافظه **RAM**، بهترین تکنیک در سطح الگوریتم استفاده از رمزنگاری تفاضلی (**Delta Encoding**) به همراه کدگذاری بایت متغیر (**Variable Byte Encoding**) است.

منطق الگوریتم: از آنجا که رویدادهای زمانی (یا موقعیت کلمات در یک متن) همواره به صورت صعودی خوانده و ذخیره می‌شوند، نیازی نیست اعداد بزرگ و مطلق را برای هر موقعیت ذخیره کنیم. به جای آن، تنها موقعیت اولین رخداد را به صورت کامل ذخیره کرده و برای رخدادهای بعدی، تنها **اختلاف (Delta)** آن‌ها را نسبت به رخداد قبلی ثبت می‌کنیم. رابطه ریاضی این فشرده‌سازی به شکل زیر است:

$$\Delta_i = \text{Pos}_i - \text{Pos}_{i-1} \quad (\text{for } i > 1)$$

اعمال تکنیک روی رکوردهای دستگاه D_101:

آرایه موقعیت‌های خام و مطلق برای **Temp_Spike** در دستگاه **D_101** عبارتند از: **[10050, 10058, 10090]**

موقعیت اول همان مقدار خام می‌ماند: **10050**

موقعیت دوم: $10058 - 10050 = 8$

موقعیت سوم: $10090 - 10058 = 32$

آرایه جدید فشرده‌شده به این شکل در **RAM** ساخته می‌شود: **[10050, 8, 32]**

تحلیل صرفه‌جویی در بایت‌های مصرفی (Memory Savings Analysis):

برای درک میزان بهینه‌سازی، باید نحوه ذخیره‌سازی این اعداد در حافظه را بررسی کنیم. در حالت عادی، هر عدد صحیح (**Integer**) به طور استاندارد 4 بایت (32 بیت) از حافظه **RAM** را اشغال می‌کند.

حالت بدون فشردن سازی (اعداد خام): 3 عدد 10050، 10058 و 10090 هر کدام 4 بایت فضا نیاز دارند. **مجموع مصرف:** 12 بایت.

حالت با تکنیک $\Delta + \text{Variable Byte Encoding}$: در کدگذاری بایت متغیر (**VBE**)، اعدادی که کوچکتر از 128 باشند، تنها در 1 بایت ذخیره می‌شوند.

- عدد 10050: چون بزرگتر از 128 است، به 2 بایت فضا نیاز دارد.
- عدد 8 (**Delta**): در 1 بایت ذخیره می‌شود.
- عدد 32 (**Delta**): در 1 بایت ذخیره می‌شود.

مجموع مصرف: 4 بایت.

نتیجه‌گیری نهایی:

با اعمال تکنیک رمزنگاری تفاضلی در لحظه تولید بلوک **SPIMI**، فضای مصرفی برای آرایه موقعیت‌های این سنسور خاص از 12 بایت به 4 بایت کاهش یافت (حدود 66% صرفه‌جویی در حافظه).

این صرفه‌جویی عظیم به **SPIMI** اجازه می‌دهد توکن‌های بسیار بیشتری را پیش از رسیدن به آستانه حافظه (**Memory Limit**) در **RAM** پردازش کند. در نتیجه، تعداد بلوک‌های موقتی که روی دیسک نوشته می‌شوند به شدت کاهش یافته و گلوگاه هزینه فاز ادغام (**Merge Phase**) با موفقیت مهار می‌گردد.